{wait}

Summary: Pauses snippet insertion for a set period of time.

Sample Usage Copy to Text Blaze

...the notice. ⌨ **tab** 🖱 click ⏱ delay **10s** Also it is important that...

Snippet preview

...the notice. ⌨ TAB 🖱 CLICK ⏱ WAIT 10S Also it is important that...

Settings

Setting Name	Type	Description
delay	text	An optional delay period. If omitted, defaults to `1s` (1 second).
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `delay` setting uses the same syntax to specify the delay as the `shift` setting of the {time} command. Click here to learn about time shifting in Text Blaze.

The `{wait}` command is useful when constructing Autopilot scripts where you need to wait for some change to occur on the underlying page (e.g. for it to finish loading data).

Examples

Wait Example Copy to Text Blaze

⌨ **tab** ⌨ **enter** ⏱ delay **2s** ⌨ **tab**

Snippet preview

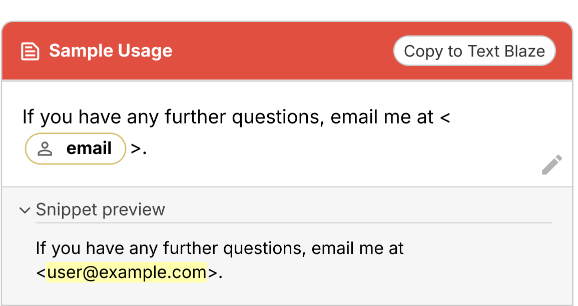
⌨ TAB ⌨ ENTER ⏱ WAIT 2S ⌨ TAB

The above example might be one where you tab to an enter button, press enter, the website loads an additional part of the form, and then you tab to the rest of the form.

To learn more about automating actions in a web page, see the **autopilot guide**.

{user}

Summary: Inserts information about the current user



If you have any further questions, email me at < email >.

Snippet preview

If you have any further questions, email me at <user@example.com>.

Settings

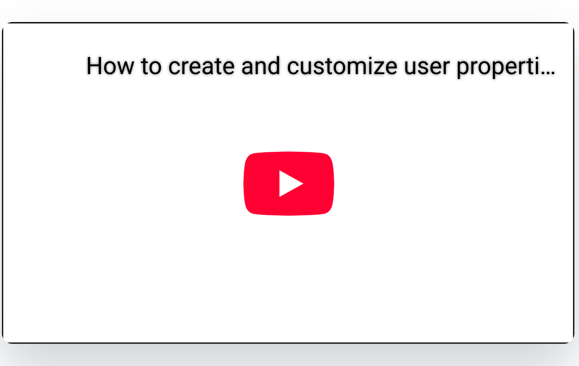
Setting Name	Type	Description
Positional	text	The user property to get
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

> The `{user}` command is a Text Blaze Business feature and **may only be used by teams using Text Blaze Business and Enterprise**.

The `{user}` command allows Text Blaze Business users to include information, known as **properties**, about the current *snippet user* in the snippet.

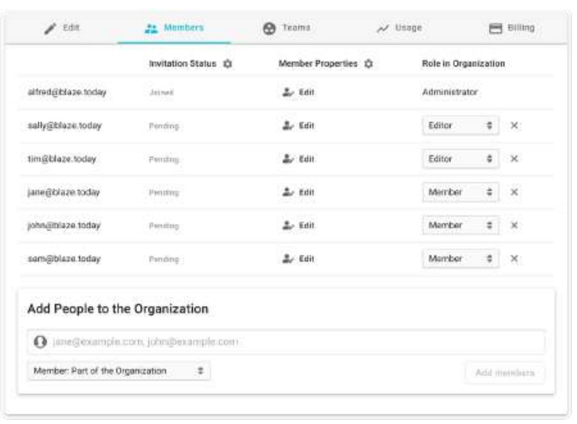
In this case, the *snippet user* is the member of your organization that's using the snippet.

Defining your own properties



You can define *custom properties* for each member of your organization.

Custom properties can be defined in the *Members* tab of the organization and clicking on the gear icon beside the *Member Properties* column header.



	Invitation Status	Member Properties	Role in Organization
alfred@blaze.today	Joined	Edit	Administrator
sally@blaze.today	Pending	Edit	Editor
tim@blaze.today	Pending	Edit	Editor
jane@blaze.today	Pending	Edit	Member
john@blaze.today	Pending	Edit	Member
sam@blaze.today	Pending	Edit	Member

Add People to the Organization

jane@example.com, john@example.com

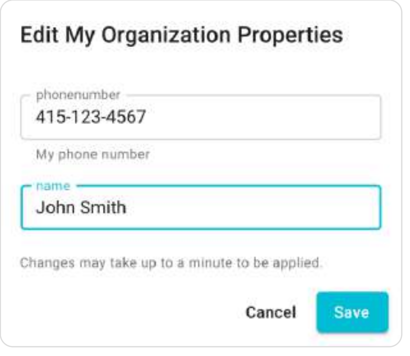
Member: Part of the Organization

Add members

For example, you may want to include employees' names in your organization's snippets. You can do that by first creating a custom `name` property.

Now you can configure the newly defined `name` property for each employee individually by clicking on the `Edit` button next to their name under the *Member Propeties* column.

Here is an example of configuring two properties for a member of your organization.



Edit My Organization Properties

phonenumber: 415-123-4567

My phone number

name: John Smith

Changes may take up to a minute to be applied.

Editing Properties

Organization administrators and editors **can modify properties for all organization members** from the Members tab.

If the property is *user editable*, individual members can modify their own properties.

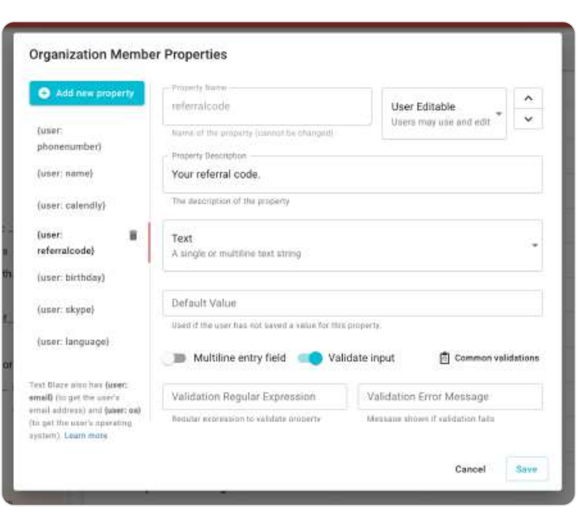
Setting a property's name and description

The property name you use is how you will reference that property in your snippets. If your property name is `department`, you reference the value in a snippet with `{user: department}`.

The capitalization that you use is important. For example, `{user: department}` will work, but `{user: Department}` will not.

Text Blaze **does not allow you to change a property's name** once set initially.

You can also optionally add a description that allows you to provide additional information to users.



Organization Member Properties

{user: phonenumber}
{user: name}
{user: calendly}
{user: referralcode}
{user: birthday}
{user: skype}
{user: language}

Text Blaze also has **{user: email}** (to get the user's email address) and **{user: os}** (to get the user's operating system). Learn more

Property Name: referralcode

Name of the property (cannot be changed)

User Editable: Users may use and edit

Property Description: Your referral code.

The description of the property

Text: A single or multiline text string

Default Value

Used if the user has not saved a value for this property.

Multiline entry field

Validate input

Validation Regular Expression

Regular expression to validate property

Validation Error Message

Message shown if validation fails

Setting a property's data type and default value

The default value that you set for the property is used if an administrator, editor, or individual member has not yet configured a specific value for the property.

You can also configure the **data type** of a property.:

There are **four** data types available:

1. **Text**: Text properties can be used to represent things like names or phone numbers. You can also specify validation rules using regular expressions.
2. **Number**: Number properties only accept numeric values. You can configure maximum and minimum allowed values.
3. **Boolean**: Boolean properties are yes/no values that can be turned on and off.
4. **Categorical**: Categorical properties allow you to specify a drop-down list of options that the user may select an option from.
5. **Dates**: Date properties that allow you to select a specific date from a date field.

> **If the data type of an existing property is changed, or new validations are added to it, *existing values* for it will not be changed even if they are no longer valid**.
>
> The newly changed data type and validations will only apply when manually updated the values.

Built-in properties

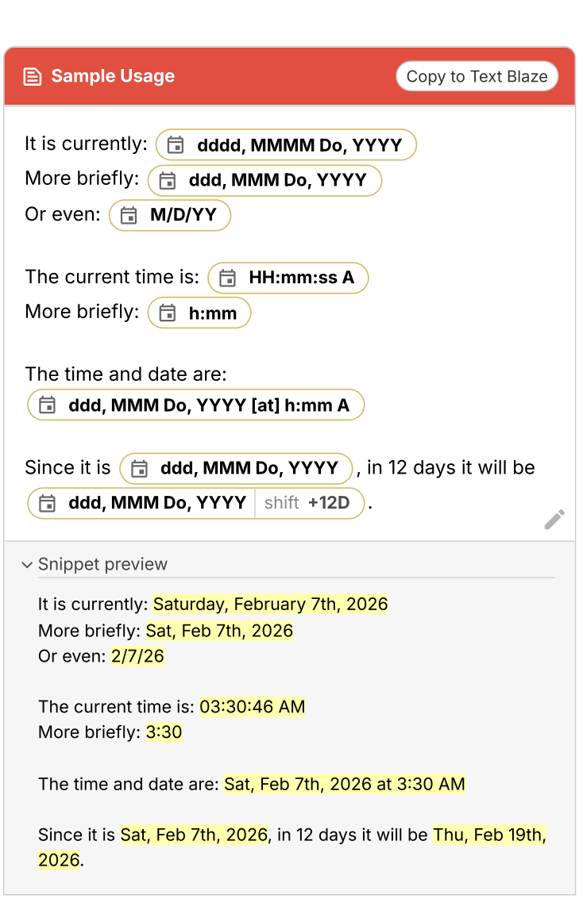
Besides allowing you to define your own custom properties, each member of your organization will come with built-in properties by default.

Property	Description	Example
email	The user's email.	`user@example.com`

Using `{user: email}` outputs the email that the user signed up to Text Blaze to join your organization with.

{time}

Summary: Powerful and configurable time and date formatting.


Sample Usage Copy to Text Blaze

It is currently: dddd, MMMM Do, YYYY
More briefly: ddd, MMM Do, YYYY
Or even: M/D/YY

The current time is: HH:mm:ss A
More briefly: h:mm

The time and date are:
ddd, MMM Do, YYYY [at] h:mm A

Since it is ddd, MMM Do, YYYY, in 12 days it will be ddd, MMM Do, YYYY shift +12D.

Snippet preview

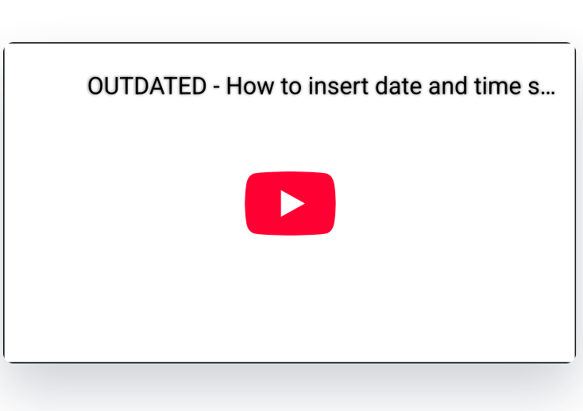
It is currently: Saturday, February 7th, 2026
More briefly: Sat, Feb 7th, 2026
Or even: 2/7/26

The current time is: 03:30:46 AM
More briefly: 3:30

The time and date are: Sat, Feb 7th, 2026 at 3:30 AM

Since it is Sat, Feb 7th, 2026, in 12 days it will be Thu, Feb 19th, 2026.

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	text	The date/time format.
at	text	Set's the date/time.
pattern	text	Format used to interpret `at`.
shift	text	An optional date/time shift.
locale	text	An optional locale code (e.g. `en-us` or just `en`) used to format the time and date.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Time/Date Formatting

Text Blaze's time/date formatting syntax **allows you to customize the output of your times and dates in various ways**. It allows you to mix formatting with *tokens* that have special meanings.

For example, one way the current date could be entered is `MMM Do, YYYY` (resulting in the output of `Nov 13th, 2017`), and the current time could be specified with `h:mm` (resulting in the output of `11:24`).

Text Blaze's time/date formatting syntax is based on Moment, a JavaScript library for parsing and formatting dates.

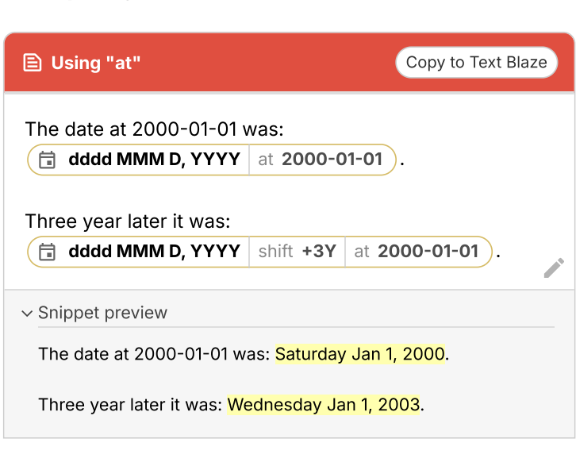
The following table (adapted from the Moment.js documentation website) contains the full list of support tokens:

Token	Output
Month	
M	1 2 ... 11 12
Mo	1st 2nd ... 11th 12th
MM	01 02 ... 11 12
MMM	Jan Feb ... Nov Dec
MMMM	January February ... November December
Quarter	
Q	1 2 3 4
Qo	1st 2nd 3rd 4th
Day of Month	
D	1 2 ... 30 31
Do	1st 2nd ... 30th 31st
DD	01 02 ... 30 31
Day of Year	
DDD	1 2 ... 364 365
DDDo	1st 2nd ... 364th 365th
DDDD	001 002 ... 364 365
Day of Week	
d	0 1 ... 5 6
do	0th 1st ... 5th 6th
dd	Su Mo ... Fr Sa
ddd	Sun Mon ... Fri Sat
dddd	Sunday Monday ... Friday Saturday
Day of Week (Locale)	
e	0 1 ... 5 6
Day of Week (ISO)	
E	1 2 ... 6 7
Week of Year	
w	1 2 ... 52 53
wo	1st 2nd ... 52nd 53rd
ww	01 02 ... 52 53
Week of Year (ISO)	
W	1 2 ... 52 53
Wo	1st 2nd ... 52nd 53rd
WW	01 02 ... 52 53
Year	
YY	70 71 ... 29 30
YYYY	1970 1971 ... 2029 2030
AM/PM	
A	AM PM
a	am pm
Hour	
H	0 1 ... 22 23
HH	00 01 ... 22 23
h	1 2 ... 11 12
hh	01 02 ... 11 12
k	1 2 ... 23 24
kk	01 02 ... 23 24
Minute	
m	0 1 ... 58 59
mm	00 01 ... 58 59
Second	
s	0 1 ... 58 59
ss	00 01 ... 58 59
Locale Formatted	
LT	*Ex:* 7:10 AM
LTS	*Ex:* 7:10:46 AM
l	*Ex:* 9/22/1985
L	*Ex:* 09/22/1985
ll	*Ex:* Sept 22, 1985
LL	*Ex:* September 22, 1985
lll	*Ex:* Sept 22, 1985 7:10 AM
LLL	*Ex:* September 22, 1985 7:10 AM
llll	*Ex:* Mon, Sept 22, 1985 7:10 AM
LLLL	*Ex:* Monday, September 22, 1985 7:10 AM
Unix Timestamp	
X	1360013296
Unix Millisecond Timestamp	
x	1360013296123

> The Text Blaze time/date formatting syntax above is also used in the {formdate} command.

Setting the Time/Date

The optional `at` setting allows you to specify the time/date used by the time command. **If the `at` setting is not explicitly set, the current time/date will be used.**


Using "at" Copy to Text Blaze

The date at 2000-01-01 was:
dddd MMM D, YYYY at 2000-01-01.

Three year later it was:
dddd MMM D, YYYY shift +3Y at 2000-01-01.

Snippet preview

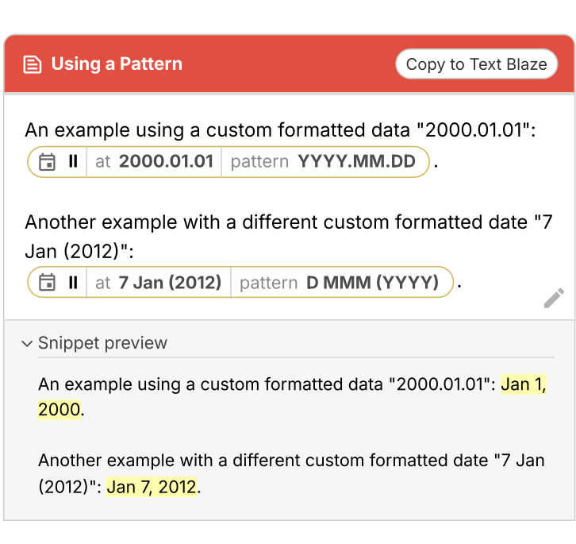
The date at 2000-01-01 was: Saturday Jan 1, 2000.

Three year later it was: Wednesday Jan 1, 2003.

The `at` setting will interpret many common date formats. If the `at` format does not recognize your date format, or your date format is ambiguous, or you want to specify a time; you can use the `pattern` setting to tell Text Blaze how to understand your date.

The `pattern` setting uses the Text Blaze time/date formatting syntax to tell Text Blaze how your time/date is structured.

For example, if your date was 2000-01-01 where you have Year-Month-Day, you would use a pattern of `YYYY-MM-DD`. If your date was instead Year-Day-Month, you could use the pattern `YYYY-DD-MM`.


Using a Pattern Copy to Text Blaze

An example using a custom formatted data "2000.01.01":
ll at 2000.01.01 pattern YYYY.MM.DD.

Another example with a different custom formatted date "7 Jan (2012)":
ll at 7 Jan (2012) pattern D MMM (YYYY).

Snippet preview

An example using a custom formatted data "2000.01.01": Jan 1, 2000.

Another example with a different custom formatted date "7 Jan (2012)": Jan 7, 2012.

Time/Date Shifting

The optional `shift` setting lets you display a time/date in either the future or past. A time shift is specified to select a time period relative to the current time.

For example, to show the date 10 days from now, you could use `{time: MMM Do, YYYY; shift=+10D}`.

If the `at` setting is specified, any time shifts will be relative to that date.

Shifts are specified in terms of a base period like days or weeks. You can have a fixed shift like `+2W` (two weeks from the current time) or `-5D` (five days ago). You can also shift to the beginning or end of any standard date period. For example, `<M` will shift to the start of the current month, while `>Y` will shift to the end of the current year.

The following table summarizes the shifts relative to the standard periods.

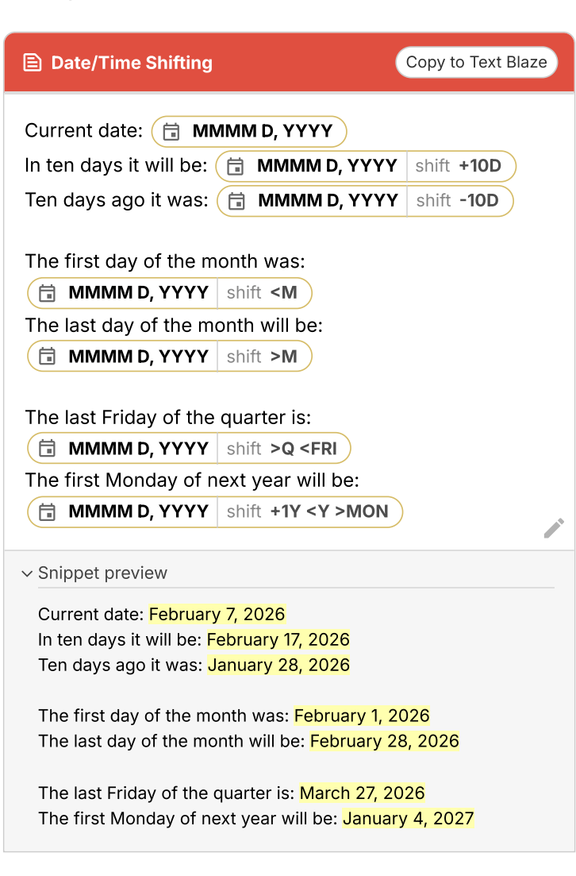
Period	Base Period	Example Fixed Shift	Start and End Period Shift
Seconds	s	`5s`	`<s` and `>s`
Minutes	m	`-4m`	`<m` and `>m`
Hours	h	`24h`	`<h` and `>h`
Days	D	`2D`	`<D` and `>D`
Weeks	W	`5W`	`<W` and `>W`
Months	M	`-10M`	`<M` and `>M`
Quarters	Q	`1Q`	`<Q` and `>Q`
Years	Y	`3Y`	`<Y` and `>Y`

You can also shift forward or backward to a specific day of the week. For example, `>MON` will shift forward to the nearest Monday. If the current date is a Monday, no shift will take place. `<MON` will shift backwards to the nearest Monday. Again, if the current date is a Monday, no shift will take place.

Day	Label	Current or Previous Shift	Current or Following Shift
Sunday	SUN	`<SUN`	`>SUN`
Monday	MON	`<MON`	`>MON`
Tuesday	TUE	`<TUE`	`>TUE`
Wednesday	WED	`<WED`	`>WED`
Thursday	THU	`<THU`	`>THU`
Friday	FRI	`<FRI`	`>FRI`
Saturday	SAT	`<SAT`	`>SAT`

> **You can combine multiple shifts together**. For example, `+10Y -2D` would shift the date by a total of 10 years minus 2 days. `+10D >MON` would shift the date 10 days and then to the following Monday.
>
> This is particularly useful if you want to implement something like "find the next Monday". If you just used `>MON` and the current day is a Monday, it will not shift at all. You instead could use `+1D >MON` to increment the day by one, and then move forward to the nearest Monday.

The following examples show how you might use date shifting to specify specific dates.


Date/Time Shifting Copy to Text Blaze

Current date: MMMM D, YYYY
In ten days it will be: MMMM D, YYYY shift +10D
Ten days ago it was: MMMM D, YYYY shift -10D

The first day of the month was:
MMMM D, YYYY shift <M
The last day of the month will be:
MMMM D, YYYY shift >M

The last Friday of the quarter is:
MMMM D, YYYY shift >Q <FRI
The first Monday of next year will be:
MMMM D, YYYY shift +1Y <Y >MON

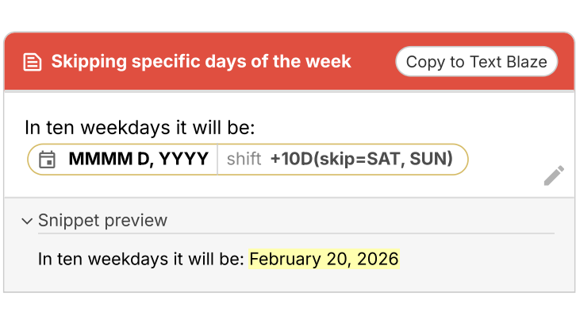
Snippet preview

Current date: February 7, 2026
In ten days it will be: February 17, 2026
Ten days ago it was: January 28, 2026

The first day of the month was: February 1, 2026
The last day of the month will be: February 28, 2026

The last Friday of the quarter is: March 27, 2026
The first Monday of next year will be: January 4, 2027

When shifting by days, you can skip certain days of the week. For example, if you wanted to shift 10 weekdays into the future you could use the following:


Skipping specific days of the week Copy to Text Blaze

In ten weekdays it will be:
MMMM D, YYYY shift +10D(skip=SAT, SUN)

Snippet preview

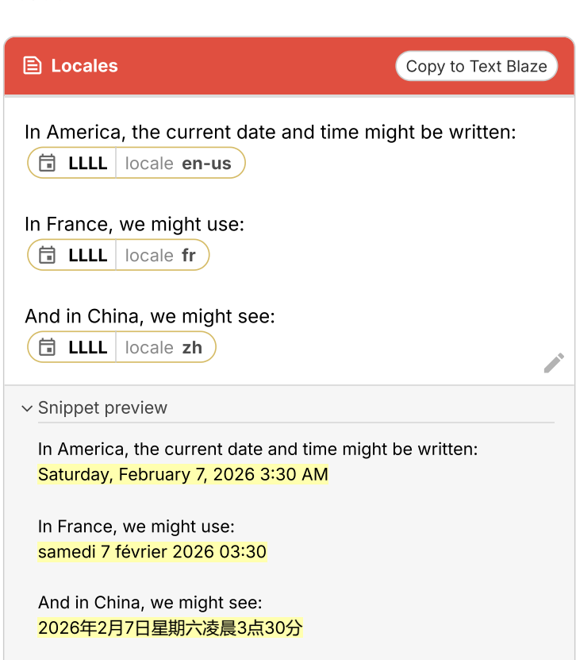
In ten weekdays it will be: February 20, 2026

Locales and Localization

It's possible to override the locale for individual `{time}` commands using the `locale` setting.

If not specified, `{time}` command will attempt to use the locale specified in your Text Blaze account settings when formatting the time/date.

A list of available language locales are available in your Account Settings page under the **Locale & Formatting** header.


Locales Copy to Text Blaze

In America, the current date and time might be written:
LLLL locale en-us

In France, we might use:
LLLL locale fr

And in China, we might see:
LLLL locale zh

Snippet preview

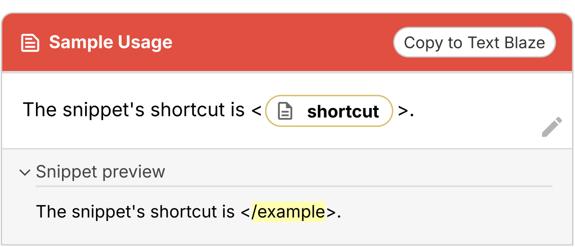
In America, the current date and time might be written:
Saturday, February 7, 2026 3:30 AM

In France, we might use:
samedi 7 février 2026 03:30

And in China, we might see:
2026年2月7日星期六凌晨3点30分

{snippet}

Summary: Inserts information about the current snippet in the snippet itself



Sample Usage Copy to Text Blaze

The snippet's shortcut is <**shortcut**>.

Snippet preview

The snippet's shortcut is </example>.

Settings

Setting Name	Type	Description
Positional	text	The snippet property to get
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{snippet}` command allows you to **include information about the snippet itself** within the snippet.

If you're looking to **import** another snippet into your current snippet, that would be the {import} command.

The `{snippet}` command takes a single positional setting that supports **three** different values.

Property	Description	Example
id	The snippet's id.	`xyz987`
shortcut	The snippet's shortcut.	`/example`
trigger	The snippet's shortcut as it was typed to trigger the insertion.	`/Example`

Capitalization is important. Each property must be typed **in lowercase** into the command. For example, `{snippet: id}` will work, but `{snippet: ID}` will not.

> The `id` is a unique identifier for the snippet set automatically when the snippet is created. It will not change even if the snippet's name or shortcut is changed.

Using the {snippet} command and the {import} command together

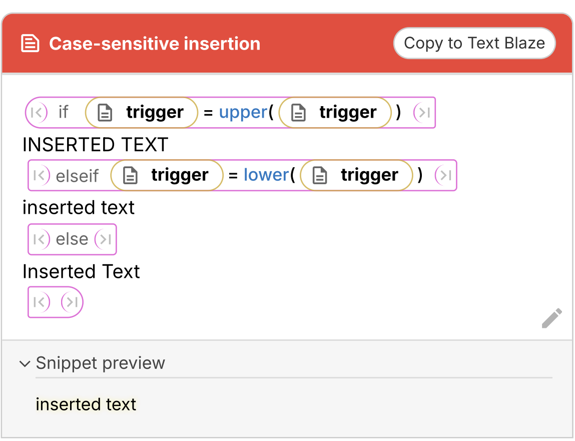
The `{snippet}` command will always refer to the snippet that was initially triggered by the user **even if the command is nested within an imported snippet**.

This means that if the `{snippet}` command is used within an {import} command, the `{snippet}` command will still refer to the initial snippet that was triggered, not the snippet being imported with the {import} command.

Case-sensitive insertion

`{snippet: trigger}` will return the shortcut exactly as it was typed including capitalization. This can be used to implement case-sensitive replacements.

For example, the following will insert the text "inserted text" while matching the capitalization of the shortcut. For example, if the shortcut was "/go" and you typed "/GO" it would insert "INSERTED TEXT". On the other hand, if you typed "/Go", it would insert "Inserted Text".



Case-sensitive insertion Copy to Text Blaze

if **trigger** = upper(**trigger**)
INSERTED TEXT
elseif **trigger** = lower(**trigger**)
inserted text
else
Inserted Text

Snippet preview

inserted text

See the {if} command for more information on using conditional logic in your snippets.

`upper()` is a predefined function available to you in Text Blaze. See the Blaze Formula Language Reference for more information on the various predefined functions available to you.

{site}

Summary: Inserts information from any open webpage into your snippet.

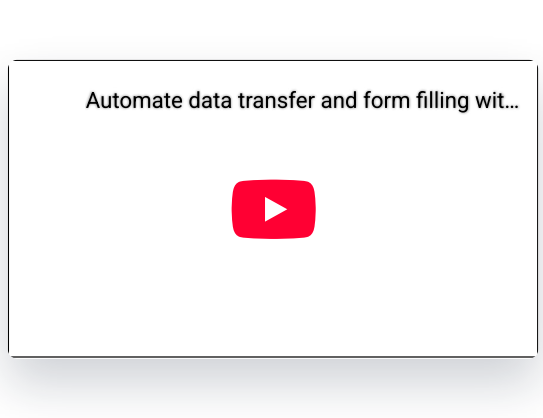
Sample Usage Copy to Text Blaze

Page URL: url
Page Domain: domain
Page Path: path
Page Title: title

Snippet preview

Page URL: https://blaze.today/commands/site/
Page Domain: blaze.today
Page Path: /commands/site/
Page Title: Text Blaze | {site}

Quick Overview Video


Automate data transfer and form filling wit...

Settings

Setting Name	Type	Description
Positional	text	The type of page information to get.
frame	self/top	Which frame to get information from (defaults to `top`).
selector	text	A CSS selector for use with the `html` or `text` types.
multiple	yes/no	When `selector` is specified, whether to return multiple results or just the first result (defaults to `no`).
page	string	When `page` is specified, the {site} command will read data from tabs whose URLs match the given `page` URL.
select	yes/no/ifneeded	When `page` is specified, this setting controls whether to show the tab selector. `yes` will always show the tab selector. `no` does not show the tab selector and only matches the current page. `ifneeded` (default) shows the tab selector only when multiple tabs are matching the given `page` pattern.
group	string	Allows different `{site}` commands with the same `page` setting to read data from different tabs.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Getting information from a web page is extremely useful in further automating your snippets.

For example, if you're using Text Blaze on your chat application, you could set it up to get specific information, like a name, email, or something else, present on the web page to use in your snippet

Throughout this documentation page, you will get several examples to help you see the `{site}` command's usefulness.

> `{site}` is an advanced Text Blaze command that assumes some familiarity with fundamental web concepts like HTML, CSS, URLs, and Regular Expressions.

Getting specific information from a web page

The first setting supported by the `{site}` command is a positional setting that tells the command the type of information to get from the web page.

The table below displays the different supported values that this command takes:

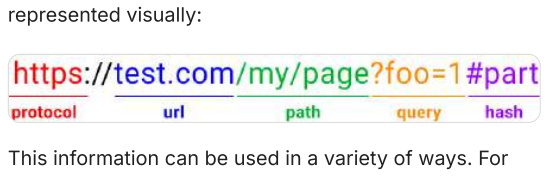
Type	Description	Example for url `https://test.com/my/page?foo=1#part`
url	The full url of the page.	`https://test.com/my/page?foo=1#part`
domain	The domain of the url.	`test.com`
path	The path of the url.	`/my/page`
protocol	The protocol of the url.	`https`
query	The query of the url.	`?foo=1`
hash	The hash of the url.	`#part`
html	The html of the webpage.	`<html>...</html>`
text	The text of the webpage.	`Welcome to...`
title	The title of the webpage.	`Site Title`
fieldtext	The text of the focused textfield.	`Hi there hello!`
selection	The currently selected text.	`there`
fieldleft	The text to the left of the cursor in the focused textfield.	`Hi`
fieldright	The text to the right of the cursor in the focused textfield.	`hello!`

Capitalization is important. For example, you can use `{site: url}` to get the web page's URL, but doing `{site: URL}` will not work.

Getting Information from the URL

The `url`, `domain`, `path`, `protocol`, `query`, and `hash` values are all concerned with getting information the **URL** of the current web page.

Here are the properties you can retrieve information for represented visually:



This information can be used in a variety of ways. For example, you could combine it with the {if} command to only insert information if you're on a specific part of the web page.

Including the Page Contents in your Snippet

The next two values that you can use in the `{site}` command's positional setting are `text` and `html`.

These two are not related to the URL, but instead get contents of the web page.

You can use them to dynamically populate parts of your snippets.

For example, if you use a Text Blaze snippet within your email application, you could use it to include information about the email recipient's name in your snippet.

To illustrate the usage of `text` and `html` in the `{site}` command, we're going to consider some HTML typical of what you might see on a site:

> The `text` and `html` types are both in **Beta** and their functionality may change in the future.
>
> In particular, we are exploring possibilities to include nested `<iframe>` content in each of them and the possibility of including nested `<input>` and `<textarea>` elements directly in the `text` type.

Sample HTML:

```
<div class="user-info">
  Name: <span class="name">John Smith</span>
  <ul class="favorites">
    <li>Oreos</li>
    <li>Twizzlers</li>
    <li>Snickers</li>
  </ul>
</div>
```

How the above sample HTML would render in a web page:

User Name: John Smith

- Oreos
- Twizzlers
- Snickers

We can use the `{site}` command to extract information from this sample. `{site: text}` and `{site: html}` will give you access to all the contents of the page. For example:

Page Contents Copy to Text Blaze

HTML for whole page (truncated for readability):
= substring(html , 1, 50) ...

Text for whole page (truncated for readability):
= substring(text , 1, 50) ...

Snippet preview

HTML for whole page (truncated for readability):
<html class="js" lang="en-us"><head><style data-em...

Text for whole page (truncated for readability):
Text Blaze
Product
Work
Resources
Pricing
ADD TO C...

> The above example is using the `{site}` command on the Text Blaze website so the contents will be related to this web page.

Also, one very important thing to note is that the `text` type pulls text content from the page. **This does not include contents inside of `<input>` or `<textarea>` HTML tags**. To include the contents of these elements, you need to specify a selector that specifically selects one of these elements and not a parent element. We'll review selectors in the next section.

Getting specific parts of a page

Using `{site: html}` or `{site: text}` will give you the entire contents of the page.

Generally, though you just want to get a specific part of the contents of the page, like a name or address. Text Blaze has two ways to do that: CSS Selectors and Regular Expressions.

If you do use `html` or `text` as the value for the `{site}` command's positional setting you have access to 2 additional named settings.

Those named settings are `selector` and `multiple`. **These 2 named settings are concerned with *how* to get the contents of the page**.

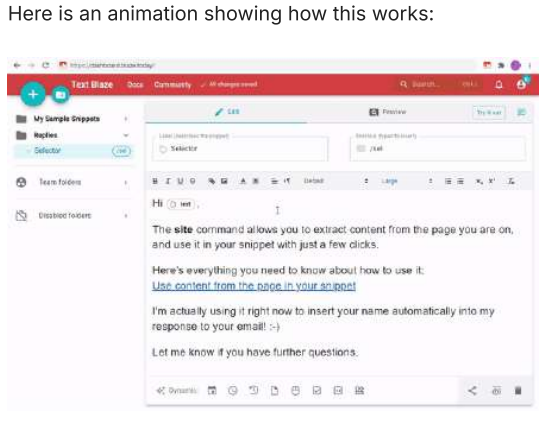
Using CSS Selectors to Get Part of the Page

One way that Text Blaze can access specific parts of the page is with CSS selectors. CSS selectors provide a very concise method of getting specific parts of a page.

The details of CSS selectors are out of the scope of this documentation, and we refer you to this website to understand them in more detail.

Fortunately, for most use cases, Text Blaze can automatically create the right selector for you with point and click. Just use the "Select from website" feature of Text Blaze and click on the desired part of the page.

Here is an animation showing how this works:



Once you have your selector, the snippet automatically populates its contents when used. For example, in the html sample from the previous section, you can get the name using a CSS selector. You can also use another CSS selector to get different parts of the favorites list.

Using CSS Selectors Copy to Text Blaze

Using a CSS selector to get the user's name:
text selector .user-info .name

Using a CSS selector to get the first favorite item:
text selector .user-info .favorites li

Snippet preview

Using a CSS selector to get the user's name:
John Smith

Using a CSS selector to get the first favorite item:
Oreos

Reading information from any tab

The {site} command supports pulling information from any tab in Chrome and inserting it into your current tab.

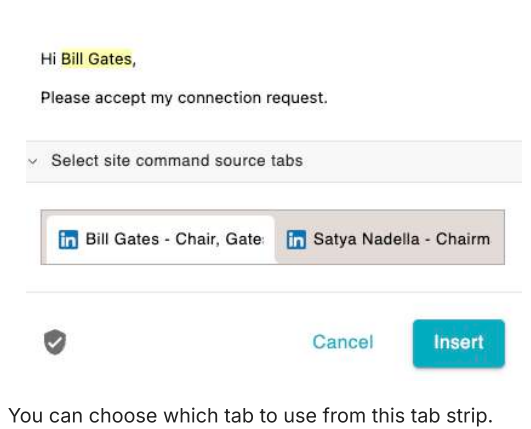
For example, when writing an email to a client, you can:

1. Use your snippet in the email webpage
2. Use {site} to pull client information from your CRM open in a different Chrome tab

This is the default behavior in Text Blaze and requires no additional setup.

Text Blaze matches your open Chrome tabs using the wildcard pattern in the `page` setting. For example, setting `page` to `https://blaze.today/commands/*` will match this page but not `https://blaze.today/docs/`

When more than one Chrome tab matches the `page` setting, Text Blaze shows a tab strip in your form window:


Hi Bill Gates,

Please accept my connection request.

Select site command source tabs

You can choose which tab to use from this tab strip.

Controlling the tab selector

The `select` setting controls when this tab strip appears. It can have one of these three values:

1. `yes`: Always shows the tab selector, even with just one matching tab.
2. `no`: Never shows the tab selector. The {site} command will show an error if the `page` pattern doesn't match the current page.
3. `ifneeded` (default): Only shows the tab selector when multiple tabs match.

By default, all {site} commands with the same `page` setting will pull data from the same tab. For example, `{site: text; page=https://blaze.today/*}` and `{site: url; page=https://blaze.today/*}` will always use the same Chrome tab.

You can use the `group` setting to pull data from different tabs even with the same `page` pattern. For example, the following two commands `{site: text; page=https://blaze.today/*; group=First item}` and `{site: text; page=https://blaze.today/*; group=Second item}` can pull data from two different Chrome tabs (both matching the same `page` pattern).

Getting more than one element with the `multiple` setting

Notice how in the above CSS selectors example that using `.user-info .favorites li` gets the first item in the favorites list even though multiple elements have the same CSS selector.

There is where the named setting `multiple` comes in. If `multiple` is `no` or omitted only the first match will be returned.

If `multiple` is `yes` all the matches will be returned as a list.

Using the multiple setting Copy to Text Blaze

Using a CSS selector to get all the favorite items as a list:
text selector .user-info .favorites li multiple yes

Snippet preview

Using a CSS selector to get all the favorite items as a list:
["Oreos", "Twizzlers", "Snickers"]

Handling Errors

If `multiple` is `no` and there is no match for that specific CSS selector on the web page you're using the snippet on, **an error will be displayed**.

Text Blaze provides you with `catch()`, a predefined function to help you deal with an error scenario. You can use it to provide a default value in an error case in your formulas:

CSS Selector Error Handling Copy to Text Blaze

Error shown:
text selector .does-not-exist

Error caught and replaced with another value:
= catch(text selector .does-not-exist , "My Defau

Snippet preview

Error shown:
No match found for selector: ".does-not-exist"

Error caught and replaced with another value:
My Default Value

If `multiple` is set to `yes`, on the other hand, and there is no match, Text Blaze will return an empty list (`[]`).

Controlling where information is loaded from using the `frame` setting

Web pages can embed other web pages (called 'frames'). If you use a Text Blaze snippet inside of a frame, you can use the `frame` named setting to load information either the frame (by setting the `frame` setting to `self`) or from the web page embedding the frame (by setting the `frame` setting to `top`).

> Using the `frame` setting affects the output of both domains and also the page contents.

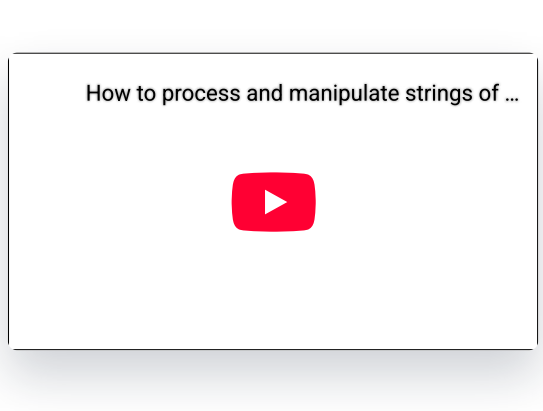
If Text Blaze is used in a normal web page without frames, both `self` and `top` are equivalent. Text Blaze defaults to `top`. This will return the URL you see in Chrome's URL bar.

Using Regular Expressions to Get Part of the Page

Regular expressions are a powerful way to search through text and find specific patterns of text.

For example, if you're looking for a phone number or an email address (both pieces of textual content that follow a specific pattern), then regular expressions can help you find those.

The following video will help you further understand how Regular Expressions work.


How to process and manipulate strings of ...

Text Blaze has an `extractregex()` predefined function that takes two string parameters.

If you use `{site: text}` or `{site: html}`, then you can use Regular Expressions as an alternative to CSS selectors to retrieve a specific part of the web page.

For example, if you want to get the name from the html sample in the previous section, you can use the following regular expression: "`Name: ([\w ]+)`".

This tells Text Blaze, that we want all the text after the text "Name: " that consists of spaces or letter characters.

Using Regular Expressions Copy to Text Blaze

Using a regular expression to get just the user's name.

Name from the page:
= extractregex(text , "Name: ([\w]+)")

Snippet preview

Using a regular expression to get just the user's name.

Name from the page: John Smith

You can read more about regular expressions in our Blaze Formula Language Reference, reading our non-scary guide on the community forums, and watching the above video. Also, there are some great online resources you can use to learn more about them.

Reading data based on user selection in a page

The `selector` and `xpath` settings allow you to read data from a fixed element. You can also instead read data from the page based on the active user selection. For example: assume that you have a textbox with this text "Hi there hello!" (without quotes). Now, you have selected the word "there" with the mouse. Now, you can check the following four `{site}` commands:

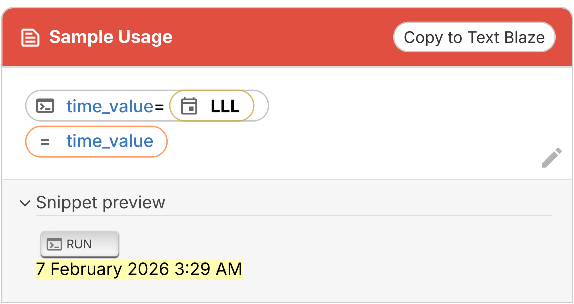
1. `{site: selection}` - outputs the currently selected text, whether it is inside or outside a textbox. In this case, it outputs "`there`" (without quotes).
2. `{site: fieldtext}` - outputs the text of the focused textfield. In this case, it outputs "`Hi there hello!`" (without quotes).
3. `{site: fieldleft}` - outputs the text to the left of the cursor in the focused textfield. In this case, it outputs "`Hi `" (without quotes).
4. `{site: fieldright}` - outputs text to the right of the cursor in the focused textfield. In this case, it outputs "` hello!`" (without quotes).

As the names suggest, `{site: selection}` works whether or not a textfield is focused, whereas the other three commands output blank if no textfield is focused.

Note that all four commands can be combined with the `page` setting to read data from another webpage in another browser tab.

{run}

Summary: Runs a code block once.



Sample Usage
Copy to Text Blaze

time_value= LLL
= time_value

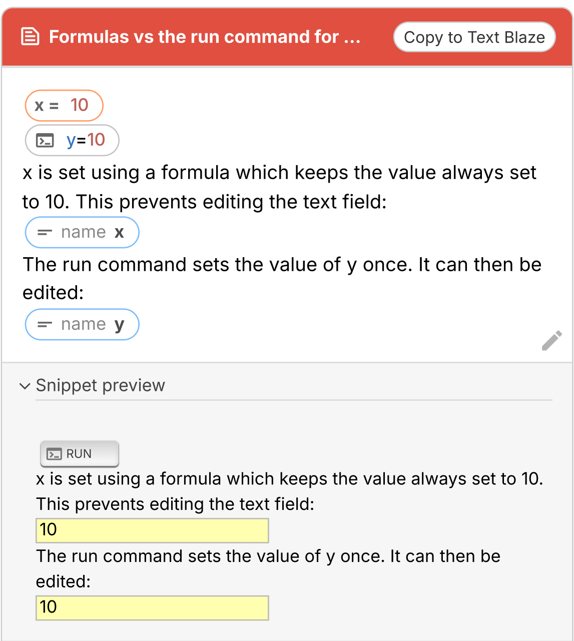
Snippet preview
RUN

7 February 2026 3:29 AM

Settings

Setting Name	Type	Description
Positional	code	The code block to run once.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

`{run}` executes a code block once. It is particularly useful for initializing form variables that you want to manually change later. For example



Formulas vs the run command for ...
Copy to Text Blaze

x = 10
y=10
x is set using a formula which keeps the value always set to 10. This prevents editing the text field:
= name x
The run command sets the value of y once. It can then be edited:
= name y

Snippet preview
RUN

x is set using a formula which keeps the value always set to 10. This prevents editing the text field:
10
The run command sets the value of y once. It can then be edited:
10

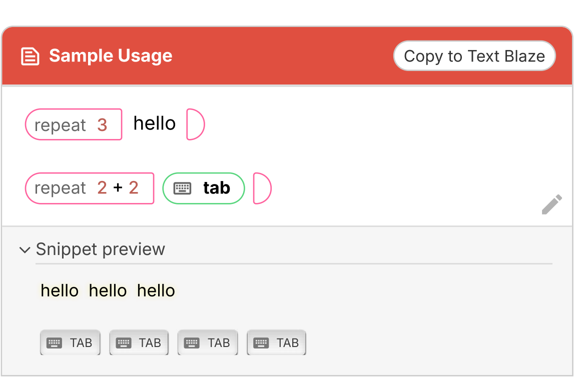
The `{run}` command is often used with the {button} to make interactive interfaces.

Learn more about code blocks in the code block reference.

> Run commands are in Beta and may change in the future.

{repeat}

Summary: Repeat a portion of a snippet multiple times.



repeat 3 hello

repeat 2 + 2 tab

Snippet preview

hello hello hello

Settings

Setting Name	Type	Description
Positional	formula	The number of times to repeat or a list comprehension.
locals	text	Optional identifier providing a list of form data in the command.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{repeat}` command replicates a portion of a snippet multiple times. It repeats everything that is between the `{repeat}` and `{endrepeat}` commands.

This command takes a formula as the first positional setting that *must* evaluate to one of two things:

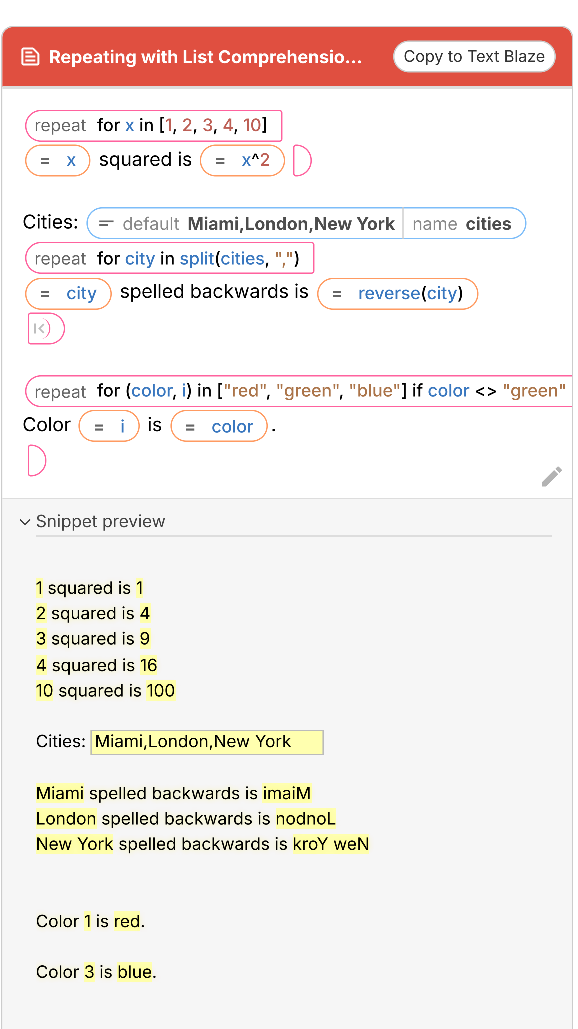
1. A Number
2. A List Comprehension

Also, see the {if} command for an additional command that helps you build dynamic snippets.

Repeating with List Comprehensions

List Comprehensions are a special formula syntax in Text Blaze that allow you to transform lists.

As the first positional setting in the `{repeat}` command instead of placing a number you place a list comprehension.



repeat for x in [1, 2, 3, 4, 10]
= x squared is = x^2

Cities: = default Miami,London,New York name cities
repeat for city in split(cities, ",")
= city spelled backwards is = reverse(city)

repeat for (color, i) in ["red", "green", "blue"] if color <> "green"
Color = i is = color .

Snippet preview

1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
10 squared is 100

Cities: Miami,London,New York

Miami spelled backwards is imaiM
London spelled backwards is nodnoL
New York spelled backwards is kroY weN

Color 1 is red.

Color 3 is blue.

- Notice how in the second example above, the output of the {formtext} is converted into a list by using the `split()` predefined function.
- In the final example you'll notice an `if` statement next to the list comprehension. This is the syntax you can use to filter your lists. Visit the Blaze Formula Reference to learn more.

Variable Scoping in the `{repeat}` command

This section assumes that you're already familiar with how variables work in Text Blaze.

Visit the Blaze Formula Reference to familiarize yourself with variables before proceeding.

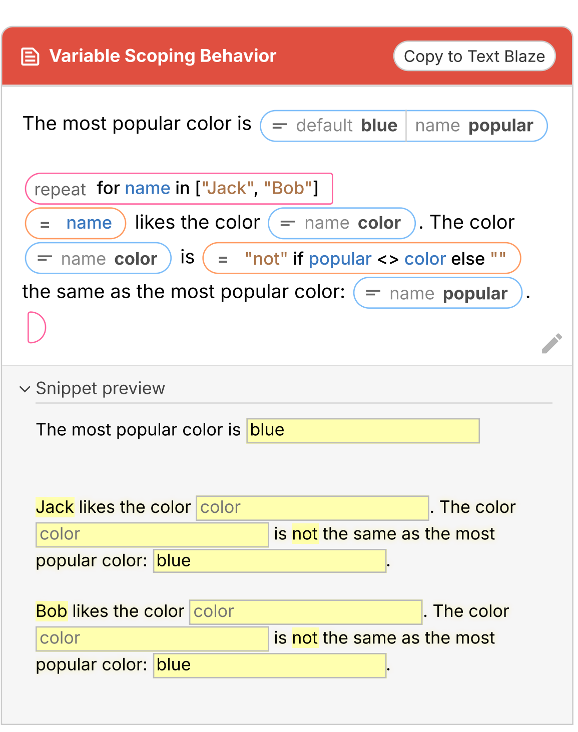
When using the list comprehensions with the `{repeat}` command, a *separate variable scope* gets created.

This means that the value of variables within a `{repeat}` may differ from variable values outside the repeat.

This is called *variable scoping* and the following rules summarize how this works:

1. The value of variables in the repeat iterator (e.g. `x` in the iterator `for x in [1, 2, 3]`) are only available inside of the command, between the `{repeat}` and `{endrepeat}` commands. If the variables are defined outside the block (e.g. you have a `{formtext: name=x}` command outside the block), then the values will not be changed outside the block but will be overridden within the block.
2. If a variable is defined outside the repeat block, it can be used in formulas inside the block. Form fields that have the same name inside the block will all use and change the same global value.
3. If a variable is not defined outside the repeat block, any form fields inside the block will each refer to a unique instance of that variable for each iteration of the `{repeat}` command.

The following example illustrates this scoping behavior:



The most popular color is = default blue name popular

repeat for name in ["Jack", "Bob"]
= name likes the color = name color . The color = name color is = "not" if popular <> color else "" the same as the most popular color: = name popular .

Snippet preview

The most popular color is blue

Jack likes the color color . The color color is not the same as the most popular color: blue .

Bob likes the color color . The color color is not the same as the most popular color: blue .

- Notice how all instances of the `{formtext: name=color}` are not all linked because they are scoped to each instance of the {repeat} command.

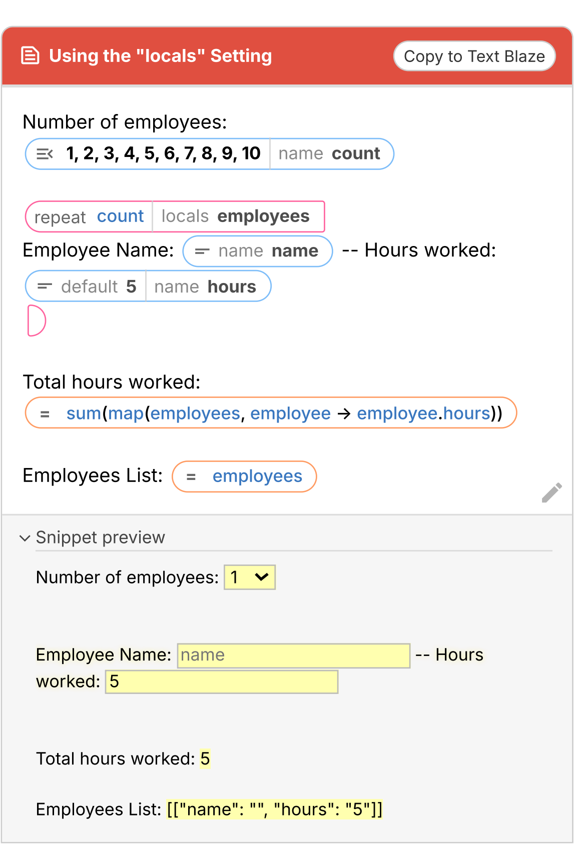
Accessing scoped variables outside of the `{repeat}` command

If you need access to the variables scoped to inside of the `{repeat}` command, you can use the optional `locals` named setting.

If used, the `locals` setting will create a list for you, with the identifier provided, where each item in the list is a keyed list for each repetition the command does.

Each key in the keyed list will be the name of the variable and the value for that key will the value of the variable.

For example, if we had a time sheet for our employees and we wanted to sum up their hours worked. We could implement it something like this:



Number of employees:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10 name count

repeat count locals employees
Employee Name: = name name -- Hours worked:
= default 5 name hours

Total hours worked:
= sum(map(employees, employee -> employee.hours))

Employees List: = employees

Snippet preview

Number of employees: 1

Employee Name: name -- Hours worked: 5

Total hours worked: 5

Employees List: [["name": "", "hours": "5"]]

Notice how after the Employees List, **the value is a list that contains keyed lists.**

Visit the Blaze Formula Reference to learn more about lists.

A note on whitespace

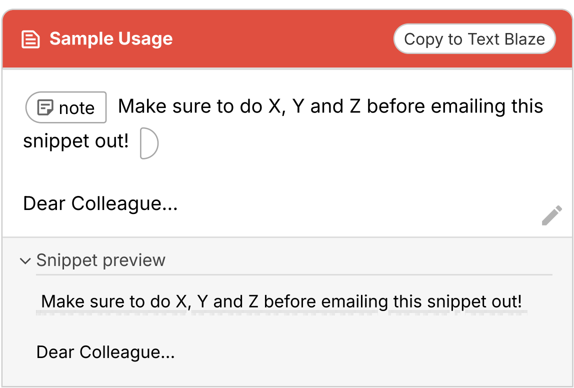
The `{repeat}` command does not output anything, but it still takes up whitespace in your snippet.

The optional `trim` setting is useful for removing whitespace around the `{repeat}` and `{endrepeat}` commands.

See the whitespace guide for more information on why this happens in Text Blaze.

{note}

Summary: A block of content that is not inserted into a snippet.



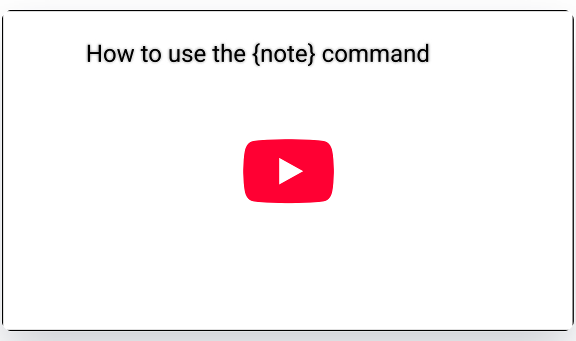
Make sure to do X, Y and Z before emailing this snippet out!

Dear Colleague...

Make sure to do X, Y and Z before emailing this snippet out!

Dear Colleague...

Quick Overview Video



Settings

Setting Name	Type	Description
insert	yes/no	Whether to include when inserted (defaults to `no`)
preview	yes/no	Whether to include in snippet previews (defaults to `yes`)
color	none/red/green/...	Change the color of the note's underline, or remove it completely.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `note` command can be used to display instructions or information you do not want to show up in the final inserted snippet.

By default, the contents of a `note` are shown in the snippet's preview, but are not inserted as part of the snippet.

For example, this can be useful if you're sharing a snippet folder with your team and need to document what's happening in the snippet.

To also hide the note contents in the snippet's preview, set the `preview` setting to `no`.

A note on whitespace

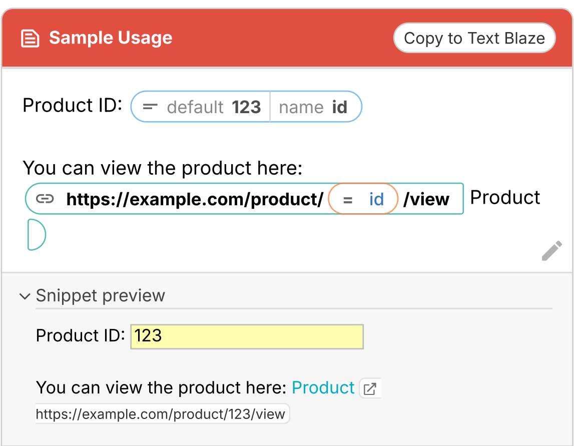
The `note` command does not output anything, but it still takes up whitespace in your snippet.

The optional `trim` setting is useful for removing whitespace around the command.

See the whitespace guide for more information on why this happens in Text Blaze.

{link}

Summary: Include a dynamic link in the snippet.



Product ID: = default 123 | name id

You can view the product here:
https://example.com/product/ = id /view Product

Product ID: 123

You can view the product here: Product
https://example.com/product/123/view

Settings

Setting Name	Type	Description
Positional	text	The URL for the link.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `link` command creates links in snippets where the URL may change based on some data.

The first positional setting that a URL for the link. You're able to use the `{=}` (Formula Command) here to build dynamic links.

> If the link URL contains characters that have special significance in Text Blaze commands, they must be escaped with backslashes.
>
> Notably `{`, `}`, `\` and `;` should be escaped by placing a single `\` before them.
>
> For example, `{` would become `\{` and `\` would become `\\`.

See the Blaze Formula Reference for more information on defining custom URLs based on dynamic data.

The `endlink` command

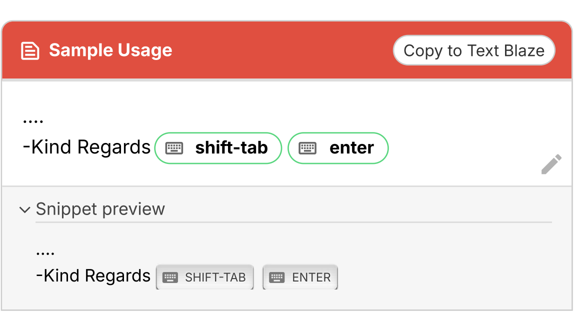
If you leave out the optional `endlink` command, then the link will display as is. If you insert content between the `link` and `endlink` the link will display with the inserted content instead.

Including a static link

If you want to include a link in the snippet that does not change based on some data, rather than using the `link` command you should use the link button in the styling toolbar.

{key}

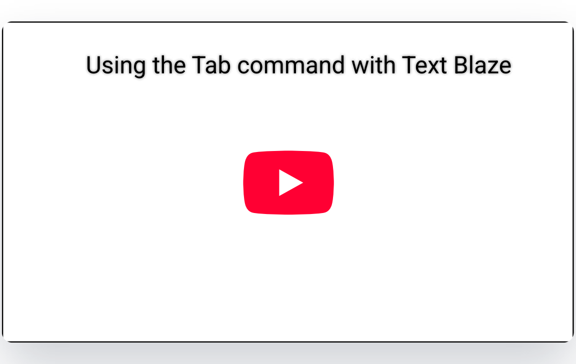
Summary: Simulates a key press in the current webpage.



....
-Kind Regards shift-tab enter

....
-Kind Regards SHIFT-TAB ENTER

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	text	The key press to simulate.
mac	text	Override key press when on a Mac
win	text	Override key press when on Windows
linux	text	Override key press when on Linux
cros	text	Override key press when on a ChromeBook
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{key}` command allows you to emulate a key press on the current page. For example, you can do `{key: s}` to simulate pressing the `s` key which will output an `s` character into the text field.

But, **the most common use case** is utilizing the `{key}` command to trigger shortcuts (`{key: ctrl-s}`), move the focus to another element (`{key: tab}`), or submit a form (`{key: enter}`). Click here to see more examples for the key command.

The syntax for the `{key}` command's positional setting is `modifier-character`. A `character` is the key that is pressed (e.g. `s` or a non-character like `tab`). The `modifier`, which is optional, is a key like `shift` key or the `ctrl` key. **For example: `{key: shift-tab}`** is utilizing both a modifier key and a character key.

> Text Blaze key presses can trigger actions **within a web page**, but they cannot trigger standard system commands on your operating system.
>
> For example, `ctrl-c` will not copy the currently selected text to the clipboard and `ctrl-q` will not quit the web browser as both those are system commands.
>
> This also means that different websites might behave differently.
>
> See the autopilot guide to learn more about what you can and cannot do with the `{key}` command.

Click here to see an example of automating writing your email utilizing the `{key}` command and form commands.

Modifiers

The following modifiers are supported:

Key	Modifier
Shift Key	`shift`
Control Key	`ctrl`
Alt (Option) Key	`alt`
Command (Meta) Key	`cmd`

Also, it's possible to use multiple modifiers joined together with hyphens. For example `ctrl-shift-x` or `alt-shift-tab`.

Non-Character Keys

In addition to key presses that can be represented with a standard keyboard character (e.g. `{key: s}` or `{key: p}`), the following non-character keys are supported:

Key	Key Token
Tab Key	`tab`
Enter Key	`enter`
Backspace Key	`backspace`
Left Arrow Key	`leftarrow`
Right Arrow Key	`rightarrow`
Up Arrow Key (experimental)	`uparrow`
Down Arrow Key (experimental)	`downarrow`

An example of usage of these keys would be `{key: tab}` to tab to the next field in the webpage (or `{key: shift-tab}` to tab to the prior field in the document). The remainder of the snippet would then be entered in that field, allowing you to fill out multiple fields with a single snippet.

The `enter` key is also useful for pressing a button to submit a form or trigger some action. `tab` and `enter` key commands can often be used sequentially to enable a variety of scripting behaviors.

Click here to see a real-world example utilizing the `{click}` and `{key}` autopilot commands.

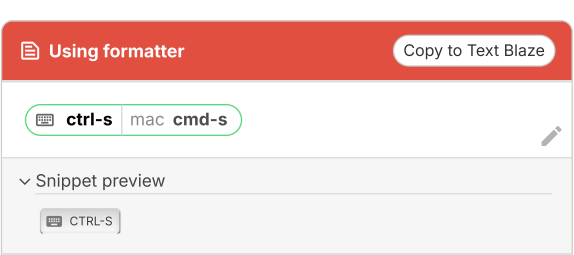
Supporting Multiple Operating Systems

Key presses may be different on different operating systems. For example, saving a file may be `ctrl-s` on a Windows computer, but `cmd-s` on a Macintosh computer.

Text Blaze lets you define different key presses for each operating system via the optional `win`, `mac`, `linux` and `cros` override settings. When one of these is defined and the snippet is used on that operating system, Text Blaze will use the key press you defined for that operating system, rather than the default key press, which you define in the first positional setting.

For example, if you want to use `ctrl-s` on Windows, Linux or ChromeBooks; but `cmd-s` on a Macintosh computer, you could use the following:

This is most useful if you're using Text Blaze with your team and sharing snippets with them that will be used on a variety of operating systems.



ctrl-s mac cmd-s

CTRL-S

To learn more about automating actions in a web page, see the **autopilot guide**.

{import}

Summary: Imports another snippet into the current snippet.

Sample Usage Copy to Text Blaze

...and best wishes.

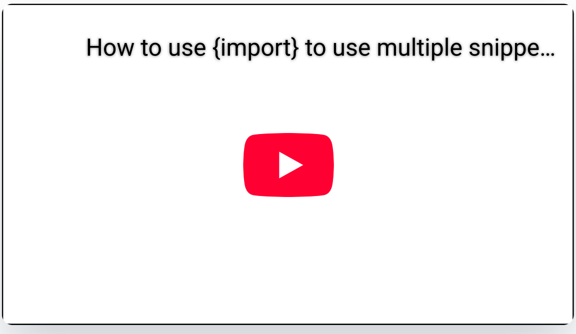
signature

Snippet preview

...and best wishes.

[Imported Snippet: "signature"]

Quick Overview Video


How to use {import} to use multiple snippe...

Settings

Setting Name	Type	Description
Positional	text	The shortcut of the snippet to import (what you type to insert the snippet, not the snippet's label)
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{import}` command can be used to combine snippets and reuse common parts of snippets. Form fields with the same name will be linked across the combined snippet.

The shortcut you use in the `import` command should be the shortcut text you actually type in to insert the snippet, not the snippet's descriptive label.

If the shortcut contains characters that have special meaning in Text Blaze commands (";", "=", "{" and "}"), you will need to escape them by placing a backslash before each special character.

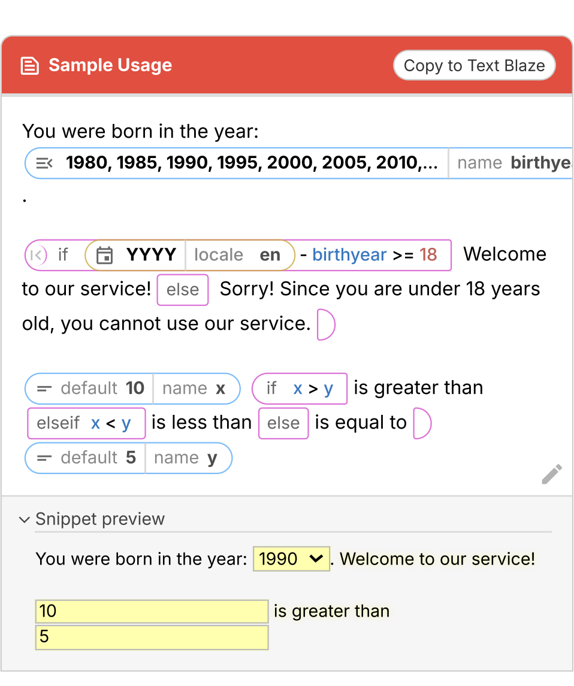
For example, if your snippet shortcut is ";go;", you can import it with "{import: \;go\;}".

Learn more about escaping characters, by visiting the Formula Reference.

> There are a few caveats when using the {snippet} command together with the `{import}` command. Click here to learn more.

{if}

Summary: Conditionally includes content in the snippet.



Sample Usage

Copy to Text Blaze

You were born in the year: 1980, 1985, 1990, 1995, 2000, 2005, 2010,... name birthye
.

if YYYY locale en - birthyear >= 18 Welcome to our service! else Sorry! Since you are under 18 years old, you cannot use our service.

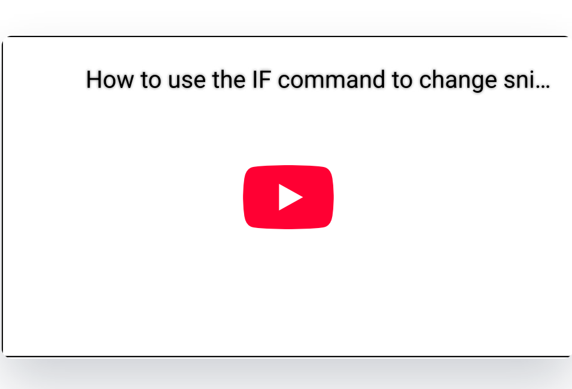
default 10 name x if x > y is greater than elseif x < y is less than else is equal to
default 5 name y

Snippet preview

You were born in the year: 1990. Welcome to our service!

10 is greater than
5

Quick Overview Video



How to use the IF command to change sni...

Settings

Setting Name	Type	Description
Positional	formula	The condition to test. Must evaluate to `yes` or `no`.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The {if} command lets you conditionally include or exclude content based on a condition. That condition is a Text Blaze Formula that must evaluate to either `yes` or `no`.

Everything within the {if} command and the required `{endif}` command will be inserted if the condition evaluates to `yes`.

> Note the use of {time: YYYY; locale=en} to get the current year in the `if` command in the example at the top of the page. This allows the snippet to automatically update the check when Jan 1 passes without you having to update it manually.
>
> The `locale=en` setting ensures the year number will be printed out in the Western formatting of `2017` or `2018` which is required for use in the `if` command. Some locales have other types of number formatting. You may omit this if your locale has Western number formatting.

See also the {=} (Formula Command) command and the {repeat} command for additional tools to help you build dynamic snippets.

The required {endif} command

The `if` command is similar to the {note} command in that the closing command must be present. In this case, it's the `{endif}` command.

Everything inside of these two commands will or will not get rendered depending on the output of the condition, the `if` command's positional setting.

> The `trim` setting is useful for removing whitespace around the `if`, `elseif`, `else`, and `endif` commands. See the whitespace guide to learn more.

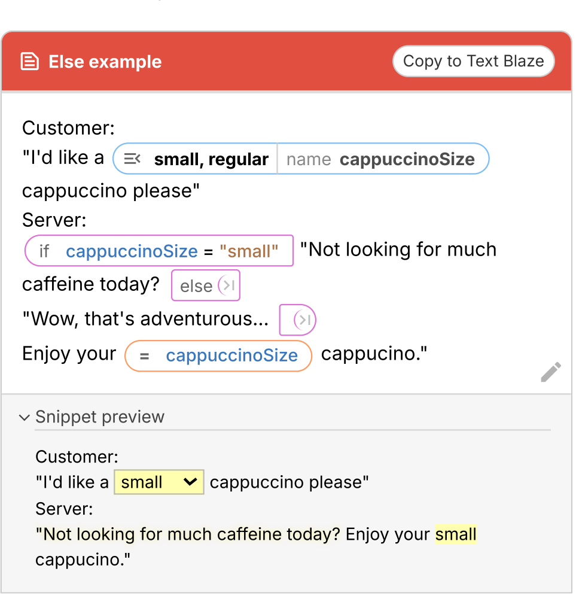
The optional {else} command

What if you would like to do something different if the condition evaluates to `no`?

In that case, you have access to an `{else}` command. The `{else}` command must be placed within the starting `if` command and the closing `{endif}` command.

There can only be one `{else}` command.

Here's an example:



Else example

Copy to Text Blaze

Customer:
"I'd like a small, regular name cappuccinoSize cappuccino please"
Server:
if cappuccinoSize = "small" "Not looking for much caffeine today? else
"Wow, that's adventurous...
Enjoy your = cappuccinoSize cappucino."

Snippet preview

Customer:
"I'd like a small cappuccino please"
Server:
"Not looking for much caffeine today? Enjoy your small cappucino."

> ### Try it out
>
> - Try changing the {formmenu} command's value above and notice which part of the `if` command gets rendered.

Multiple if conditions with the optional {elseif} command

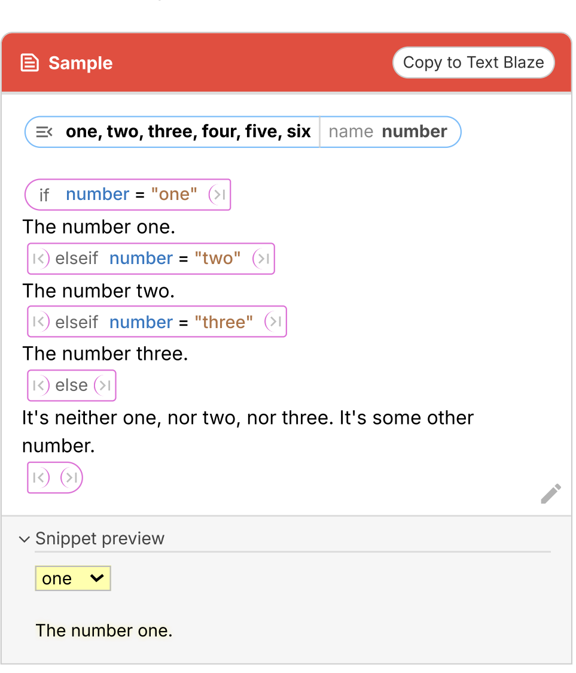
It's possible to test for multiple conditions with the help of the `{elseif}` command.

Similar to the {if} command, the `{elseif}` also takes a condition as its positional setting that is a Blaze Formula.

Also, just like the `{else}` command, they must be placed within the starting `if` command and the ending `{endif}` command.

You can have multiple `{elseif}` commands.

Here's an example:



Sample

Copy to Text Blaze

one, two, three, four, five, six name number

if number = "one"
The number one.
elseif number = "two"
The number two.
elseif number = "three"
The number three.
else
It's neither one, nor two, nor three. It's some other number.

Snippet preview

one

The number one.

Here's what we're telling Text Blaze in the snippet above:

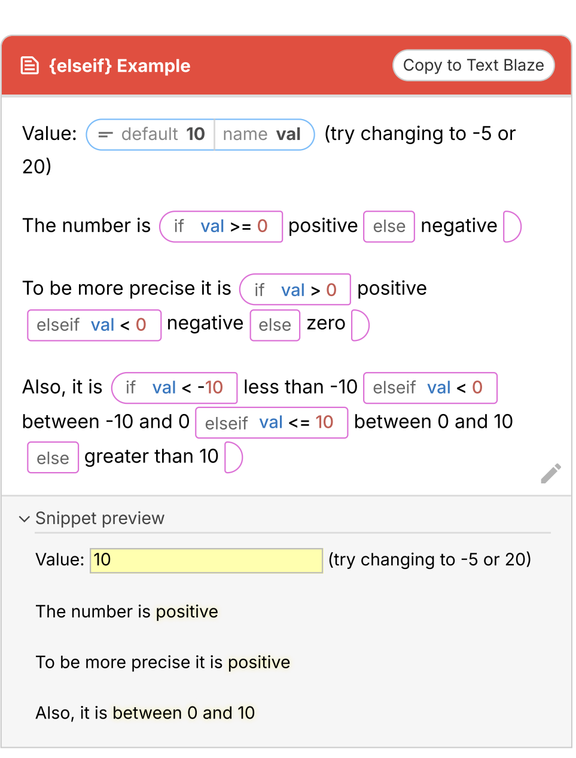
1. If the form name called *number* contains the text "one", then give us the text "The number one"
2. If the above isn't true, check whether it contains the text "two".
3. If the above is true, then give us the text "The number two". If not, check whether it contains the text "three".

And so on.

At the very end, you'll need to end with an `{endif}` command. The `{endif}` command is always placed at the very end.

If the initial `if` conditional evaluates to `no`, Text Blaze will look for the next `{elseif}` command and evaluate its condition. It evaluates to `yes`, its contents will be shown, otherwise, Text Blaze will move on to the next `{elseif}`.

If all `{elseif}` commands within the `if` command are evaluated to `no` and there is an `{else}` command, the contents of the `{else}` command will be shown. The following examples illustrate the use of `{elseif}` and `{else}`.



{elseif} Example

Copy to Text Blaze

Value: default 10 name val (try changing to -5 or 20)

The number is if val >= 0 positive else negative

To be more precise it is if val > 0 positive elseif val < 0 negative else zero

Also, it is if val < -10 less than -10 elseif val < 0 between -10 and 0 elseif val <= 10 between 0 and 10 else greater than 10

Snippet preview

Value: 10 (try changing to -5 or 20)

The number is positive

To be more precise it is positive

Also, it is between 0 and 10

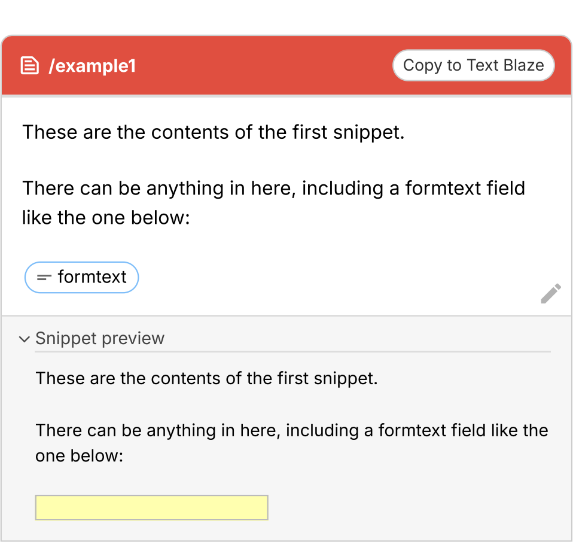
Dynamic content using the `if` command

Text Blaze becomes incredibly powerful when you start combining different commands like {formmenu}, {if} and {import}.

For this example, try saving the following three snippets below into your Text Blaze dashboard. This will help you understand better how Text Blaze uses these functions by seeing them in action.

To copy the snippet examples to your dashboard, click the "Copy to Text Blaze" button in the top right of the interactive snippet.

First, save the snippet below and give it the shortcut `/example1`.



/example1

Copy to Text Blaze

These are the contents of the first snippet.

There can be anything in here, including a formtext field like the one below:

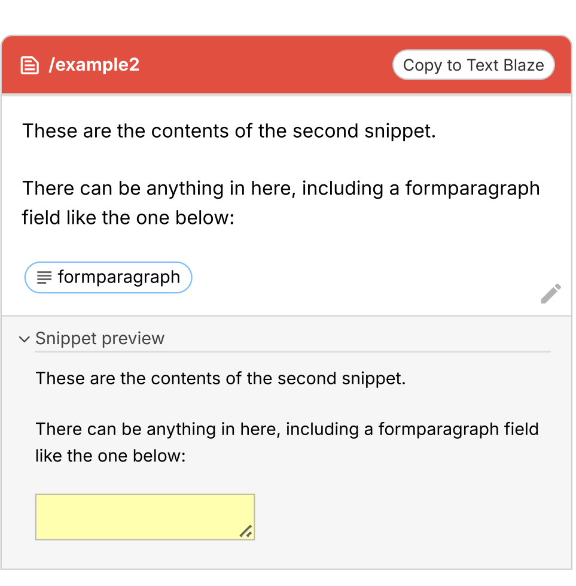
formtext

Snippet preview

These are the contents of the first snippet.

There can be anything in here, including a formtext field like the one below:

Now, save this second snippet below and give it the shortcut `/example2`.



/example2

Copy to Text Blaze

These are the contents of the second snippet.

There can be anything in here, including a formparagraph field like the one below:

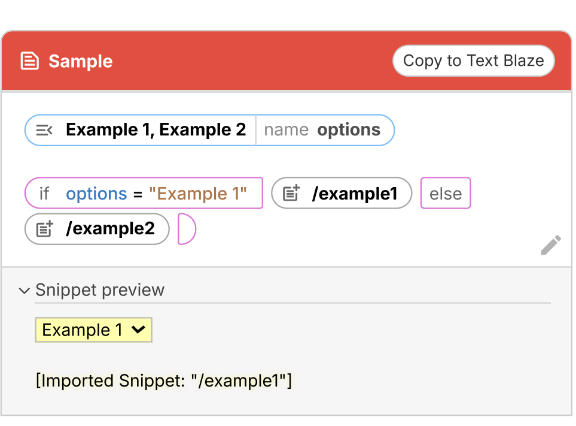
formparagraph

Snippet preview

These are the contents of the second snippet.

There can be anything in here, including a formparagraph field like the one below:

Finally, save this third snippet below and give it any shortcut you please.



Sample

Copy to Text Blaze

Example 1, Example 2 name options

if options = "Example 1" /example1 else /example2

Snippet preview

Example 1

[Imported Snippet: "/example1"]

Now try running the third snippet somewhere in a text field.

Depending on the option you choose in the drop-down menu, Text Blaze will give you the contents of the respective snippet.

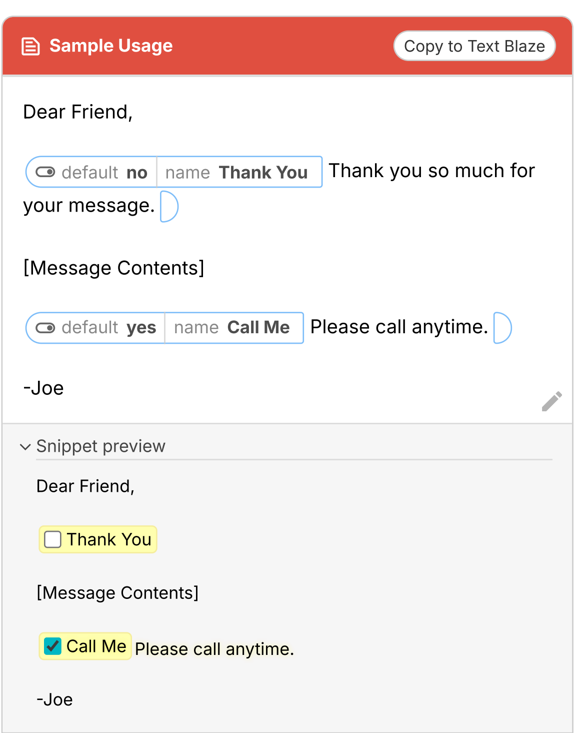
Now here's how that works.

1. We created a drop-down menu with {formmenu} and gave it two options – *Example 1* and *Example 2*. We also gave it the name "*options*".
2. We created an `if` command and gave it the following instructions:
    - If the form name called *options* is equal to "Example 1", then you should import the contents of the snippet with the shortcut `/example1`
    - If the above is not true, then you should import the contents of the snippet with the shortcut `/example2` (that's where the {else} command comes in).

You don't have to use an {import} command when using {if} commands, but they can help you better manage your snippets.

{formtoggle}

Summary: Creates a toggle switch that can enable or disable a block of text.



Dear Friend,

default **no** | name **Thank You** Thank you so much for your message.

[Message Contents]

default **yes** | name **Call Me** Please call anytime.

-Joe

Dear Friend,

☐ Thank You

[Message Contents]

☑ Call Me Please call anytime.

-Joe

Settings

Setting Name	Type	Description
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

Toggles allow you to selectively hide and show parts of your snippet. If a toggle is unchecked, everything between the `{formtoggle}` and `{endformtoggle}` commands will be hidden.

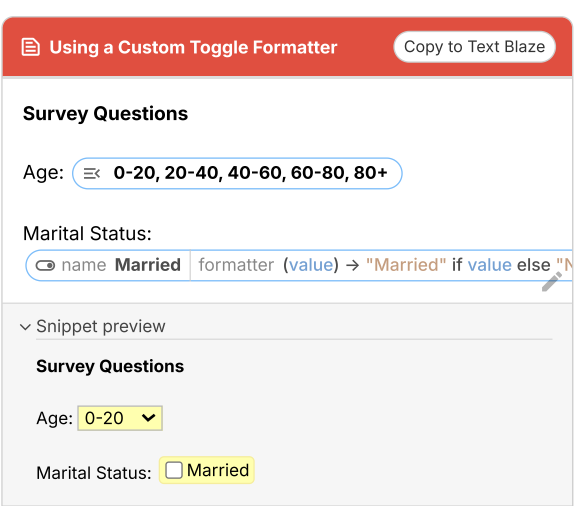
This command is similar to the {if} command, but the `{if}` command uses formulas to dynamically include or exclude content.

The optional {endformtoggle} command

The `{endformtoggle}` is optional and if omitted, `yes` or `no` will be shown when the snippet is inserted based on the value of the toggle's value.

You may change the output using the optional `formatter` setting. The `formatter` setting takes a function where the first parameter is the value of the toggle, either `yes` or `no`.

The following example illustrates this (you will need to copy it into Text Blaze and trigger the snippet to see it in practice).



Survey Questions

Age: **0-20, 20-40, 40-60, 60-80, 80+**

Marital Status:
name **Married** | formatter (value) -> "Married" if value else "N

Survey Questions

Age: 0-20

Marital Status: ☐ Married

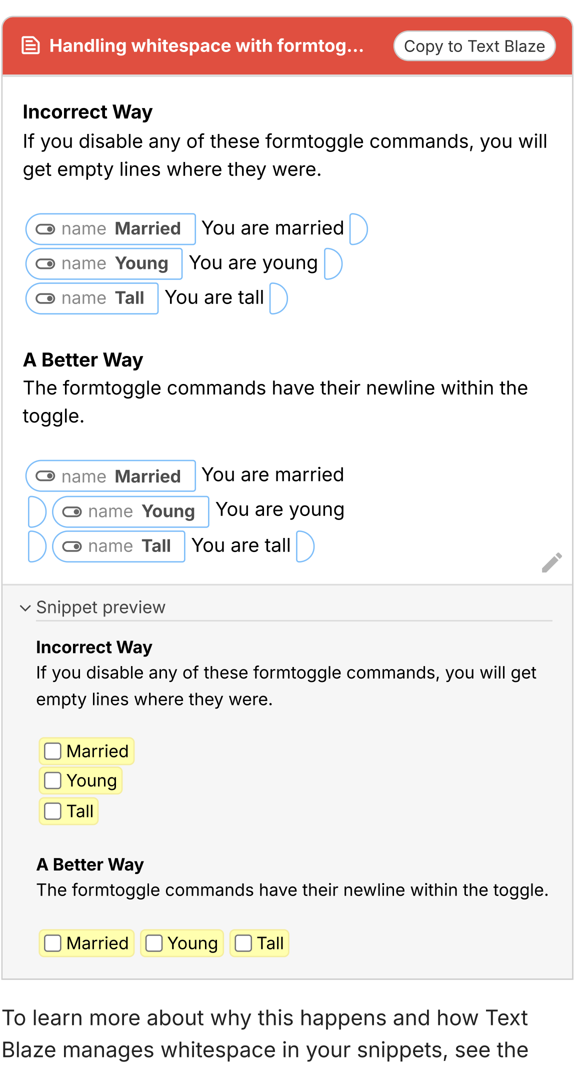
The above snippet will output `Married` instead of `yes`, or `Not Married` if the toggle's value is `no`.

Click here to learn more about the formatter function available to you.

{formtoggle} and whitespace

With multiline commands like {if} or `{formtoggle}` you generally want to have trailing or leading new lines to be *within* the command. **This can be counterintuitive as it's generally not how you would write the snippet by default**.

This prevents multiple extraneous newlines from showing up in your snippets. Here's an example demonstrating the issue:



Incorrect Way
If you disable any of these formtoggle commands, you will get empty lines where they were.

name **Married** You are married
name **Young** You are young
name **Tall** You are tall

A Better Way
The formtoggle commands have their newline within the toggle.

name **Married** You are married
name **Young** You are young
name **Tall** You are tall

Incorrect Way
If you disable any of these formtoggle commands, you will get empty lines where they were.

☐ Married
☐ Young
☐ Tall

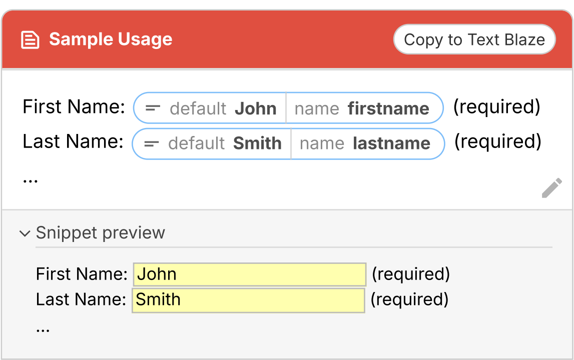
A Better Way
The formtoggle commands have their newline within the toggle.

☐ Married ☐ Young ☐ Tall

To learn more about why this happens and how Text Blaze manages whitespace in your snippets, see the whitespace guide.

{formtext}

Summary: Creates a single line text input field.

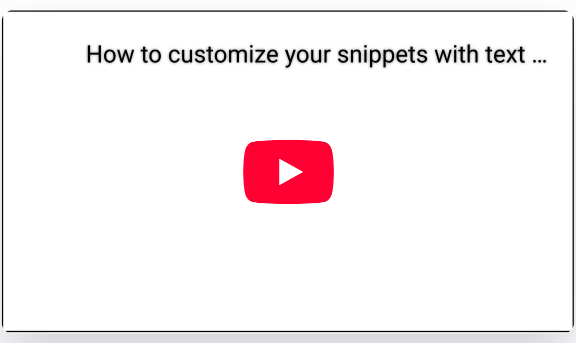

Sample Usage
Copy to Text Blaze

First Name: default **John** name **firstname** (required)
Last Name: default **Smith** name **lastname** (required)
...

Snippet preview

First Name: John (required)
Last Name: Smith (required)
...

Quick Overview Video


How to customize your snippets with text ...

Settings

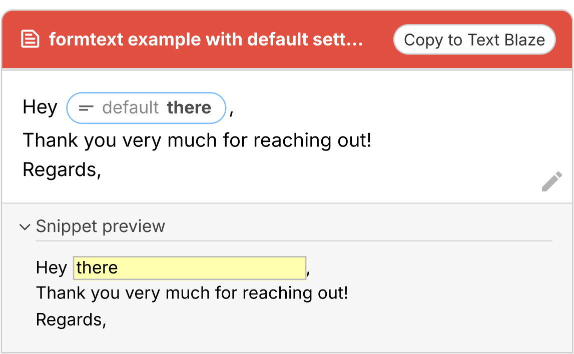
Setting Name	Type	Description
cols	number	The width of the form field in columns of characters.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{formtext}` command is like your traditional input field that you would use in forms across the web. This lets you customize your snippets before inserting them.

To learn more about form commands and their capabilities, see the **forms guide**.

The `default` setting

The `{formtext}` command has access to an optional `default` setting. One example is utilizing `there` as the value so that a generic message like `Hey there`, appears if you do not customize the form.


formtext example with default sett...
Copy to Text Blaze

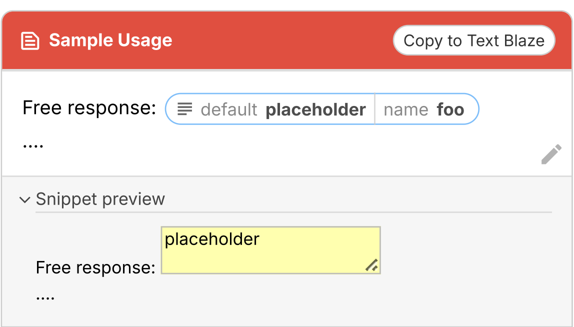
Hey default **there**,
Thank you very much for reaching out!
Regards,

Snippet preview

Hey there,
Thank you very much for reaching out!
Regards,

{formparagraph}

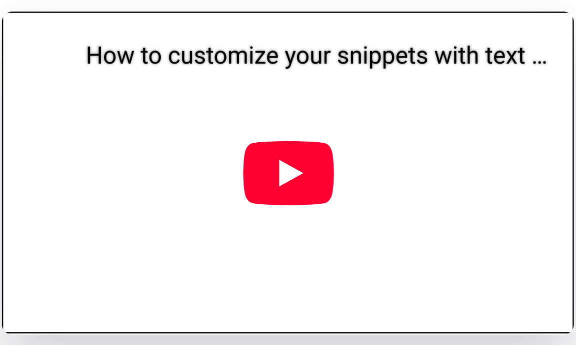
Summary: Creates a multi-line text input field.



Free response: {default placeholder | name foo}
....

Free response: placeholder
....

Quick Overview Video



Settings

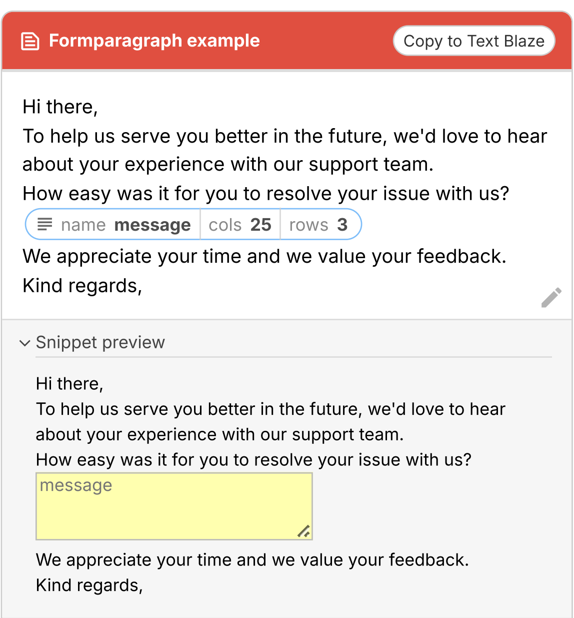
Setting Name	Type	Description
cols	number	The width of the form field in columns of characters.
rows	number	The height of the form field in rows.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

The `{formparagraph}` command is very similar to the {formtext} command, but it allows you to see more text by giving you access to the optional `rows` setting.

It's very useful if you set up a snippet as a template and you want to insert more custom content somehwhere in the snippet.

For example, you can have a general template that you use at the top and bottom of your snippet but you would like to customize the middle.



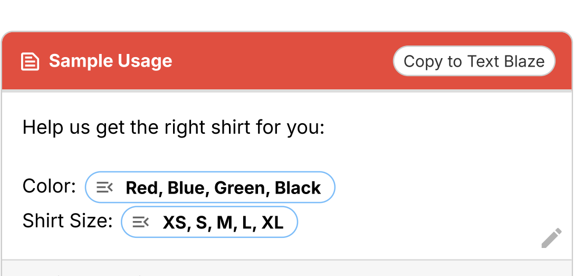
Hi there,
To help us serve you better in the future, we'd love to hear about your experience with our support team.
How easy was it for you to resolve your issue with us?
{name message | cols 25 | rows 3}
We appreciate your time and we value your feedback.
Kind regards,

Hi there,
To help us serve you better in the future, we'd love to hear about your experience with our support team.
How easy was it for you to resolve your issue with us?
message
We appreciate your time and we value your feedback.
Kind regards,

You also might consider wrapping the `{formparagraph}` command with the {formtoggle} command if this will be an optional part of your snippet.

{formmenu}

Summary: Creates a menu input to select one or more items.



Sample Usage Copy to Text Blaze

Help us get the right shirt for you:

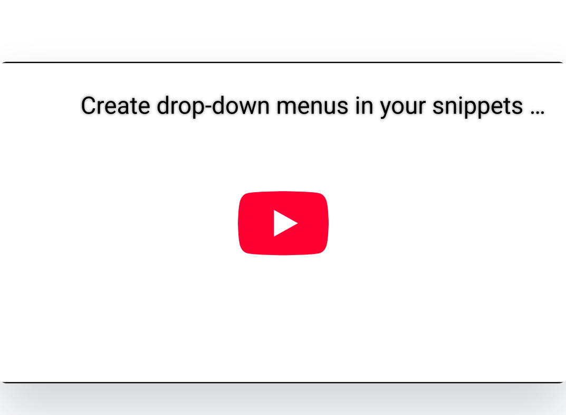
Color: Red, Blue, Green, Black
Shirt Size: XS, S, M, L, XL

Snippet preview

Help us get the right shirt for you:

Color: Red
Shirt Size: M

Quick Overview Video



Create drop-down menus in your snippets ...

Settings

Setting Name	Type	Description
Positional	text	The ordered list of menu items.
cols	number	The width of the form menu in columns of characters.
multiple	yes/no	Allow selecting multiple items (default is no).
itemformatter	function	Formats individual menu items. Takes two arguments, the text of the menu item and the index of the menu item.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

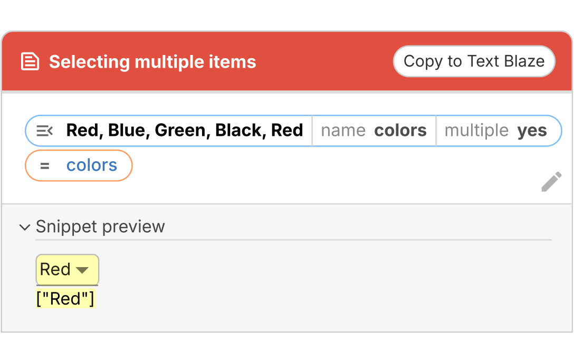
The `{formmenu}` command is a unique command in that it takes multiple positional settings. Each positional setting that you include is an item that you will be able to select in the menu.

> ### Try it out
>
> - In the example at the top of the page, add another positional setting, "Yellow", to the {formmenu} and see how another item gets added.

Selecting multiple items

The `{formmenu}` command also allows you to define a `multiple` setting. By default it's set to `no`, but if you explicitly set it to `yes`, you can now select multiple items.

If you also specifiy a `name` setting, then the results will be returned as a list if you use it in a {=} (Formula Command) command.



Selecting multiple items Copy to Text Blaze

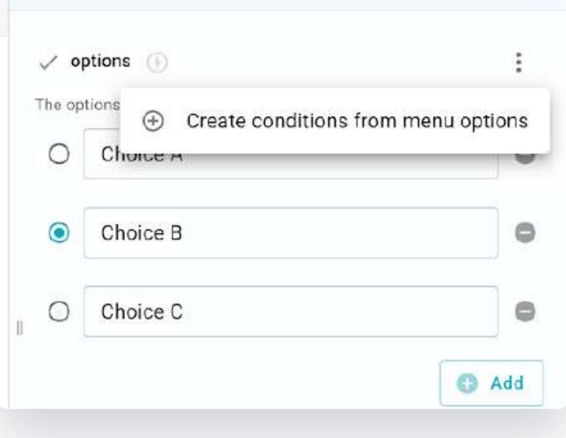
Red, Blue, Green, Black, Red | name colors | multiple yes
= colors

Snippet preview

Red
["Red"]

> ### Try it out
>
> - Notice how the result of the formmenu is initially set to "[]", an empty list if `multiple` is set to true.
> - Try adding a value for the `default` setting and see what happens to the list in output in the example above.

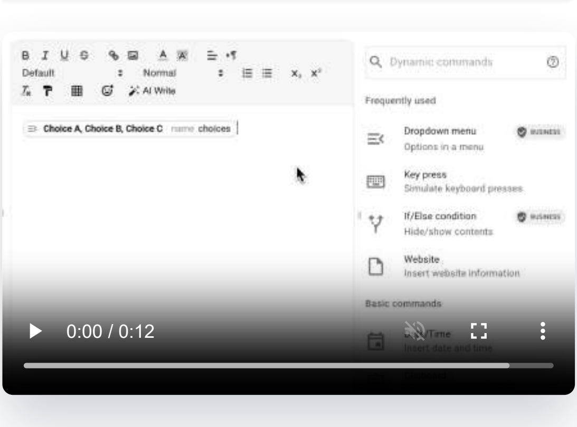
Creating conditions from menu options



You can quickly create if/else conditions for `{formmenu}` options by clicking on a form-menu, then clicking the three dots in the right corner of the menu editor.

Doing this adds conditional text that only shows up and gets inserted if a specific item in the menu is selected.

> Note that the if statements do not update if the values of the form menu change. If you change values in the menu, you will need to update the conditions accordingly.



Keep in mind that this option is not available when the 'Multiple' setting is selected. If your menu is set to 'Multiple', you will have to use the contains() function in your if statements for each item in the menu.

This option also does not work if you have the 'Advanced Editor' (lightning symbol) turned on for your drop-down menu.

Custom Formatting

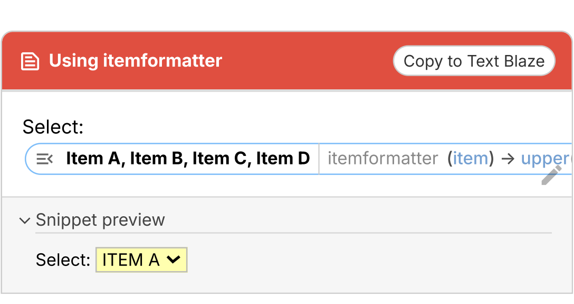
There are two settings that customize the formatting of the menu items:

- `itemformatter`: Formats each individual menu item.
- `formatter`: Formats the entire list of selected items. This is primarily useful when you have the `multiple` setting set to `yes` and are selecting multiple items from the list.

Using `itemformatter`

The `itemformatter` setting is a function that takes the text of each individual menu items and transforms it.

For example, if you wanted to uppercase each menu item, you could do something like the following.



Using itemformatter Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | itemformatter (item) → upper

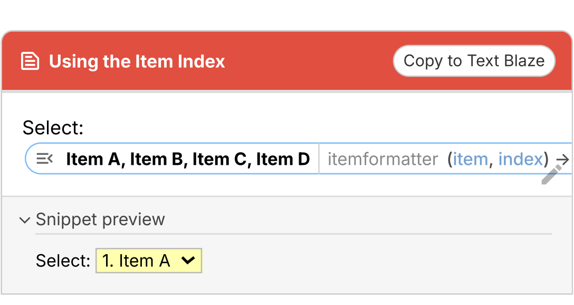
Snippet preview

Select: ITEM A

> - There are various other string predefined functions available for you in Text Blaze to use. For example: `lower()`, `reverse()`, and more.

The `itemformatter` function takes an optional second parameter which is the index of the menu item. This is the position of the item in the list.

For example, we can use that to number our items:



Using the Item Index Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | itemformatter (item, index) →

Snippet preview

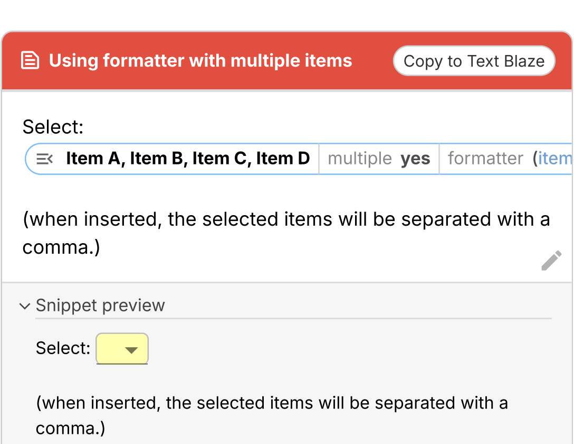
Select: 1. Item A

If the syntax of the function above looks unfamiliar, learn more about functions in the Text Blaze Formula Reference.

Using `formatter`

The `formatter` setting is a function that allows you to format **all of the items** in your `{formmenu}`.

If `multiple` is `yes`, it will be passed a list of values (which may be empty if no items have been selected).



Using formatter with multiple items Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | multiple yes | formatter (item

(when inserted, the selected items will be separated with a comma.)

Snippet preview

Select:

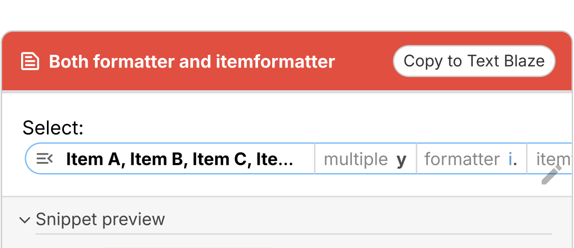
(when inserted, the selected items will be separated with a comma.)

> `join` is a predefined list function available to you. It takes items in a list and joins them together into a single string separated by a string, in this case a comma (,).

If `multiple` is `no`, the first parameter of the `formatter` setting is a single string value.

Combining Both Formatters

The following example illustrates the use of the `itemformatter` and `formatter` functions together.



Both formatter and itemformatter Copy to Text Blaze

Select:
Item A, Item B, Item C, Ite... | multiple y | formatter i. | item

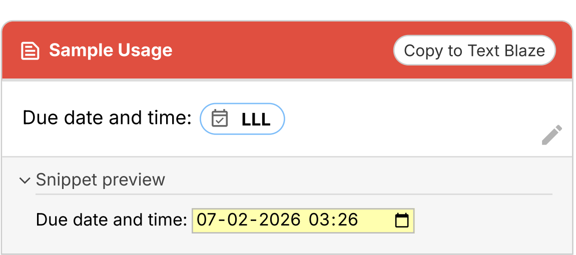
Snippet preview

Select: No Items Selected

See the Blaze Formula Reference for more information on defining formatting functions.

{formdate}

Summary: Creates a date/time input field.



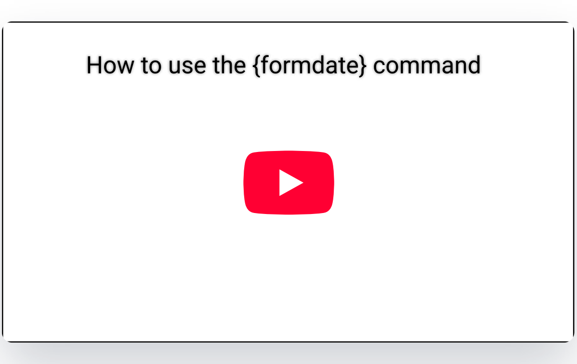
Sample Usage Copy to Text Blaze

Due date and time: LLL

Snippet preview

Due date and time: 07-02-2026 03:26

Quick Overview Video



How to use the {formdate} command

Settings

Setting Name	Type	Description
Positional	format	The format for the date/time.
start	date/time	The minimum selectable date/time.
end	date/time	The maximum selectable date/time.
cols	number	The width of the form field in columns of characters.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

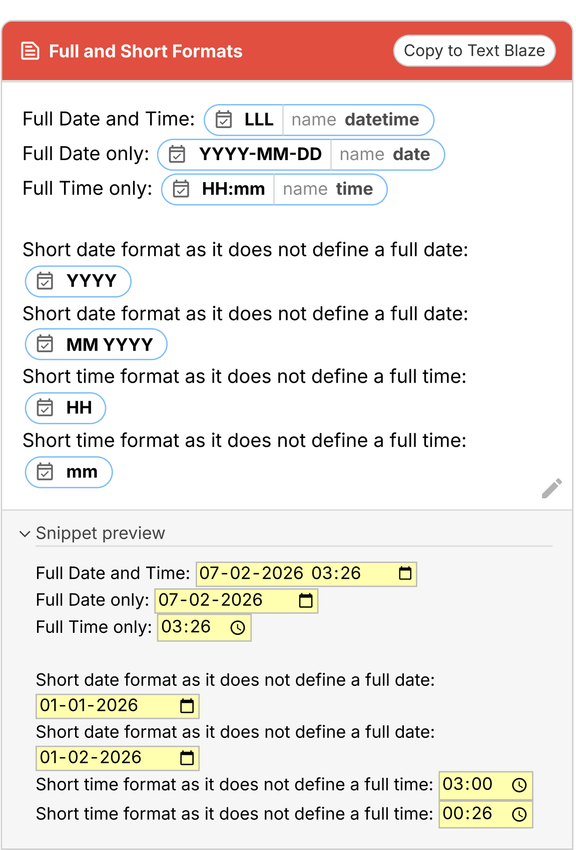
To learn more about form commands and their capabilities, see the forms guide.

Formatting Dates and Times

The `{formdate}` command requires a `format` setting that tells Text Blaze how to format the date/time when inserting. This format uses the same notation as the {time} command.

Click here to learn about time/date formatting in Text Blaze.

The `format` setting must contain enough information to define a full date or a full time. **Generally, the date needs to contain at least a day, month and year component to be full. Similarly, the time needs to contain hours and minutes to be full**. For this reason `MM DD YYYY` would be considered a full date format, whereas `YYYY` would be short as it omits the day and month. Similarly, `HH:mm` would be a full time format, while `HH` would be short as it lacks the minutes component.



Full and Short Formats Copy to Text Blaze

Full Date and Time: LLL name datetime
Full Date only: YYYY-MM-DD name date
Full Time only: HH:mm name time

Short date format as it does not define a full date: YYYY
Short date format as it does not define a full date: MM YYYY
Short time format as it does not define a full time: HH
Short time format as it does not define a full time: mm

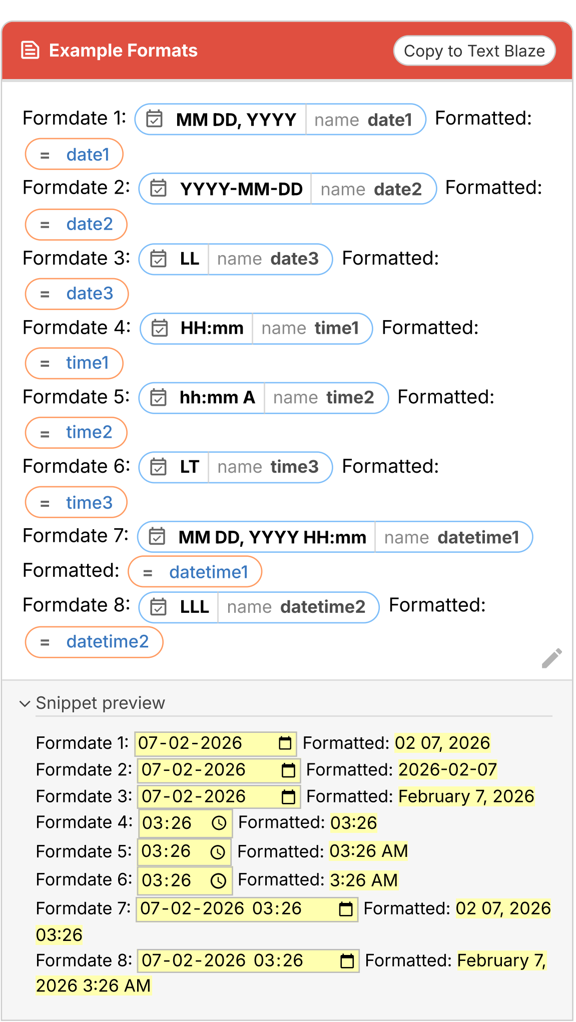
Snippet preview

Full Date and Time: 07-02-2026 03:26
Full Date only: 07-02-2026
Full Time only: 03:26

Short date format as it does not define a full date: 01-01-2026
Short date format as it does not define a full date: 01-02-2026
Short time format as it does not define a full time: 03:00
Short time format as it does not define a full time: 00:26

When entering a date/time via the `{formdate}` in the form popup, the date selector will use the same format that you define in the `{formdate}`'s positional setting.

Also, the same format will be used when using the date/time in a formula.



Example Formats Copy to Text Blaze

Formdate 1: MM DD, YYYY name date1 Formatted: = date1
Formdate 2: YYYY-MM-DD name date2 Formatted: = date2
Formdate 3: LL name date3 Formatted: = date3
Formdate 4: HH:mm name time1 Formatted: = time1
Formdate 5: hh:mm A name time2 Formatted: = time2
Formdate 6: LT name time3 Formatted: = time3
Formdate 7: MM DD, YYYY HH:mm name datetime1 Formatted: = datetime1
Formdate 8: LLL name datetime2 Formatted: = datetime2

Snippet preview

Formdate 1: 07-02-2026 Formatted: 02 07, 2026
Formdate 2: 07-02-2026 Formatted: 2026-02-07
Formdate 3: 07-02-2026 Formatted: February 7, 2026
Formdate 4: 03:26 Formatted: 03:26
Formdate 5: 03:26 Formatted: 03:26 AM
Formdate 6: 03:26 Formatted: 3:26 AM
Formdate 7: 07-02-2026 03:26 Formatted: 02 07, 2026 03:26
Formdate 8: 07-02-2026 03:26 Formatted: February 7, 2026 3:26 AM

> ### Try it out
>
> - Change the format of the `{formdate}` commands above and notice how the format of the output will also change.

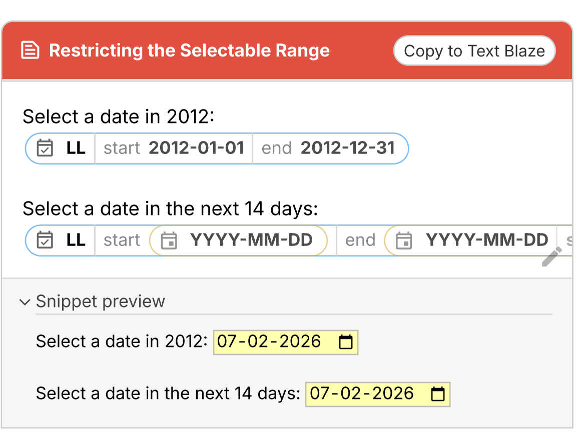
Restricting Selectable Dates and Times

You can optionally restrict the range of dates/times that can be selected using the `start` and `end` settings.

> In case of time, you will see an error notification instead of disabled values in the time picker.

The `start` and `end` settings are dates/times that **may be entered in the same format as the formdate's `format` setting**, or in a canonical `YYYY-MM-DD` or `HH:mm` format.

You can also embed {time} commands in order to create a dynamic range as seen in this example.



Restricting the Selectable Range Copy to Text Blaze

Select a date in 2012: LL start 2012-01-01 end 2012-12-31

Select a date in the next 14 days: LL start YYYY-MM-DD end YYYY-MM-DD

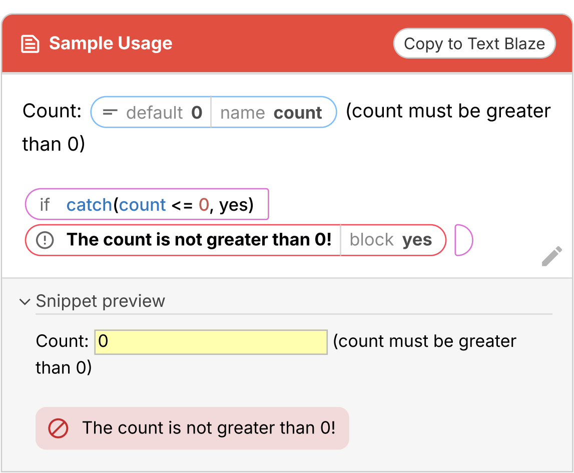
Snippet preview

Select a date in 2012: 07-02-2026

Select a date in the next 14 days: 07-02-2026

{error}

Summary: Show an error message in a snippet. Use it to validate user input and block snippet insertion.



Sample Usage

Count: = default 0 | name count (count must be greater than 0)

if catch(count <= 0, yes)

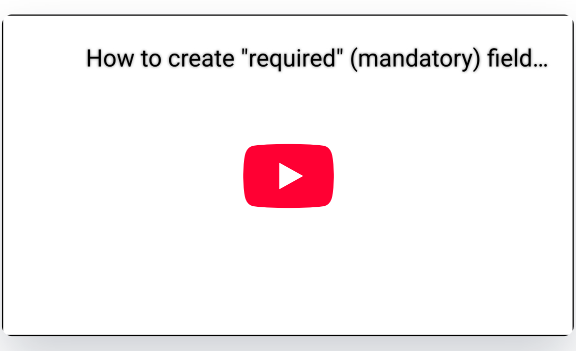
The count is not greater than 0! | block yes

Snippet preview

Count: 0 (count must be greater than 0)

The count is not greater than 0!

Quick Overview Video



How to create "required" (mandatory) field...

Settings

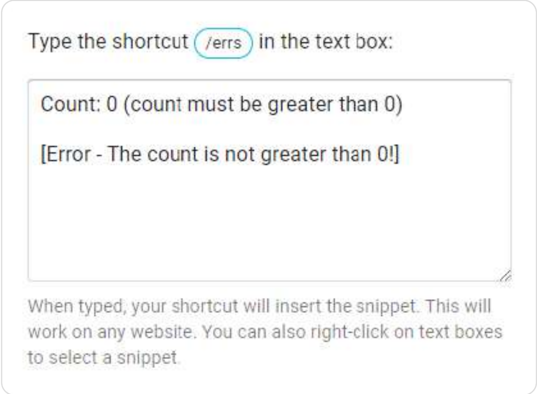
Setting Name	Type	Description
Positional	text	The error message to show to the user.
block	yes/no	If `yes`, blocks insertion while the error is shown.
show	default/validate	`validate` will show the error only after you attempt to insert the snippet.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `error` command can be used for form validation and for preventing a snippet from being inserted until certain conditions are met.

The `error` command is best used in combination with the {if} command to conditionally show an error and block snippet insertion.

The `block` setting

If the `block` setting is `no` the message that you used in the first positional setting will be inserted into your snippet.



Type the shortcut /errs in the text box:

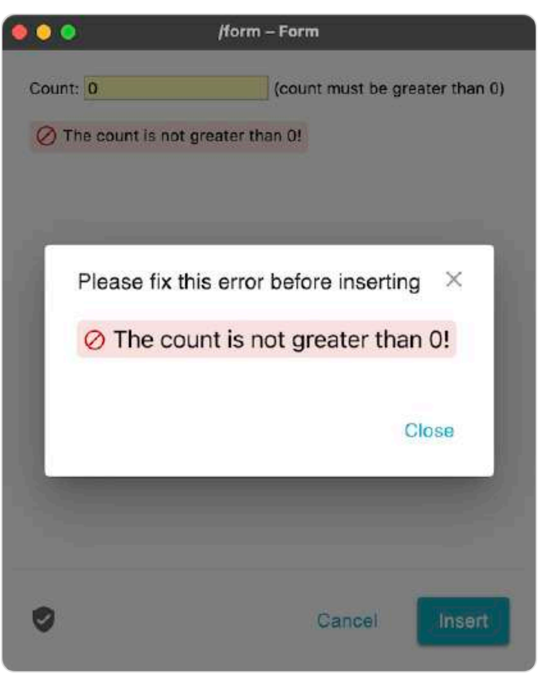
Count: 0 (count must be greater than 0)

[Error - The count is not greater than 0!]

When typed, your shortcut will insert the snippet. This will work on any website. You can also right-click on text boxes to select a snippet.

Notice how the error text is inserted into the snippet itself inside of the `[Error - ]` text.

If `block` is `yes`, the message will be displayed (not inserted) and **the snippet user will not be able to insert the snippet until the error is resolved**.



Count: 0 (count must be greater than 0)

The count is not greater than 0!

Please fix this error before inserting

The count is not greater than 0!

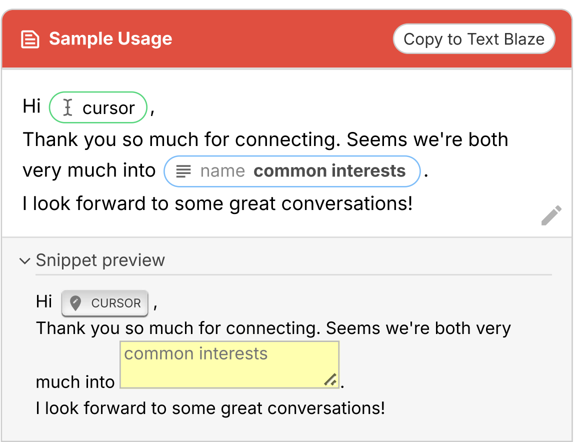
Text Blaze will not allow you to insert the snippet by showing an alert.

Error checking your team's snippets

A good use case for the `error` command is when you're sharing snippets with your team. It will ensure that everyone using the snippet is entering the right information and validating it correctly before the final insertion.

{cursor}

Summary: Moves the mouse cursor after the snippet has been entered.

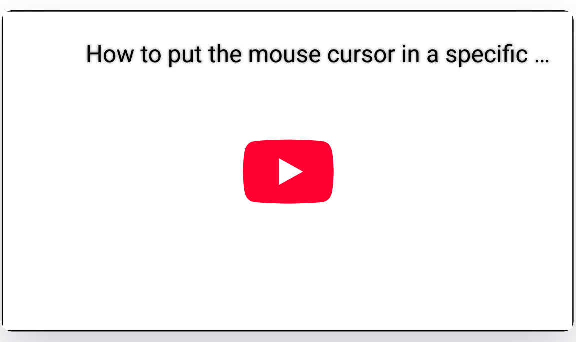


Hi cursor ,
Thank you so much for connecting. Seems we're both very much into name **common interests** .
I look forward to some great conversations!

Snippet preview

Hi CURSOR ,
Thank you so much for connecting. Seems we're both very much into common interests .
I look forward to some great conversations!

Quick Overview Video



Settings

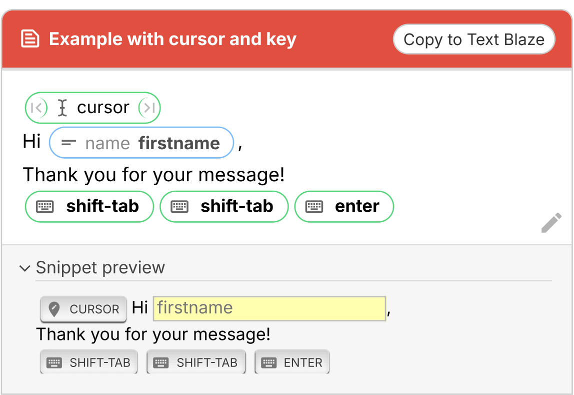
Setting Name	Type	Description
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Only one `cursor` command is allowed in a snippet.

Text Blaze allows you to specify where the cursor should end up after the snippet insertion has completed. The mouse cursor is moved to the position of where the command is within the snippet.

Example

One useful thing to do with the cursor command is to place it either at the start or the end of a snippet. You can then combine it with the `{key: tab}` or `{key: shift-tab}` command to more easily navigate the page.



cursor
Hi name **firstname** ,
Thank you for your message!
shift-tab **shift-tab** **enter**

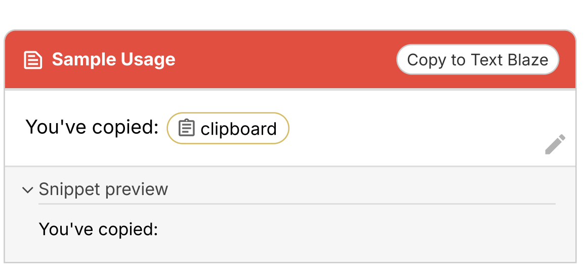
Snippet preview

CURSOR Hi firstname,
Thank you for your message!
SHIFT-TAB SHIFT-TAB ENTER

This is an example where you might need to insert a message first and then do something else in the web page afterwards with autopilot commands.

{clipboard}

Summary: Inserts the current clipboard contents.

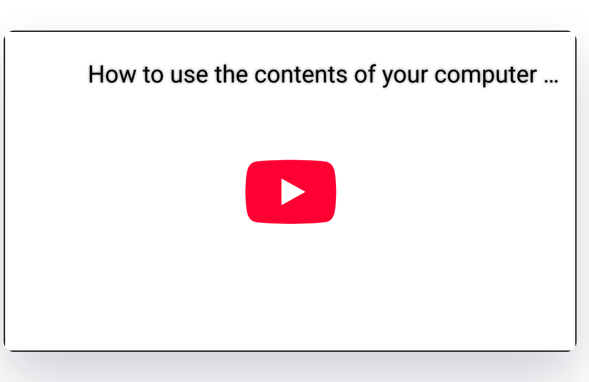

Sample Usage Copy to Text Blaze

You've copied: clipboard

Snippet preview

You've copied:

Quick Overview Video


How to use the contents of your computer ...

Settings

Setting Name	Type	Description
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `clipboard` command will insert the entire contents of your clipboard.

In many cases, you will only want to insert a part of the contents of your clipboard.

Referencing a part of the clipboard

A very common use case with the `clipboard` command is only referencing a specific part of the data in your clipboard.

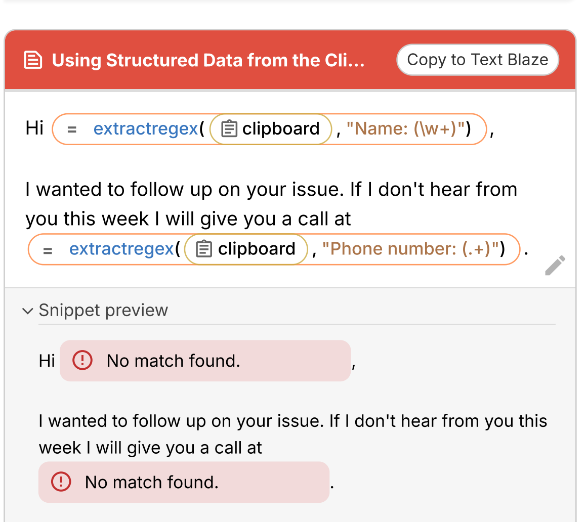
For example, you may have a customer case management system where data could be copied in the following format:

```
Name: John
Phone number: (555) 123-4567
Problem: User cannot connect to the website.
```

If you wanted to create a snippet that automatically used the name and phone number, you could do that using the `clipboard` command combined with the {=} (Formula Command) command to reference specific fields.

The following example shows this.

> To see this example work, please do the following:
>
> 1. Copy the text in the text box above
> 2. Reload this page so that the example uses your updated clipboard.
> 3. You will need to grant this website permission to access your clipboard.


Using Structured Data from the Cli... Copy to Text Blaze

Hi = extractregex(clipboard , "Name: (\w+)") ,

I wanted to follow up on your issue. If I don't hear from you this week I will give you a call at = extractregex(clipboard , "Phone number: (.+)") .

Snippet preview

Hi No match found. ,

I wanted to follow up on your issue. If I don't hear from you this week I will give you a call at No match found. .

The example above uses the `extractregex` function. This function uses a regular expression to precisely define what items of the clipboard you want to use. You can learn more about regular expressions in Text Blaze here.

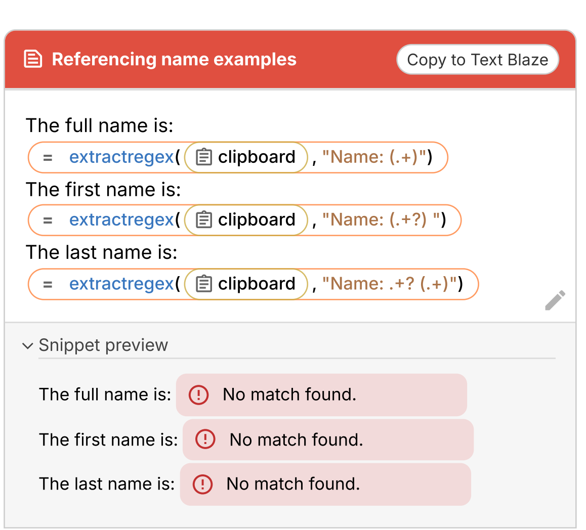
Referencing specific data in your clipboard with regular expressions

The following examples assume your computer's clipboard contains the following text:

```
Name: John Smith
Age: 32
ID: RFS4215
```

> If you have not yet copied the text and reloaded this page, please do so now.
>
> If you don't, you are going to see a lot of "No match found" errors in the following examples.

To reference the name from this text, we could use the following regular expressions:

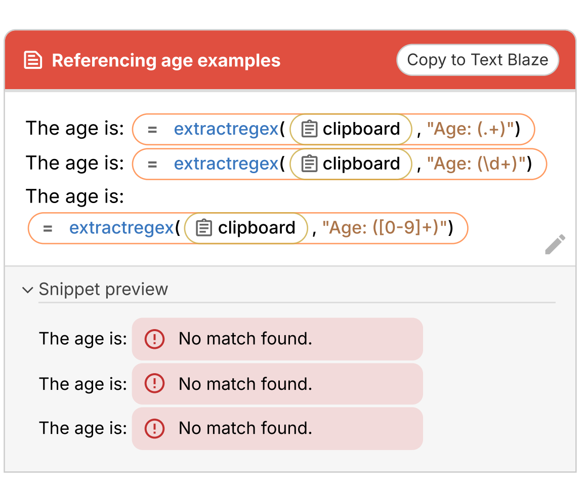

Referencing name examples Copy to Text Blaze

The full name is: = extractregex(clipboard , "Name: (.+)")
The first name is: = extractregex(clipboard , "Name: (.+?) ")
The last name is: = extractregex(clipboard , "Name: .+? (.+)")

Snippet preview

The full name is: No match found.
The first name is: No match found.
The last name is: No match found.

Now let's write a regular expression to get the user's age. Here we show a couple different approaches:


Referencing age examples Copy to Text Blaze

The age is: = extractregex(clipboard , "Age: (.+)")
The age is: = extractregex(clipboard , "Age: (\d+)")
The age is: = extractregex(clipboard , "Age: ([0-9]+)")

Snippet preview

The age is: No match found.
The age is: No match found.
The age is: No match found.

There are many ways to write a regular expression to grab a specific result that you're looking for.

> Diving in-depth into Regular Expressions is out of scope for this documentation page.
>
> To learn more, visit the section on Regular Expressions in the Blaze Formula Reference page.

Handling badly formatted data

Occasionally, you will try to use a regular expression on data that is incorrectly formatted so that the regular expression does not correctly match the data.

For example, imagine the following data was given to you:

```
Name:
Age: 32
ID: RFS4215
```

The data is missing the name. If you use the regular expression from the previous section to process this you'll get a "No match found" error.

Sometimes you want to explicitly want an error to occur, but other times you want to insert a fallback value when the regular expression match fails.

There are two basic ways to handle these cases:

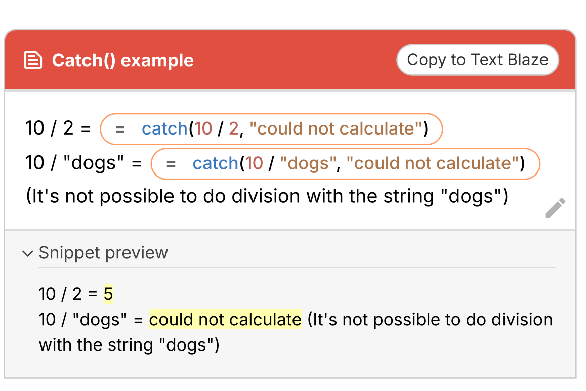
1. Using the `catch()` function.
2. Using the {if} command and the `testregex` function.

Handling errors with the `catch()` function

The `catch()` function evaluates its first argument and returns it.

However, if an error occurs when it is evaluating its first argument, it will return the second argument as a fallback.

Here is a quick example of the `catch()` function in action:

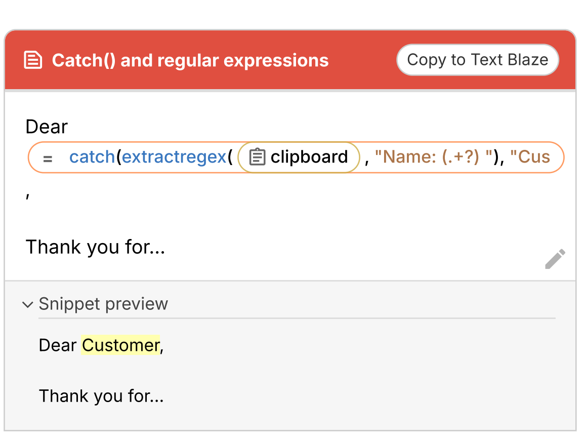

Catch() example Copy to Text Blaze

10 / 2 = = catch(10 / 2, "could not calculate")
10 / "dogs" = = catch(10 / "dogs", "could not calculate")
(It's not possible to do division with the string "dogs")

Snippet preview

10 / 2 = 5
10 / "dogs" = could not calculate (It's not possible to do division with the string "dogs")

Here's another example using the `catch()` function together with a regular expression:


Catch() and regular expressions Copy to Text Blaze

Dear
= catch(extractregex(clipboard , "Name: (.+?) "), "Cus
,

Thank you for...

Snippet preview

Dear Customer,

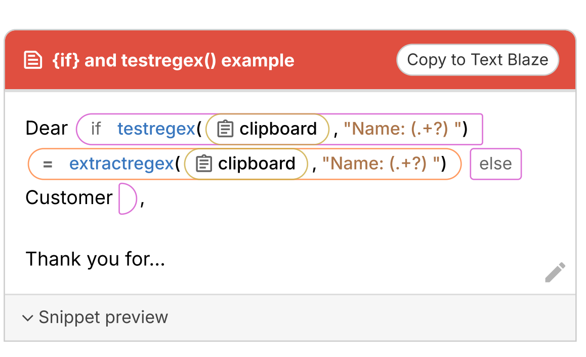
Thank you for...

If the contents of your clipboard contains the correctly formatted text "Name: John", for example, the snippet will output "Dear John".

If the regular epression above does not match with the contents of the clipboard, then there will be an error. In that case the `catch()` command's second argument will output. It will display "Dear Customer".

Handling errors with the `if` command and the `testregex()` function

The `testregex()` function returns a Boolean (yes/no) which can be used as a condition in the {if} command.


{if} and testregex() example Copy to Text Blaze

Dear if testregex(clipboard , "Name: (.+?) ")
= extractregex(clipboard , "Name: (.+?) ") else
Customer ,

Thank you for...

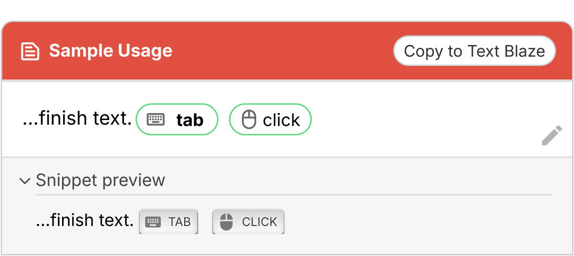
Snippet preview

That's a very short introduction to using regular expressions together with the `clipboard` command to only grab the data that you need.

To go more in-depth, see the Regular Expressions section of the Formula Reference page.

{click}

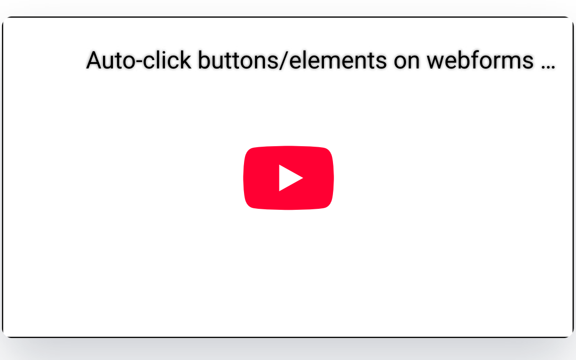
Summary: Simulates a click on any element on any open webpage.



...finish text. tab click

...finish text. TAB CLICK

Quick Overview Video



Settings

Setting Name	Type	Description
selector	selector	CSS selector to click on a specific element in the page.
maxdelay	time	The selected element is clicked as soon as it becomes clickable. If the selected element does not become clickable within 'maxdelay' time, the snippet stops and an error is shown.
frame	self/top	Controls which frame to start matching the given selector from (defaults to 'self').
page	string	When `page` is specified, the {click} command will switch to a tab whose URL matches the given `page` URL.
select	yes/ifneeded	Controls when to show the tab selector when `page` setting is specified. `yes` always shows it. `ifneeded` (default) shows it only when multiple tabs match the `page` pattern and the current page does not match.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{click}` command, an autopilot command, allows you to *emulate* a click on any element on any web page. This can be used to submit a form, focus another textbox or trigger other events in the page. You can use the `{click}` command in three approaches:

Click on the currently focused element

You can use the `{click}` command in conjunction with the {key} command, another autopilot command, to move the focus to another element in the page and then use the `{click}` command to click the element. Click here to learn more about moving the focus back and forth between different elements on a page.

For example: the snippet `{key: tab}{key: tab}{key: tab}{click}` will emulate the Tab key three times - changing the focused element on each step - and then emulate a click on the latest focused element.

Click on any element in the page

Use the "Select from website" button in the `{click}` command to select an element that you wish to click on. The Text Blaze snippet will click the selected element, which will then also become focused.

We recommend this approach for most users because it's very fast. It is useful to:

- Jump directly to another textbox in the page
  Example: running the snippet `Alpha{click: selector=selector}Beta` will insert "Alpha" in the first textbox (where you started your snippet), then jump to the second textbox (selected by "selector") and then insert "Beta" in the second textbox
- Click on consecutive checkboxes or buttons
  Example: When submitting a form, the snippet `{click: selector=button1}{click: selector=checkbox1}{click: selector=submit_button}` will click a button, then another checkbox, and then the submit button on the form.

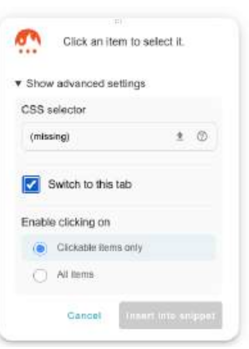
The given snippets are examples. Please use the "Select from website" button in the `{click}` command to generate a selector for your specific website.

Text Blaze will wait up to one second for an interactive element that matches the specified selector to appear on the page. If you want to wait longer, you can specify the `maxdelay` attribute. For example, the following command `{click: selector=selector; maxdelay=5s}` will wait up to five seconds to match the given selector to an element in the page.

Switch to another tab and then click an element in it

You can also use {click} to switch to another open tab and click an element there. Your snippet will continue running in that open tab. This lets you automate actions across multiple tabs with a single snippet.

To enable the cross-tab capability, check the "Switch to this tab" checkbox when setting up the {click} command's selector, under the "Show advanced settings" option, as shown in this image:



(missing)

☑ Switch to this tab

Enable clicking on

● Clickable items only

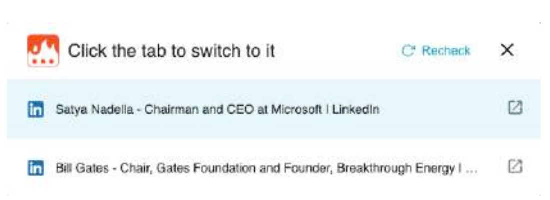
○ All items

For example, the following command switches to a tab matching `https://your-crm.com/*` and clicks the Submit button:

```
{click: selector=.submit; page=https://your-crm.com/*; frame=top}
```

When `page` setting is specified, the default behavior of {click} is as follows:

- If your current page matches the `page` pattern, the snippet will continue to run on the current page
- If only one other tab matches the `page` pattern, the snippet will switch to that tab automatically, and continue to run there
- If multiple tabs match the `page` pattern, you will see a tab selector dialog as shown in the image below:



Satya Nadella - Chairman and CEO at Microsoft | LinkedIn

Bill Gates - Chair, Gates Foundation and Founder, Breakthrough Energy | ...

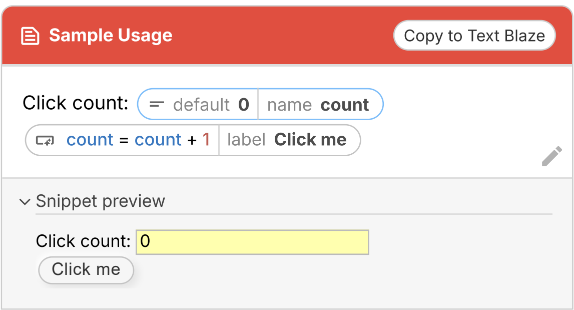
To always show the tab selector, set the `select` setting to 'yes' in your command, as shown in the example below:

```
{click: selector=.submit; page=https://your-crm.com/*; frame=top; select=yes}
```

To learn more about automating actions on a web page, see the **autopilot guide**.

{button}

Summary: Runs a code block when clicked.



Click count: default 0 name count
count = count + 1 label Click me

Click count: 0
Click me

Settings

Setting Name	Type	Description
Positional	code	The code block to run when the button is clicked.
label	text	The label for the button.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{button}` command adds an interactive button to your snippet. When clicked, the button runs a code block.

Buttons do not insert any content when the snippet is inserted. They are only for building interfaces in form snippets.

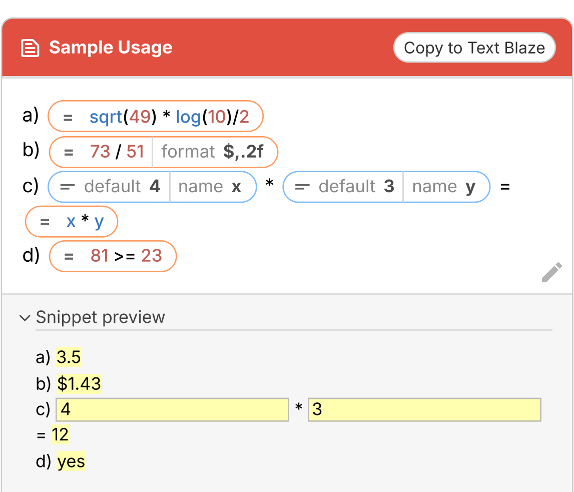
Buttons are often used with the {run} command which allows you to execute a code block once.

Learn more about code blocks in the code block reference.

> Button commands are in Beta and may change in the future.

{=} (Formula Command)

Summary: Dynamic updating formula that can use and create variables

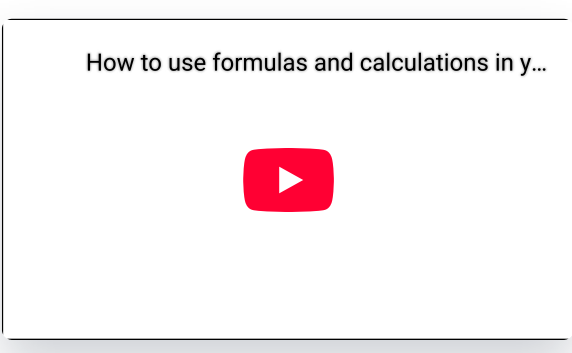


a) = sqrt(49) * log(10)/2
b) = 73 / 51 format $,.2f
c) = default 4 name x * = default 3 name y =
= x * y
d) = 81 >= 23

Snippet preview

a) 3.5
b) $1.43
c) 4 * 3
= 12
d) yes

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	formula	The formula.
format	text	How to format a numeric output of the formula.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The Formula Command is a special command in Text Blaze that allows you to use dynamic math, create variables, and utilize variables previously created throughout the snippet.

Also, notice how this command looks slightly different from other commands. It uses an equal sign between the `{}` instead of a word like the other commands.

Despite the difference in looks, the formula command has the same syntax as other commands. The first setting, a positional setting, takes a Text Blaze formula. The second optional setting, `format`, allows you to format numeric outputs.

Blaze Formula Reference

The {=} (Formula Command), {if}, and {repeat} commands, all utilize the **Blaze Formula Syntax** to let you use formulas and calculations as part of your snippets. There are features like predefined functions, lists, regular expressions, and more.

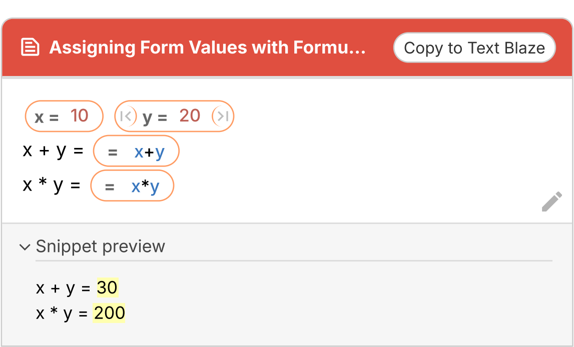
Visit the Blaze Formula Reference page and learn about what's possible.

Creating Variables With Assignment Mode

The formula command can also be used to create variables which can then be used in your other formulas within the same snippet.

To use it this way put the name of the variable before the formula command's = sign.

When used in assignment mode, the formula command won't display any output.



x = 10 y = 20
x + y = = x+y
x * y = = x*y

Snippet preview

x + y = 30
x * y = 200

To learn more about using variables in your snippets, visit the Blaze Formula Reference page.

It's also possible to use the {=} (Formula Command) command in combination with forms and form values.

Formatting numbers with the `format` setting

The optional `format` setting allows you to control how numeric outputs are formatted in formulas. The formatting will attempt to use the user's locale specified in the Text Blaze account settings.

The following described the available options:

```
[[fill]align][sign][symbol][0][width][,][.precision][type]
```

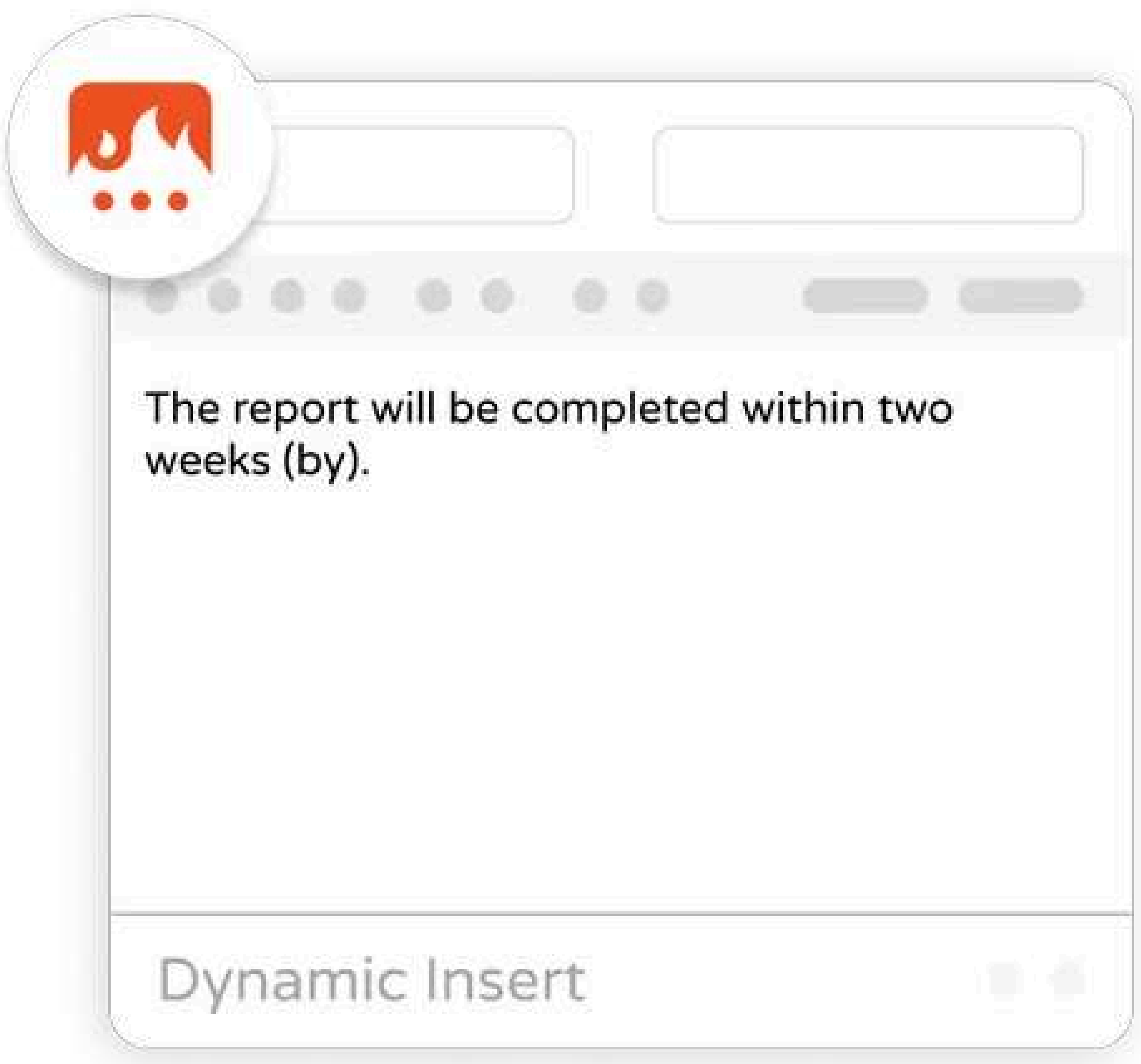
> **The output number formatting syntax and the blaze formula syntax are not the same.**
>
> More information and examples on output number formatting can be found here.

It is useful to see examples of format definitions and the resulting outputs. The following table shows how different formats would be applied to different sample numbers.

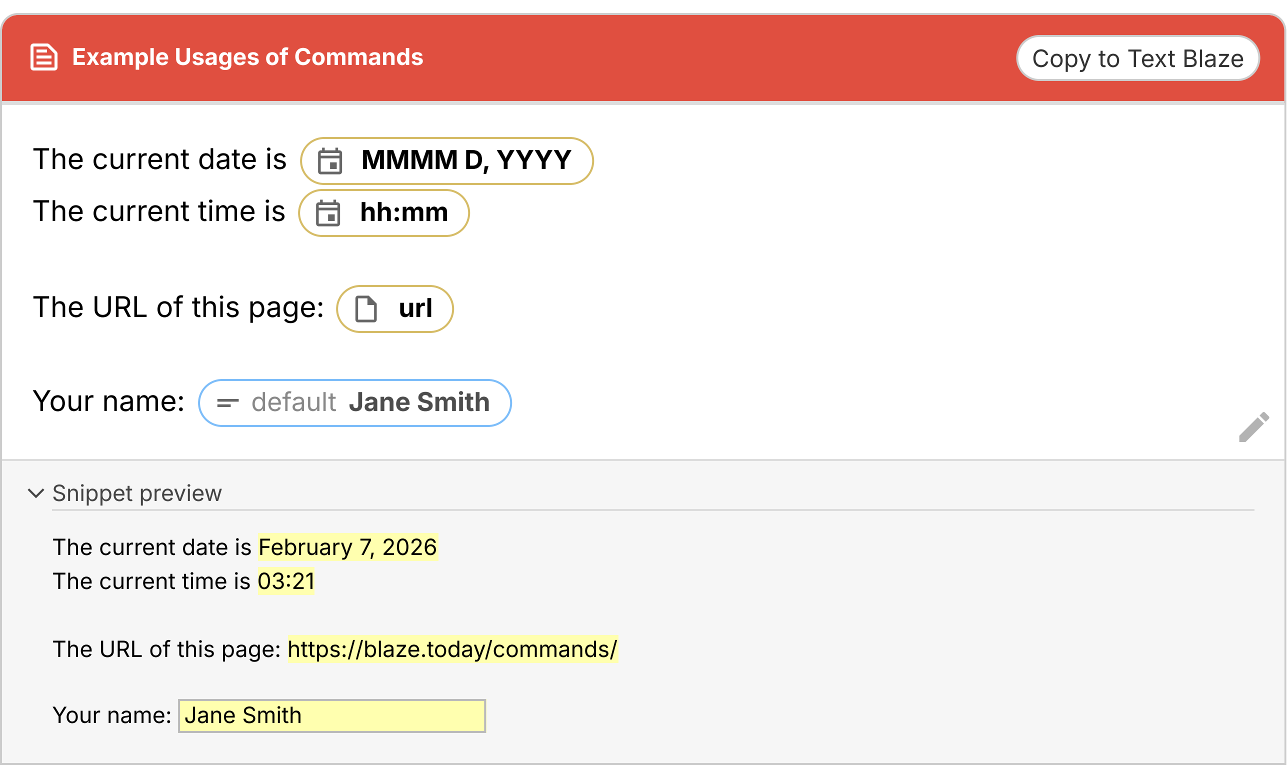
Format	Applied to `0.934`	Applied to `1245.67`
s	934.2m	1.24567k
,	0.9342	1,245.67
,%	93%	124,567%
+,	+0.9342	+1,245.67
.3r	0.934	1250
.3n	0.934	1.25e+3
.0f	1	1246
.1f	0.9	1245.7
.1e	9.3e-1	1.2e+3
$,.0f	$1	$1,246

Using Dynamic Commands

Text Blaze commands allow you to create dynamic snippets. Commands are replaced when the snippet is inserted with a dynamic value or action. For example, the {time} command lets you insert times and dates.


The report will be completed within two weeks (by).
Dynamic Insert

Here are a some command examples:



The current date is MMMM D, YYYY
The current time is hh:mm

The URL of this page: url

Your name: default Jane Smith

Snippet preview

The current date is February 7, 2026
The current time is 03:21

The URL of this page: https://blaze.today/commands/

Your name: Jane Smith

Commands are made up of two parts: the command name and the command settings which configure how the command behaves. A command and its settings are enclosed by a `{` and `}`.

For example {cursor} will move the mouse cursor after the snippet has been entered to the position where you entered the command.

Command Settings

Some commands also require settings to work correctly. There are two types of settings: **positional settings and named settings**.

For example, `{link: https://example.com/product/}Product{endlink}` inserts a link into the current snippet. The command is {link} and *"https://example.com/product/"* is a required positional setting telling Text Blaze which link to display. The text between the start command and the end command is how the link will display.

All settings are separated by semicolons (;). Positional settings come immediately after the colon following the name of the command as the first setting. They only require a value.

There can also be multiple named settings on a command. Named settings require you to insert the name of the setting followed by "=" and then the value.

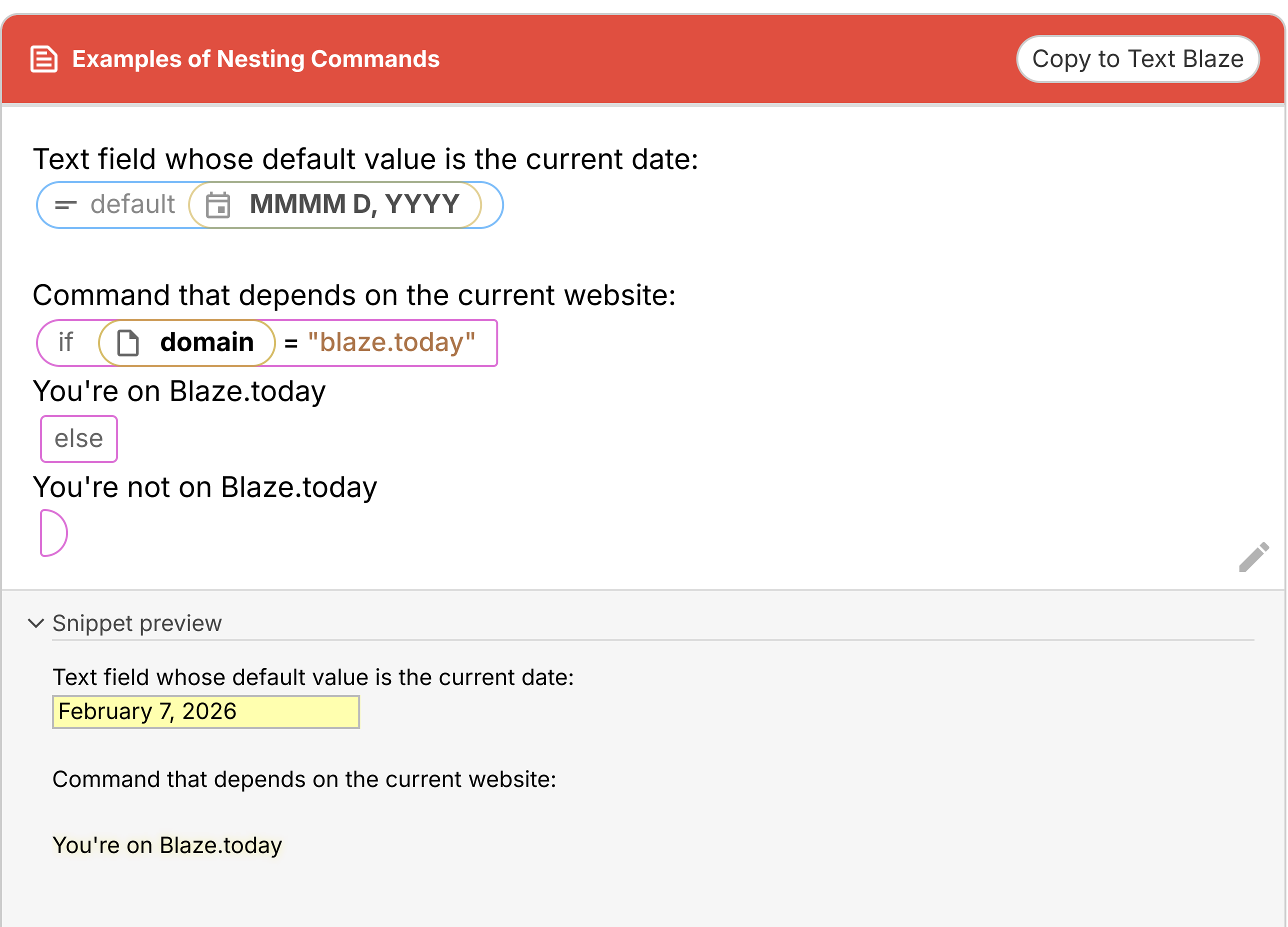
One example is `{time: YYYY; at=2000-01-01; shift=+3Y}` which displays the time. Here the command name is {time} and there are three settings. One is a positional setting and there are two named settings.

The first positional setting, `"YYYY"`, tells Text Blaze how to format the date ("YYYY" means show the current date's four digit year). The second named setting, `at`, is an optional named setting that allows you to explicitly set the date for the {time} command. If it's excluded it will use the current time. Finally, the last named setting, `shift`, tells Text Blaze to move the date forwards or backwards by some specified amount.

Commands can mix positional settings and named settings. There can also be multiple settings per command. Also, some settings are optional. See the documentation for each command to see which settings are optional and which are required.

Nesting Commands

An important feature of commands is that you can nest them within each other. For instance, you can use a {time} command as the default value for a {formtext} command or {site} command as part of the condition for an {if} command.



Text field whose default value is the current date:
default MMMM D, YYYY

Command that depends on the current website:
if domain = "blaze.today"
You're on Blaze.today
else
You're not on Blaze.today

Snippet preview

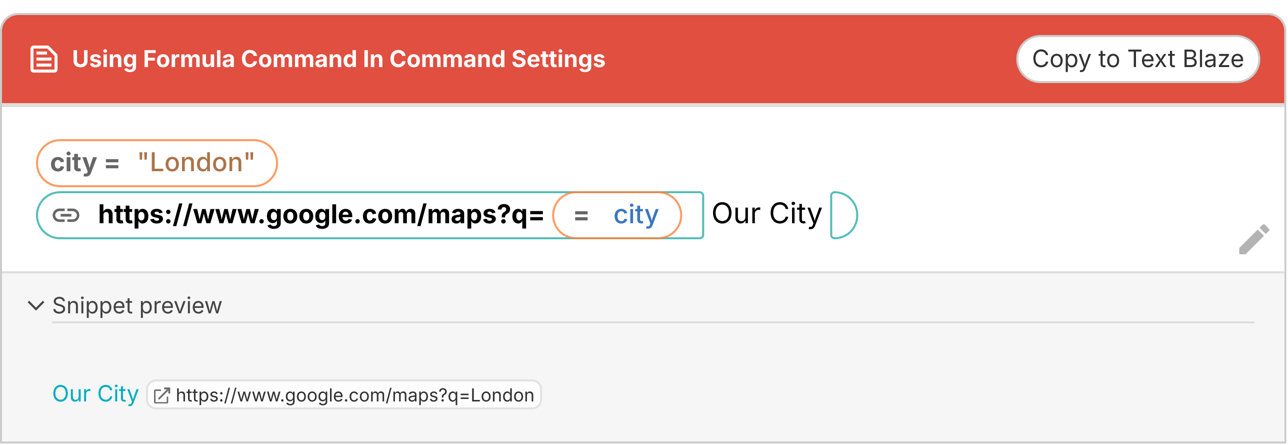
Text field whose default value is the current date:
February 7, 2026

Command that depends on the current website:

You're on Blaze.today

It's also possible for you to use the formula command in your commands for dynamic math or to utilize variables previously saved throughout the snippet.

For example, if you have a saved city or address in one your snippets you can utilize that to dynamically generate a Google Maps link.



city = "London"
https://www.google.com/maps?q= = city Our City

Snippet preview

Our City https://www.google.com/maps?q=London

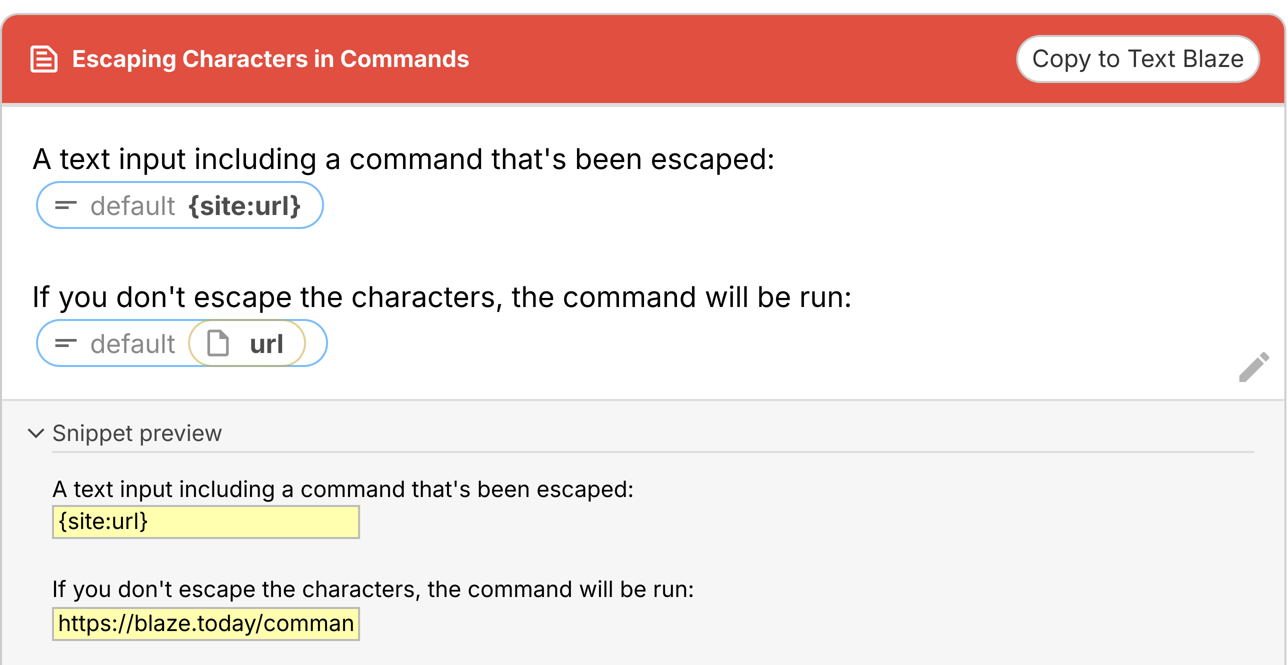
Nesting allows you to create powerful behavior be composing commands together.

Escaping Special Characters in Commands

Some characters have special meanings in commands. Primarily "{", "}", "=", and ";". If you want to use these special characters normally, you can do so by "escaping" them: by placing a backslash "" before them. For example this text input will have the default value "{site:url}":

```
{formtext: default=\{site:url\}}
```

If you didn't escape the `{` and `}`, you would get a different result as shown in the following example:



A text input including a command that's been escaped:
default {site:url}

If you don't escape the characters, the command will be run:
default url

Snippet preview

A text input including a command that's been escaped:
{site:url}

If you don't escape the characters, the command will be run:
https://blaze.today/comman

Non-escaped whitespace around a setting will be trimmed. If you want a setting value to start or end with whitespace, you need to escape the whitespace by placing a backslash in front of it.

Available Commands

{wait}

Summary: Pauses snippet insertion for a set period of time.

Sample Usage Copy to Text Blaze

...the notice. ⌨ tab 🖱 click ⏱ delay 10s Also it is important that...

Snippet preview

...the notice. ⌨ TAB 🖱 CLICK ⏱ WAIT 10S Also it is important that...

Settings

Setting Name	Type	Description
delay	text	An optional delay period. If omitted, defaults to `1s` (1 second).
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `delay` setting uses the same syntax to specify the delay as the `shift` setting of the {time} command. Click here to learn about time shifting in Text Blaze.

The `{wait}` command is useful when constructing Autopilot scripts where you need to wait for some change to occur on the underlying page (e.g. for it to finish loading data).

Examples

Wait Example Copy to Text Blaze

⌨ tab ⌨ enter ⏱ delay 2s ⌨ tab

Snippet preview

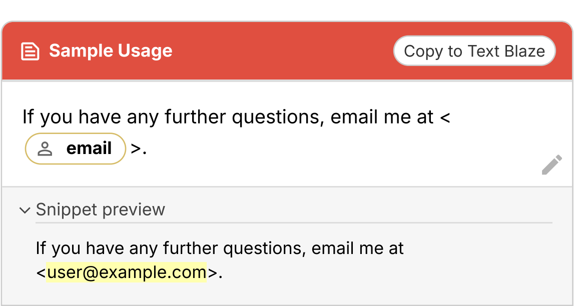
⌨ TAB ⌨ ENTER ⏱ WAIT 2S ⌨ TAB

The above example might be one where you tab to an enter button, press enter, the website loads an additional part of the form, and then you tab to the rest of the form.

To learn more about automating actions in a web page, see the **autopilot guide**.

{user}

Summary: Inserts information about the current user



If you have any further questions, email me at < email >.

If you have any further questions, email me at <user@example.com>.

Settings

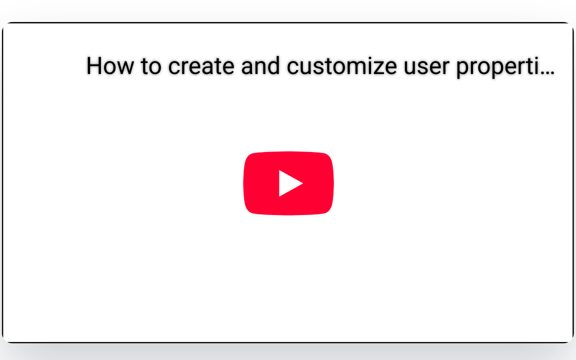
Setting Name	Type	Description
Positional	text	The user property to get
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

> The `{user}` command is a Text Blaze Business feature and **may only be used by teams using Text Blaze Business and Enterprise**.

The `{user}` command allows Text Blaze Business users to include information, known as **properties**, about the current *snippet user* in the snippet.

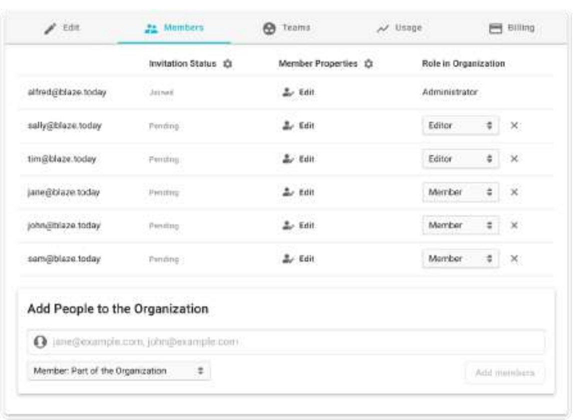
In this case, the *snippet user* is the member of your organization that's using the snippet.

Defining your own properties



You can define *custom properties* for each member of your organization.

Custom properties can be defined in the *Members* tab of the organization and clicking on the gear icon beside the *Member Properties* column header.



	Invitation Status	Member Properties	Role in Organization
alfred@blaze.today	Joined	Edit	Administrator
sally@blaze.today	Pending	Edit	Editor
tim@blaze.today	Pending	Edit	Editor
jane@blaze.today	Pending	Edit	Member
john@blaze.today	Pending	Edit	Member
sam@blaze.today	Pending	Edit	Member

Add People to the Organization

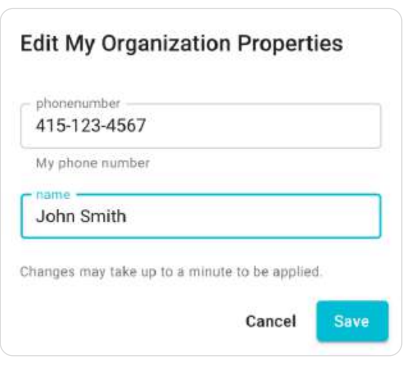
jane@example.com, john@example.com

Member: Part of the Organization

For example, you may want to include employees' names in your organization's snippets. You can do that by first creating a custom `name` property.

Now you can configure the newly defined `name` property for each employee individually by clicking on the `Edit` button next to their name under the *Member Propeties* column.

Here is an example of configuring two properties for a member of your organization.


Edit My Organization Properties

phonenumber: 415-123-4567

My phone number

name: John Smith

Changes may take up to a minute to be applied.

Editing Properties

Organization administrators and editors **can modify properties for all organization members** from the Members tab.

If the property is *user editable*, individual members can modify their own properties.

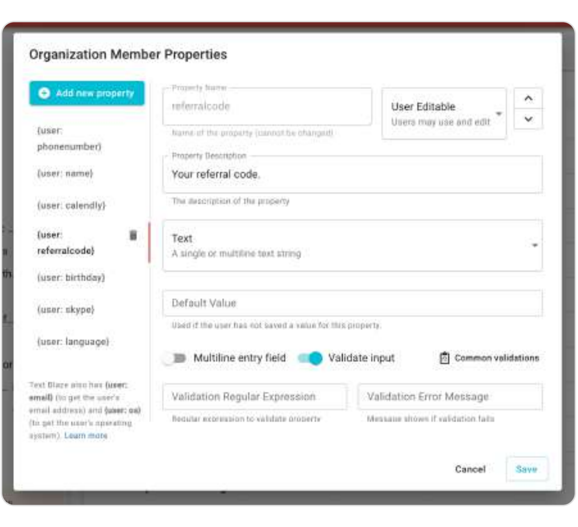
Setting a property's name and description

The property name you use is how you will reference that property in your snippets. If your property name is `department`, you reference the value in a snippet with `{user: department}`.

The capitalization that you use is important. For example, `{user: department}` will work, but `{user: Department}` will not.

Text Blaze **does not allow you to change a property's name** once set initially.

You can also optionally add a description that allows you to provide additional information to users.


Organization Member Properties

{user: phonenumber}
{user: name}
{user: calendly}
{user: referralcode}
{user: birthday}
{user: skype}
{user: language}

Property Name: referralcode

Name of the property (cannot be changed)

User Editable: Users may use and edit

Property Description: Your referral code.

The description of the property

Text: A single or multiline text string

Default Value

Used if the user has not saved a value for this property.

Multiline entry field

Validate input

Common validations

Validation Regular Expression

Regular expression to validate property

Validation Error Message

Message shown if validation fails

Text Blaze also has {user: email} (to get the user's email address) and {user: os} (to get the user's operating system). Learn more

Setting a property's data type and default value

The default value that you set for the property is used if an administrator, editor, or individual member has not yet configured a specific value for the property.

You can also configure the **data type** of a property.:

There are **four** data types available:

1. **Text**: Text properties can be used to represent things like names or phone numbers. You can also specify validation rules using regular expressions.
2. **Number**: Number properties only accept numeric values. You can configure maximum and minimum allowed values.
3. **Boolean**: Boolean properties are yes/no values that can be turned on and off.
4. **Categorical**: Categorical properties allow you to specify a drop-down list of options that the user may select an option from.
5. **Dates**: Date properties that allow you to select a specific date from a date field.

> **If the data type of an existing property is changed, or new validations are added to it, *existing values* for it will not be changed even if they are no longer valid**.
>
> The newly changed data type and validations will only apply when manually updated the values.

Built-in properties

Besides allowing you to define your own custom properties, each member of your organization will come with built-in properties by default.

Property	Description	Example
email	The user's email.	`user@example.com`

Using `{user: email}` outputs the email that the user signed up to Text Blaze to join your organization with.

{time}

Summary: Powerful and configurable time and date formatting.

Sample Usage Copy to Text Blaze

It is currently: dddd, MMMM Do, YYYY
More briefly: ddd, MMM Do, YYYY
Or even: M/D/YY

The current time is: HH:mm:ss A
More briefly: h:mm

The time and date are:
ddd, MMM Do, YYYY [at] h:mm A

Since it is ddd, MMM Do, YYYY, in 12 days it will be ddd, MMM Do, YYYY shift +12D.

Snippet preview

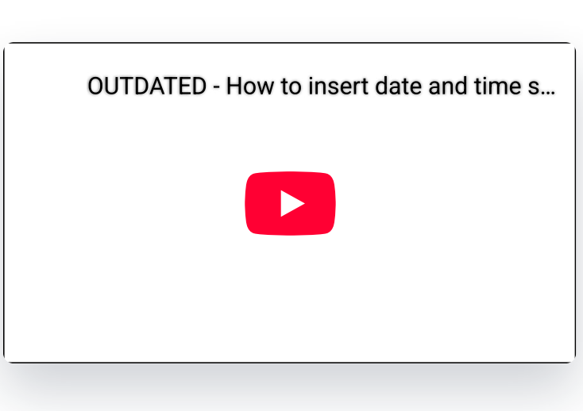
It is currently: Saturday, February 7th, 2026
More briefly: Sat, Feb 7th, 2026
Or even: 2/7/26

The current time is: 03:30:46 AM
More briefly: 3:30

The time and date are: Sat, Feb 7th, 2026 at 3:30 AM

Since it is Sat, Feb 7th, 2026, in 12 days it will be Thu, Feb 19th, 2026.

Quick Overview Video


OUTDATED - How to insert date and time s...

Settings

Setting Name	Type	Description
Positional	text	The date/time format.
at	text	Set's the date/time.
pattern	text	Format used to interpret `at`.
shift	text	An optional date/time shift.
locale	text	An optional locale code (e.g. `en-us` or just `en`) used to format the time and date.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Time/Date Formatting

Text Blaze's time/date formatting syntax **allows you to customize the output of your times and dates in various ways**. It allows you to mix formatting with *tokens* that have special meanings.

For example, one way the current date could be entered is `MMM Do, YYYY` (resulting in the output of `Nov 13th, 2017`), and the current time could be specified with `h:mm` (resulting in the output of `11:24`).

Text Blaze's time/date formatting syntax is based on Moment, a JavaScript library for parsing and formatting dates.

The following table (adapted from the Moment.js documentation website) contains the full list of support tokens:

Token	Output
Month	
M	1 2 ... 11 12
Mo	1st 2nd ... 11th 12th
MM	01 02 ... 11 12
MMM	Jan Feb ... Nov Dec
MMMM	January February ... November December
Quarter	
Q	1 2 3 4
Qo	1st 2nd 3rd 4th
Day of Month	
D	1 2 ... 30 31
Do	1st 2nd ... 30th 31st
DD	01 02 ... 30 31
Day of Year	
DDD	1 2 ... 364 365
DDDo	1st 2nd ... 364th 365th
DDDD	001 002 ... 364 365
Day of Week	
d	0 1 ... 5 6
do	0th 1st ... 5th 6th
dd	Su Mo ... Fr Sa
ddd	Sun Mon ... Fri Sat
dddd	Sunday Monday ... Friday Saturday
Day of Week (Locale)	
e	0 1 ... 5 6
Day of Week (ISO)	
E	1 2 ... 6 7
Week of Year	
w	1 2 ... 52 53
wo	1st 2nd ... 52nd 53rd
ww	01 02 ... 52 53
Week of Year (ISO)	
W	1 2 ... 52 53
Wo	1st 2nd ... 52nd 53rd
WW	01 02 ... 52 53
Year	
YY	70 71 ... 29 30
YYYY	1970 1971 ... 2029 2030
AM/PM	
A	AM PM
a	am pm
Hour	
H	0 1 ... 22 23
HH	00 01 ... 22 23
h	1 2 ... 11 12
hh	01 02 ... 11 12
k	1 2 ... 23 24
kk	01 02 ... 23 24
Minute	
m	0 1 ... 58 59
mm	00 01 ... 58 59
Second	
s	0 1 ... 58 59
ss	00 01 ... 58 59
Locale Formatted	
LT	*Ex:* 7:10 AM
LTS	*Ex:* 7:10:46 AM
l	*Ex:* 9/22/1985
L	*Ex:* 09/22/1985
ll	*Ex:* Sept 22, 1985
LL	*Ex:* September 22, 1985
lll	*Ex:* Sept 22, 1985 7:10 AM
LLL	*Ex:* September 22, 1985 7:10 AM
llll	*Ex:* Mon, Sept 22, 1985 7:10 AM
LLLL	*Ex:* Monday, September 22, 1985 7:10 AM
Unix Timestamp	
X	1360013296
Unix Millisecond Timestamp	
x	1360013296123

> The Text Blaze time/date formatting syntax above is also used in the {formdate} command.

Setting the Time/Date

The optional `at` setting allows you to specify the time/date used by the time command. **If the `at` setting is not explicitly set, the current time/date will be used.**

Using "at" Copy to Text Blaze

The date at 2000-01-01 was:
dddd MMM D, YYYY at 2000-01-01.

Three year later it was:
dddd MMM D, YYYY shift +3Y at 2000-01-01.

Snippet preview

The date at 2000-01-01 was: Saturday Jan 1, 2000.

Three year later it was: Wednesday Jan 1, 2003.

The `at` setting will interpret many common date formats. If the `at` format does not recognize your date format, or your date format is ambiguous, or you want to specify a time; you can use the `pattern` setting to tell Text Blaze how to understand your date.

The `pattern` setting uses the Text Blaze time/date formatting syntax to tell Text Blaze how your time/date is structured.

For example, if your date was 2000-01-01 where you have Year-Month-Day, you would use a pattern of `YYYY-MM-DD`. If your date was instead Year-Day-Month, you could use the pattern `YYYY-DD-MM`.

Using a Pattern Copy to Text Blaze

An example using a custom formatted data "2000.01.01":
ll at 2000.01.01 pattern YYYY.MM.DD.

Another example with a different custom formatted date "7 Jan (2012)":
ll at 7 Jan (2012) pattern D MMM (YYYY).

Snippet preview

An example using a custom formatted data "2000.01.01": Jan 1, 2000.

Another example with a different custom formatted date "7 Jan (2012)": Jan 7, 2012.

Time/Date Shifting

The optional `shift` setting lets you display a time/date in either the future or past. A time shift is specified to select a time period relative to the current time.

For example, to show the date 10 days from now, you could use `{time: MMM Do, YYYY; shift=+10D}`.

If the `at` setting is specified, any time shifts will be relative to that date.

Shifts are specified in terms of a base period like days or weeks. You can have a fixed shift like `+2W` (two weeks from the current time) or `-5D` (five days ago). You can also shift to the beginning or end of any standard date period. For example, `<M` will shift to the start of the current month, while `>Y` will shift to the end of the current year.

The following table summarizes the shifts relative to the standard periods.

Period	Base Period	Example Fixed Shift	Start and End Period Shift
Seconds	s	`5s`	`<s` and `>s`
Minutes	m	`-4m`	`<m` and `>m`
Hours	h	`24h`	`<h` and `>h`
Days	D	`2D`	`<D` and `>D`
Weeks	W	`5W`	`<W` and `>W`
Months	M	`-10M`	`<M` and `>M`
Quarters	Q	`1Q`	`<Q` and `>Q`
Years	Y	`3Y`	`<Y` and `>Y`

You can also shift forward or backward to a specific day of the week. For example, `>MON` will shift forward to the nearest Monday. If the current date is a Monday, no shift will take place. `<MON` will shift backwards to the nearest Monday. Again, if the current date is a Monday, no shift will take place.

Day	Label	Current or Previous Shift	Current or Following Shift
Sunday	SUN	`<SUN`	`>SUN`
Monday	MON	`<MON`	`>MON`
Tuesday	TUE	`<TUE`	`>TUE`
Wednesday	WED	`<WED`	`>WED`
Thursday	THU	`<THU`	`>THU`
Friday	FRI	`<FRI`	`>FRI`
Saturday	SAT	`<SAT`	`>SAT`

> **You can combine multiple shifts together**. For example, `+10Y -2D` would shift the date by a total of 10 years minus 2 days. `+10D >MON` would shift the date 10 days and then to the following Monday.
>
> This is particularly useful if you want to implement something like "find the next Monday". If you just used `>MON` and the current day is a Monday, it will not shift at all. You instead could use `+1D >MON` to increment the day by one, and then move forward to the nearest Monday.

The following examples show how you might use date shifting to specify specific dates.

Date/Time Shifting Copy to Text Blaze

Current date: MMMM D, YYYY
In ten days it will be: MMMM D, YYYY shift +10D
Ten days ago it was: MMMM D, YYYY shift -10D

The first day of the month was:
MMMM D, YYYY shift <M
The last day of the month will be:
MMMM D, YYYY shift >M

The last Friday of the quarter is:
MMMM D, YYYY shift >Q <FRI
The first Monday of next year will be:
MMMM D, YYYY shift +1Y <Y >MON

Snippet preview

Current date: February 7, 2026
In ten days it will be: February 17, 2026
Ten days ago it was: January 28, 2026

The first day of the month was: February 1, 2026
The last day of the month will be: February 28, 2026

The last Friday of the quarter is: March 27, 2026
The first Monday of next year will be: January 4, 2027

When shifting by days, you can skip certain days of the week. For example, if you wanted to shift 10 weekdays into the future you could use the following:

Skipping specific days of the week Copy to Text Blaze

In ten weekdays it will be:
MMMM D, YYYY shift +10D(skip=SAT, SUN)

Snippet preview

In ten weekdays it will be: February 20, 2026

Locales and Localization

It's possible to override the locale for individual `{time}` commands using the `locale` setting.

If not specified, `{time}` command will attempt to use the locale specified in your Text Blaze account settings when formatting the time/date.

A list of available language locales are available in your Account Settings page under the **Locale & Formatting** header.

Locales Copy to Text Blaze

In America, the current date and time might be written:
LLLL locale en-us

In France, we might use:
LLLL locale fr

And in China, we might see:
LLLL locale zh

Snippet preview

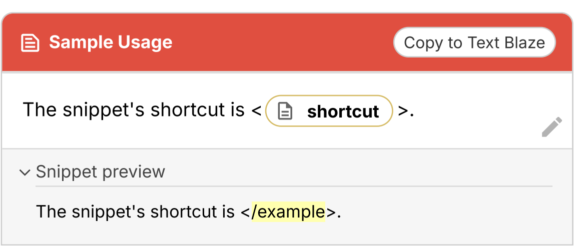
In America, the current date and time might be written:
Saturday, February 7, 2026 3:30 AM

In France, we might use:
samedi 7 février 2026 03:30

And in China, we might see:
2026年2月7日星期六凌晨3点30分

{snippet}

Summary: Inserts information about the current snippet in the snippet itself



The snippet's shortcut is <shortcut>.

The snippet's shortcut is </example>.

Settings

Setting Name	Type	Description
Positional	text	The snippet property to get
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{snippet}` command allows you to **include information about the snippet itself** within the snippet.

If you're looking to **import** another snippet into your current snippet, that would be the {import} command.

The `{snippet}` command takes a single positional setting that supports **three** different values.

Property	Description	Example
id	The snippet's id.	`xyz987`
shortcut	The snippet's shortcut.	`/example`
trigger	The snippet's shortcut as it was typed to trigger the insertion.	`/Example`

Capitalization is important. Each property must be typed **in lowercase** into the command. For example, `{snippet: id}` will work, but `{snippet: ID}` will not.

> The `id` is a unique identifier for the snippet set automatically when the snippet is created. It will not change even if the snippet's name or shortcut is changed.

Using the {snippet} command and the {import} command together

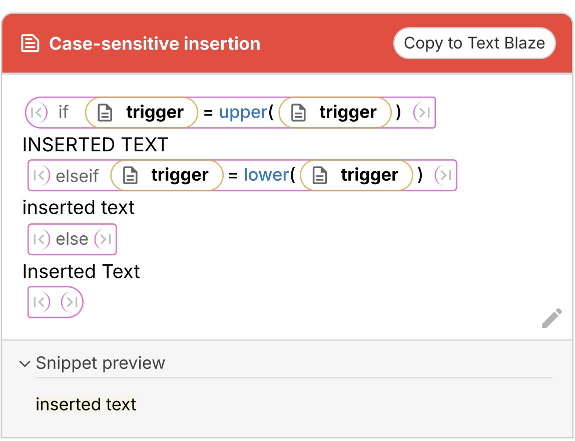
The `{snippet}` command will always refer to the snippet that was initially triggered by the user **even if the command is nested within an imported snippet**.

This means that if the `{snippet}` command is used within an {import} command, the `{snippet}` command will still refer to the initial snippet that was triggered, not the snippet being imported with the {import} command.

Case-sensitive insertion

`{snippet: trigger}` will return the shortcut exactly as it was typed including capitalization. This can be used to implement case-sensitive replacements.

For example, the following will insert the text "inserted text" while matching the capitalization of the shortcut. For example, if the shortcut was "/go" and you typed "/GO" it would insert "INSERTED TEXT". On the other hand, if you typed "/Go", it would insert "Inserted Text".



if trigger = upper(trigger)
INSERTED TEXT
elseif trigger = lower(trigger)
inserted text
else
Inserted Text

inserted text

See the {if} command for more information on using conditional logic in your snippets.

`upper()` is a predefined function available to you in Text Blaze. See the Blaze Formula Language Reference for more information on the various predefined functions available to you.

{site}

Summary: Inserts information from any open webpage into your snippet.

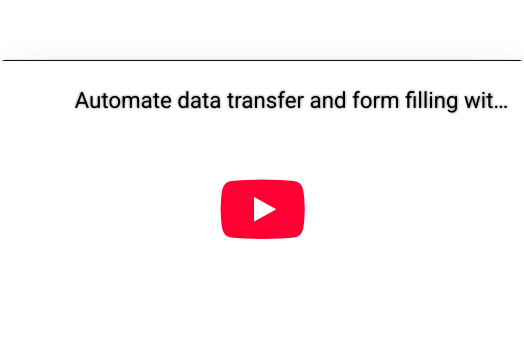
Sample Usage Copy to Text Blaze

Page URL: url
Page Domain: domain
Page Path: path
Page Title: title

Snippet preview

Page URL: https://blaze.today/commands/site/
Page Domain: blaze.today
Page Path: /commands/site/
Page Title: Text Blaze | {site}

Quick Overview Video



Automate data transfer and form filling wit...

Settings

Setting Name	Type	Description
Positional	text	The type of page information to get.
frame	self/top	Which frame to get information from (defaults to `top`).
selector	text	A CSS selector for use with the `html` or `text` types.
multiple	yes/no	When `selector` is specified, whether to return multiple results or just the first result (defaults to `no`).
page	string	When `page` is specified, the {site} command will read data from tabs whose URLs match the given `page` URL.
select	yes/no/ifneeded	When `page` is specified, this setting controls whether to show the tab selector. `yes` will always show the tab selector. `no` does not show the tab selector and only matches the current page. `ifneeded` (default) shows the tab selector only when multiple tabs are matching the given `page` pattern.
group	string	Allows different `{site}` commands with the same `page` setting to read data from different tabs.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Getting information from a web page is extremely useful in further automating your snippets.

For example, if you're using Text Blaze on your chat application, you could set it up to get specific information, like a name, email, or something else, present on the web page to use in your snippet

Throughout this documentation page, you will get several examples to help you see the `{site}` command's usefulness.

> `{site}` is an advanced Text Blaze command that assumes some familiarity with fundamental web concepts like HTML, CSS, URLs, and Regular Expressions.

Getting specific information from a web page

The first setting supported by the `{site}` command is a positional setting that tells the command the type of information to get from the web page.

The table below displays the different supported values that this command takes:

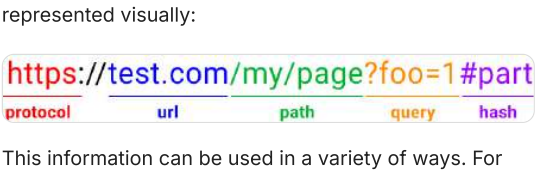
Type	Description	Example for url `https://test.com/my/page?foo=1#part`
url	The full url of the page.	`https://test.com/my/page?foo=1#part`
domain	The domain of the url.	`test.com`
path	The path of the url.	`/my/page`
protocol	The protocol of the url.	`https`
query	The query of the url.	`?foo=1`
hash	The hash of the url.	`#part`
html	The html of the webpage.	`<html>...</html>`
text	The text of the webpage.	`Welcome to...`
title	The title of the webpage.	`Site Title`
fieldtext	The text of the focused textfield.	`Hi there hello!`
selection	The currently selected text.	`there`
fieldleft	The text to the left of the cursor in the focused textfield.	`Hi`
fieldright	The text to the right of the cursor in the focused textfield.	`hello!`

Capitalization is important. For example, you can use `{site: url}` to get the web page's URL, but doing `{site: URL}` will not work.

Getting Information from the URL

The `url`, `domain`, `path`, `protocol`, `query`, and `hash` values are all concerned with getting information the **URL** of the current web page.

Here are the properties you can retrieve information for represented visually:



This information can be used in a variety of ways. For example, you could combine it with the {if} command to only insert information if you're on a specific part of the web page.

Including the Page Contents in your Snippet

The next two values that you can use in the `{site}` command's positional setting are `text` and `html`.

These two are not related to the URL, but instead get contents of the web page.

You can use them to dynamically populate parts of your snippets.

For example, if you use a Text Blaze snippet within your email application, you could use it to include information about the email recipient's name in your snippet.

To illustrate the usage of `text` and `html` in the `{site}` command, we're going to consider some HTML typical of what you might see on a site:

> The `text` and `html` types are both in **Beta** and their functionality may change in the future.
>
> In particular, we are exploring possibilities to include nested `<iframe>` content in each of them and the possibility of including nested `<input>` and `<textarea>` elements directly in the `text` type.

Sample HTML:

```
<div class="user-info">
  Name: <span class="name">John Smith</span>
  <ul class="favorites">
    <li>Oreos</li>
    <li>Twizzlers</li>
    <li>Snickers</li>
  </ul>
</div>
```

How the above sample HTML would render in a web page:

User Name: John Smith

- Oreos
- Twizzlers
- Snickers

We can use the `{site}` command to extract information from this sample. `{site: text}` and `{site: html}` will give you access to all the contents of the page. For example:

Page Contents Copy to Text Blaze

HTML for whole page (truncated for readability):
= substring(html , 1, 50) ...

Text for whole page (truncated for readability):
= substring(text , 1, 50) ...

Snippet preview

HTML for whole page (truncated for readability):
<html class="js" lang="en-us"><head><style data-em...

Text for whole page (truncated for readability):
Text Blaze
Product
Work
Resources
Pricing
ADD TO C...

> The above example is using the `{site}` command on the Text Blaze website so the contents will be related to this web page.

Also, one very important thing to note is that the `text` type pulls text content from the page. **This does not include contents inside of `<input>` or `<textarea>` HTML tags**. To include the contents of these elements, you need to specify a selector that specifically selects one of these elements and not a parent element. We'll review selectors in the next section.

Getting specific parts of a page

Using `{site: html}` or `{site: text}` will give you the entire contents of the page.

Generally, though you just want to get a specific part of the contents of the page, like a name or address. Text Blaze has two ways to do that: CSS Selectors and Regular Expressions.

If you do use `html` or `text` as the value for the `{site}` command's positional setting you have access to 2 additional named settings.

Those named settings are `selector` and `multiple`. **These 2 named settings are concerned with *how* to get the contents of the page.**

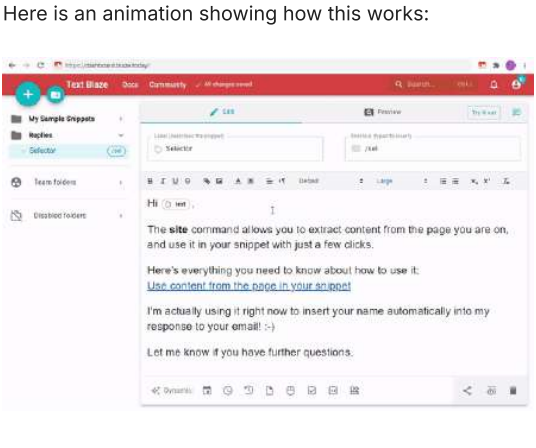
Using CSS Selectors to Get Part of the Page

One way that Text Blaze can access specific parts of the page is with CSS selectors. CSS selectors provide a very concise method of getting specific parts of a page.

The details of CSS selectors are out of the scope of this documentation, and we refer you to this website to understand them in more detail.

Fortunately, for most use cases, Text Blaze can automatically create the right selector for you with point and click. Just use the "Select from website" feature of Text Blaze and click on the desired part of the page.

Here is an animation showing how this works:



Once you have your selector, the snippet automatically populates its contents when used. For example, in the html sample from the previous section, you can get the name using a CSS selector. You can also use another CSS selector to get different parts of the favorites list.

Using CSS Selectors Copy to Text Blaze

Using a CSS selector to get the user's name:
text selector .user-info .name

Using a CSS selector to get the first favorite item:
text selector .user-info .favorites li

Snippet preview

Using a CSS selector to get the user's name:
John Smith

Using a CSS selector to get the first favorite item:
Oreos

Reading information from any tab

The {site} command supports pulling information from any tab in Chrome and inserting it into your current tab.

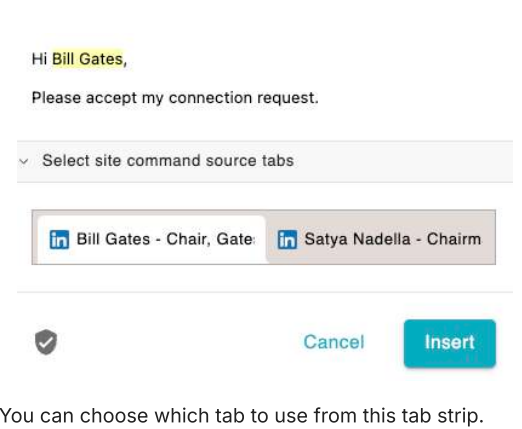
For example, when writing an email to a client, you can:

1. Use your snippet in the email webpage
2. Use {site} to pull client information from your CRM open in a different Chrome tab

This is the default behavior in Text Blaze and requires no additional setup.

Text Blaze matches your open Chrome tabs using the wildcard pattern in the `page` setting. For example, setting `page` to `https://blaze.today/commands/*` will match this page but not `https://blaze.today/docs/`

When more than one Chrome tab matches the `page` setting, Text Blaze shows a tab strip in your form window:



Hi Bill Gates,

Please accept my connection request.

Select site command source tabs

You can choose which tab to use from this tab strip.

Controlling the tab selector

The `select` setting controls when this tab strip appears. It can have one of these three values:

1. `yes`: Always shows the tab selector, even with just one matching tab.
2. `no`: Never shows the tab selector. The {site} command will show an error if the `page` pattern doesn't match the current page.
3. `ifneeded` (default): Only shows the tab selector when multiple tabs match.

By default, all {site} commands with the same `page` setting will pull data from the same tab. For example, `{site: text; page=https://blaze.today/*}` and `{site: url; page=https://blaze.today/*}` will always use the same Chrome tab.

You can use the `group` setting to pull data from different tabs even with the same `page` pattern. For example, the following two commands `{site: text; page=https://blaze.today/*; group=First item}` and `{site: text; page=https://blaze.today/*; group=Second item}` can pull data from two different Chrome tabs (both matching the same `page` pattern).

Getting more than one element with the `multiple` setting

Notice how in the above CSS selectors example that using `.user-info .favorites li` gets the first item in the favorites list even though multiple elements have the same CSS selector.

There is where the named setting `multiple` comes in. If `multiple` is `no` or omitted only the first match will be returned.

If `multiple` is `yes` all the matches will be returned as a list.

Using the multiple setting Copy to Text Blaze

Using a CSS selector to get all the favorite items as a list:
text selector .user-info .favorites li multiple yes

Snippet preview

Using a CSS selector to get all the favorite items as a list:
["Oreos", "Twizzlers", "Snickers"]

Handling Errors

If `multiple` is `no` and there is no match for that specific CSS selector on the web page you're using the snippet on, **an error will be displayed**.

Text Blaze provides you with `catch()`, a predefined function to help you deal with an error scenario. You can use it to provide a default value in an error case in your formulas:

CSS Selector Error Handling Copy to Text Blaze

Error shown:
text selector .does-not-exist

Error caught and replaced with another value:
= catch(text selector .does-not-exist , "My Defau

Snippet preview

Error shown:
No match found for selector: ".does-not-exist"

Error caught and replaced with another value:
My Default Value

If `multiple` is set to `yes`, on the other hand, and there is no match, Text Blaze will return an empty list (`[]`).

Controlling where information is loaded from using the `frame` setting

Web pages can embed other web pages (called 'frames'). If you use a Text Blaze snippet inside of a frame, you can use the `frame` named setting to load information either the frame (by setting the `frame` setting to `self`) or from the web page embedding the frame (by setting the `frame` setting to `top`).

> Using the `frame` setting affects the output of both domains and also the page contents.

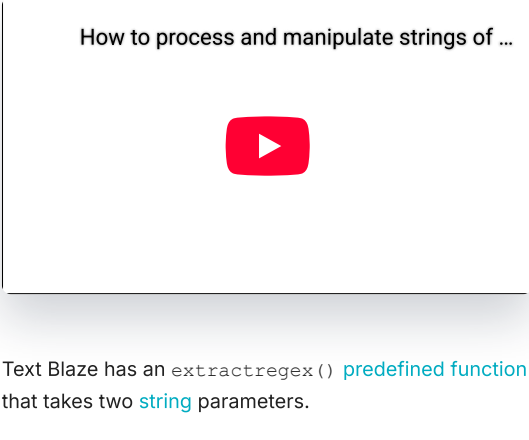
If Text Blaze is used in a normal web page without frames, both `self` and `top` are equivalent. Text Blaze defaults to `top`. This will return the URL you see in Chrome's URL bar.

Using Regular Expressions to Get Part of the Page

Regular expressions are a powerful way to search through text and find specific patterns of text.

For example, if you're looking for a phone number or an email address (both pieces of textual content that follow a specific pattern), then regular expressions can help you find those.

The following video will help you further understand how Regular Expressions work.



How to process and manipulate strings of ...

Text Blaze has an `extractregex()` predefined function that takes two string parameters.

If you use `{site: text}` or `{site: html}`, then you can use Regular Expressions as an alternative to CSS selectors to retrieve a specific part of the web page.

For example, if you want to get the name from the html sample in the previous section, you can use the following regular expression: "`Name: ([\w ]+)`".

This tells Text Blaze, that we want all the text after the text "Name: " that consists of spaces or letter characters.

Using Regular Expressions Copy to Text Blaze

Using a regular expression to get just the user's name.

Name from the page:
= extractregex(text , "Name: ([\w]+)")

Snippet preview

Using a regular expression to get just the user's name.

Name from the page: John Smith

You can read more about regular expressions in our Blaze Formula Language Reference, reading our non-scary guide on the community forums, and watching the above video. Also, there are some great online resources you can use to learn more about them.

Reading data based on user selection in a page

The `selector` and `xpath` settings allow you to read data from a fixed element. You can also instead read data from the page based on the active user selection. For example: assume that you have a textbox with this text "Hi there hello!" (without quotes). Now, you have selected the word "there" with the mouse. Now, you can check the following four `{site}` commands:

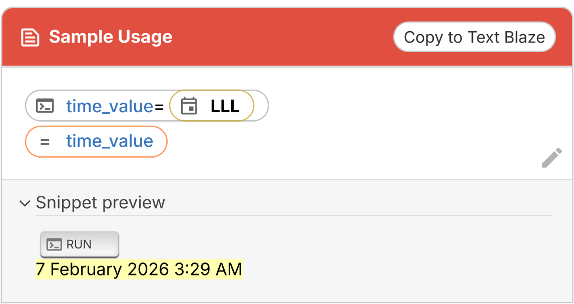
1. `{site: selection}` - outputs the currently selected text, whether it is inside or outside a textbox. In this case, it outputs "`there`" (without quotes).
2. `{site: fieldtext}` - outputs the text of the focused textfield. In this case, it outputs "`Hi there hello!`" (without quotes).
3. `{site: fieldleft}` - outputs the text to the left of the cursor in the focused textfield. In this case, it outputs "`Hi `" (without quotes).
4. `{site: fieldright}` - outputs text to the right of the cursor in the focused textfield. In this case, it outputs "` hello!`" (without quotes).

As the names suggest, `{site: selection}` works whether or not a textfield is focused, whereas the other three commands output blank if no textfield is focused.

Note that all four commands can be combined with the `page` setting to read data from another webpage in another browser tab.

{run}

Summary: Runs a code block once.

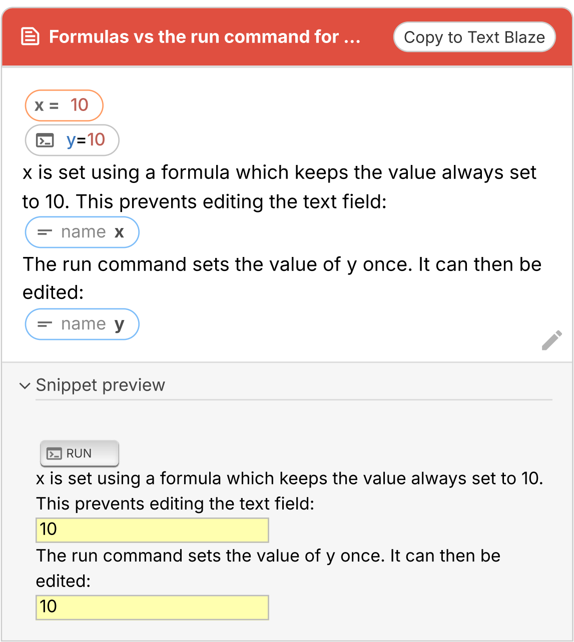

Sample Usage
Copy to Text Blaze
time_value= LLL
= time_value
Snippet preview
RUN

7 February 2026 3:29 AM

Settings

Setting Name	Type	Description
Positional	code	The code block to run once.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

`{run}` executes a code block once. It is particularly useful for initializing form variables that you want to manually change later. For example


Formulas vs the run command for ...
Copy to Text Blaze
x = 10
y=10
= name x
= name y
Snippet preview
RUN

x is set using a formula which keeps the value always set to 10. This prevents editing the text field:

10

The run command sets the value of y once. It can then be edited:

10

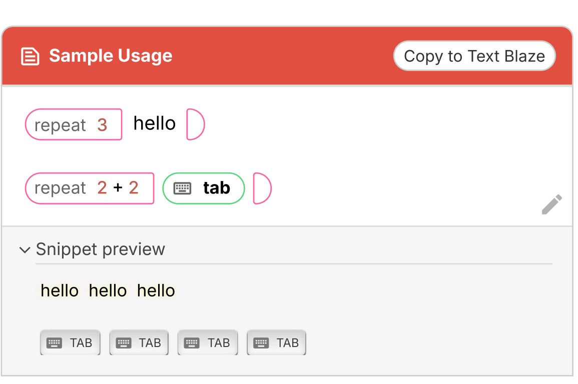
The `{run}` command is often used with the {button} to make interactive interfaces.

Learn more about code blocks in the code block reference.

> Run commands are in Beta and may change in the future.

{repeat}

Summary: Repeat a portion of a snippet multiple times.



repeat 3 hello

repeat 2 + 2 tab

Snippet preview

hello hello hello

Settings

Setting Name	Type	Description
Positional	formula	The number of times to repeat or a list comprehension.
locals	text	Optional identifier providing a list of form data in the command.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{repeat}` command replicates a portion of a snippet multiple times. It repeats everything that is between the `{repeat}` and `{endrepeat}` commands.

This command takes a formula as the first positional setting that *must* evaluate to one of two things:

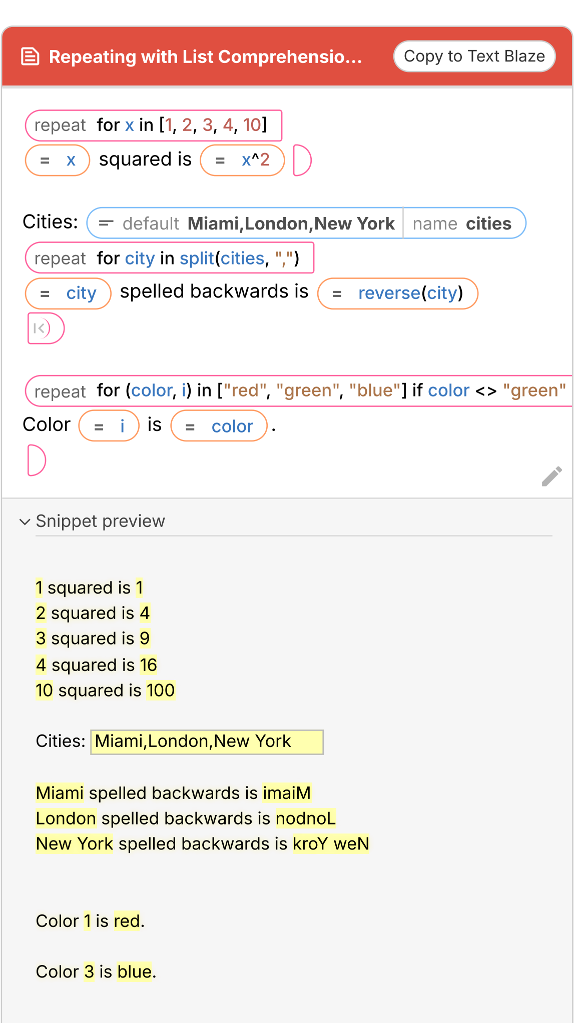
1. A Number
2. A List Comprehension

Also, see the {if} command for an additional command that helps you build dynamic snippets.

Repeating with List Comprehensions

List Comprehensions are a special formula syntax in Text Blaze that allow you to transform lists.

As the first positional setting in the `{repeat}` command instead of placing a number you place a list comprehension.



repeat for x in [1, 2, 3, 4, 10]
= x squared is = x^2

Cities: = default Miami,London,New York name cities
repeat for city in split(cities, ",")
= city spelled backwards is = reverse(city)

repeat for (color, i) in ["red", "green", "blue"] if color <> "green"
Color = i is = color .

Snippet preview

1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
10 squared is 100

Cities: Miami,London,New York

Miami spelled backwards is imaiM
London spelled backwards is nodnoL
New York spelled backwards is kroY weN

Color 1 is red.

Color 3 is blue.

- Notice how in the second example above, the output of the {formtext} is converted into a list by using the `split()` predefined function.
- In the final example you'll notice an `if` statement next to the list comprehension. This is the syntax you can use to filter your lists. Visit the Blaze Formula Reference to learn more.

Variable Scoping in the `{repeat}` command

> **This section assumes that you're already familiar with how variables work in Text Blaze**.
>
> Visit the Blaze Formula Reference to familiarize yourself with variables before proceeding.

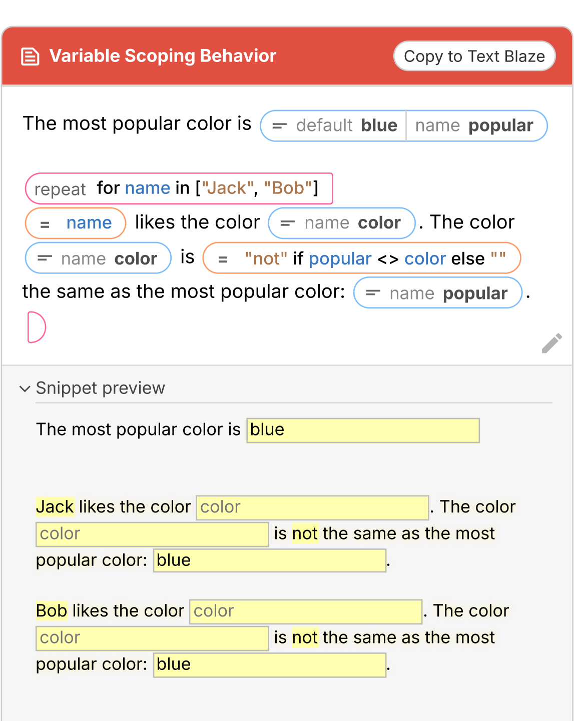
When using the list comprehensions with the `{repeat}` command, a *separate variable scope* gets created.

This means that the value of variables within a `{repeat}` may differ from variable values outside the repeat.

This is called *variable scoping* and the following rules summarize how this works:

1. The value of variables in the repeat iterator (e.g. `x` in the iterator `for x in [1, 2, 3]`) are only available inside of the command, between the `{repeat}` and `{endrepeat}` commands. If the variables are defined outside the block (e.g. you have a `{formtext: name=x}` command outside the block), then the values will not be changed outside the block but will be overridden within the block.
2. If a variable is defined outside the repeat block, it can be used in formulas inside the block. Form fields that have the same name inside the block will all use and change the same global value.
3. If a variable is not defined outside the repeat block, any form fields inside the block will each refer to a unique instance of that variable for each iteration of the `{repeat}` command.

The following example illustrates this scoping behavior:



The most popular color is = default blue name popular

repeat for name in ["Jack", "Bob"]
= name likes the color = name color . The color
= name color is = "not" if popular <> color else ""
the same as the most popular color: = name popular .

Snippet preview

The most popular color is blue

Jack likes the color color . The color color is not the same as the most popular color: blue .

Bob likes the color color . The color color is not the same as the most popular color: blue .

- Notice how all instances of the `{formtext: name=color}` are not all linked because they are scoped to each instance of the {repeat} command.

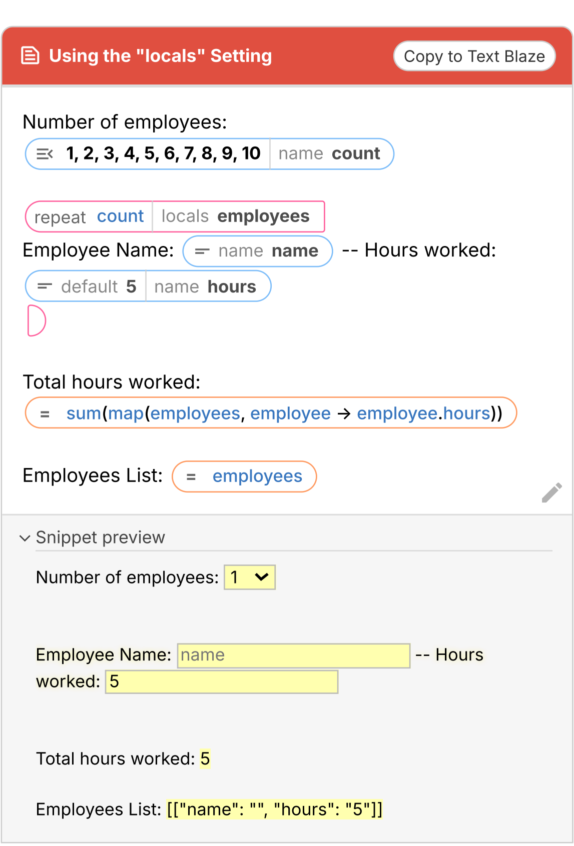
Accessing scoped variables outside of the `{repeat}` command

If you need access to the variables scoped to inside of the `{repeat}` command, you can use the optional `locals` named setting.

If used, the `locals` setting will create a list for you, with the identifier provided, where each item in the list is a keyed list for each repetition the command does.

Each key in the keyed list will be the name of the variable and the value for that key will the value of the variable.

For example, if we had a time sheet for our employees and we wanted to sum up their hours worked. We could implement it something like this:



Number of employees:
1, 2, 3, 4, 5, 6, 7, 8, 9, 10 name count

repeat count locals employees
Employee Name: = name name -- Hours worked:
= default 5 name hours

Total hours worked:
= sum(map(employees, employee → employee.hours))

Employees List: = employees

Snippet preview

Number of employees: 1

Employee Name: name -- Hours worked: 5

Total hours worked: 5

Employees List: [["name": "", "hours": "5"]]

> Notice how after the Employees List, **the value is a list that contains keyed lists**.
>
> Visit the Blaze Formula Reference to learn more about lists.

A note on whitespace

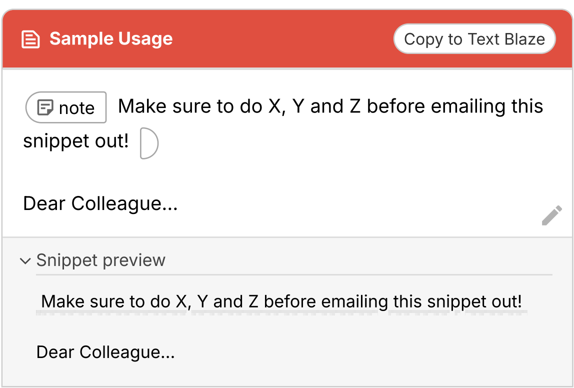
The `{repeat}` command does not output anything, but it still takes up whitespace in your snippet.

The optional `trim` setting is useful for removing whitespace around the `{repeat}` and `{endrepeat}` commands.

See the whitespace guide for more information on why this happens in Text Blaze.

{note}

Summary: A block of content that is not inserted into a snippet.



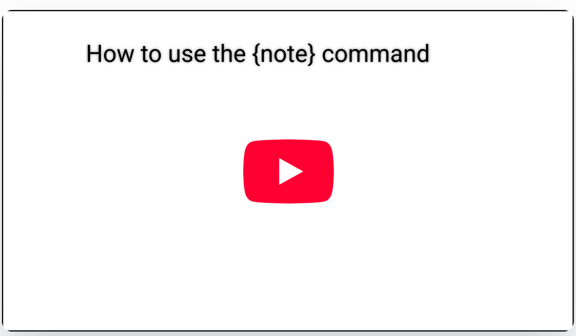
Make sure to do X, Y and Z before emailing this snippet out!

Dear Colleague...

Make sure to do X, Y and Z before emailing this snippet out!

Dear Colleague...

Quick Overview Video



Settings

Setting Name	Type	Description
insert	yes/no	Whether to include when inserted (defaults to `no`)
preview	yes/no	Whether to include in snippet previews (defaults to `yes`)
color	none/red/green/...	Change the color of the note's underline, or remove it completely.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `note` command can be used to display instructions or information you do not want to show up in the final inserted snippet.

By default, the contents of a `note` are shown in the snippet's preview, but are not inserted as part of the snippet.

For example, this can be useful if you're sharing a snippet folder with your team and need to document what's happening in the snippet.

To also hide the note contents in the snippet's preview, set the `preview` setting to `no`.

A note on whitespace

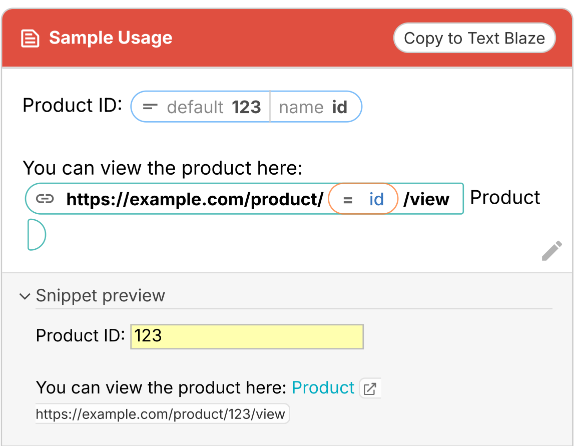
The `note` command does not output anything, but it still takes up whitespace in your snippet.

The optional `trim` setting is useful for removing whitespace around the command.

See the whitespace guide for more information on why this happens in Text Blaze.

{link}

Summary: Include a dynamic link in the snippet.



Product ID: default 123 | name id

You can view the product here:
https://example.com/product/ = id /view Product

Product ID: 123

You can view the product here: Product
https://example.com/product/123/view

Settings

Setting Name	Type	Description
Positional	text	The URL for the link.
	General Command Settings	
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `link` command creates links in snippets where the URL may change based on some data.

The first positional setting that a URL for the link. You're able to use the {=} (Formula Command) here to build dynamic links.

> If the link URL contains characters that have special significance in Text Blaze commands, they must be escaped with backslashes.
>
> Notably `{`, `}`, `\` and `;` should be escaped by placing a single `\` before them.
>
> For example, `{` would become `\{` and `\` would become `\\`.

See the Blaze Formula Reference for more information on defining custom URLs based on dynamic data.

The `endlink` command

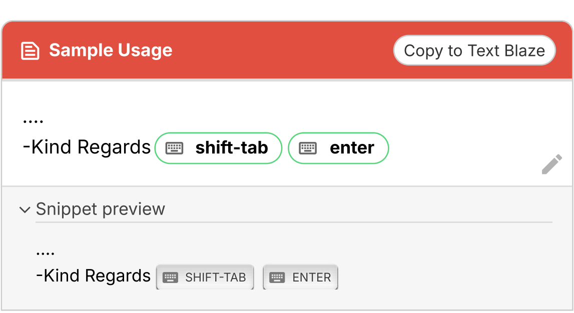
If you leave out the optional `endlink` command, then the link will display as is. If you insert content between the `link` and `endlink` the link will display with the inserted content instead.

Including a static link

If you want to include a link in the snippet that does not change based on some data, rather than using the `link` command you should use the link button in the styling toolbar.

{key}

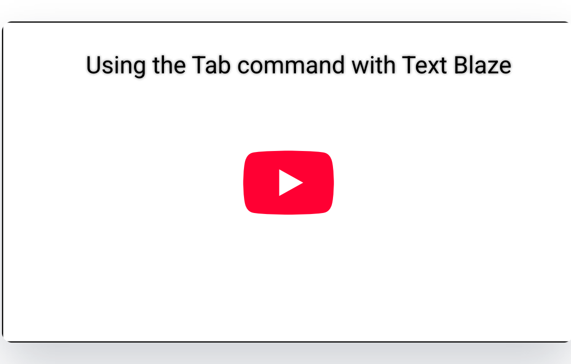
Summary: Simulates a key press in the current webpage.



....
-Kind Regards shift-tab enter

....
-Kind Regards SHIFT-TAB ENTER

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	text	The key press to simulate.
mac	text	Override key press when on a Mac
win	text	Override key press when on Windows
linux	text	Override key press when on Linux
cros	text	Override key press when on a ChromeBook
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{key}` command allows you to emulate a key press on the current page. For example, you can do `{key: s}` to simulate pressing the `s` key which will output an `s` character into the text field.

But, **the most common use case** is utilizing the `{key}` command to trigger shortcuts (`{key: ctrl-s}`), move the focus to another element (`{key: tab}`), or submit a form (`{key: enter}`). Click here to see more examples for the key command.

The syntax for the `{key}` command's positional setting is `modifier-character`. A `character` is the key that is pressed (e.g. `s` or a non-character like `tab`). The `modifier`, which is optional, is a key like `shift` key or the `ctrl` key. **For example: `{key: shift-tab}`** is utilizing both a modifier key and a character key.

> Text Blaze key presses can trigger actions **within a web page**, but they cannot trigger standard system commands on your operating system.
>
> For example, `ctrl-c` will not copy the currently selected text to the clipboard and `ctrl-q` will not quit the web browser as both those are system commands.
>
> This also means that different websites might behave differently.
>
> See the autopilot guide to learn more about what you can and cannot do with the `{key}` command.

Click here to see an example of automating writing your email utilizing the `{key}` command and form commands.

Modifiers

The following modifiers are supported:

Key	Modifier
Shift Key	`shift`
Control Key	`ctrl`
Alt (Option) Key	`alt`
Command (Meta) Key	`cmd`

Also, it's possible to use multiple modifiers joined together with hyphens. For example `ctrl-shift-x` or `alt-shift-tab`.

Non-Character Keys

In addition to key presses that can be represented with a standard keyboard character (e.g. `{key: s}` or `{key: p}`), the following non-character keys are supported:

Key	Key Token
Tab Key	`tab`
Enter Key	`enter`
Backspace Key	`backspace`
Left Arrow Key	`leftarrow`
Right Arrow Key	`rightarrow`
Up Arrow Key (experimental)	`uparrow`
Down Arrow Key (experimental)	`downarrow`

An example of usage of these keys would be `{key: tab}` to tab to the next field in the webpage (or `{key: shift-tab}` to tab to the prior field in the document). The remainder of the snippet would then be entered in that field, allowing you to fill out multiple fields with a single snippet.

The `enter` key is also useful for pressing a button to submit a form or trigger some action. `tab` and `enter` key commands can often be used sequentially to enable a variety of scripting behaviors.

Click here to see a real-world example utilizing the `{click}` and `{key}` autopilot commands.

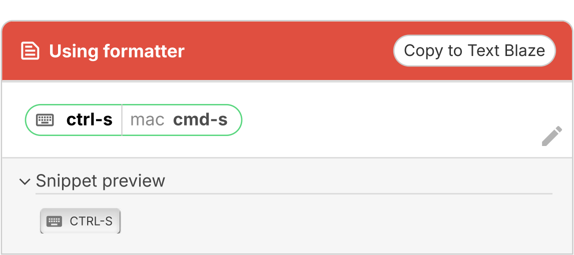
Supporting Multiple Operating Systems

Key presses may be different on different operating systems. For example, saving a file may be `ctrl-s` on a Windows computer, but `cmd-s` on a Macintosh computer.

Text Blaze lets you define different key presses for each operating system via the optional `win`, `mac`, `linux` and `cros` override settings. When one of these is defined and the snippet is used on that operating system, Text Blaze will use the key press you defined for that operating system, rather than the default key press, which you define in the first positional setting.

For example, if you want to use `ctrl-s` on Windows, Linux or ChromeBooks; but `cmd-s` on a Macintosh computer, you could use the following:

This is most useful if you're using Text Blaze with your team and sharing snippets with them that will be used on a variety of operating systems.



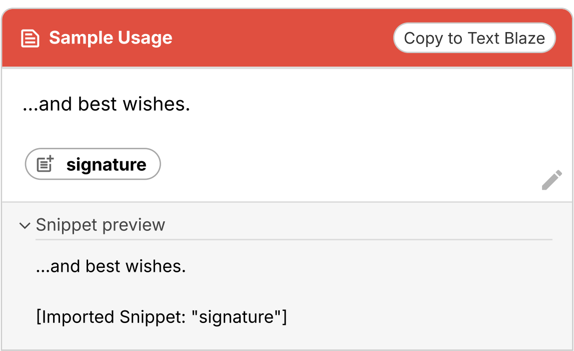
ctrl-s mac cmd-s

CTRL-S

To learn more about automating actions in a web page, see the **autopilot guide**.

{import}

Summary: Imports another snippet into the current snippet.



...and best wishes.

{import: signature}

Snippet preview

...and best wishes.

[Imported Snippet: "signature"]

Quick Overview Video

How to use {import} to use multiple snippe...

Settings

Setting Name	Type	Description
Positional	text	The shortcut of the snippet to import (what you type to insert the snippet, not the snippet's label)
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{import}` command can be used to combine snippets and reuse common parts of snippets. Form fields with the same name will be linked across the combined snippet.

The shortcut you use in the `import` command should be the shortcut text you actually type in to insert the snippet, not the snippet's descriptive label.

If the shortcut contains characters that have special meaning in Text Blaze commands (";", "=", "{" and "}"), you will need to escape them by placing a backslash before each special character.

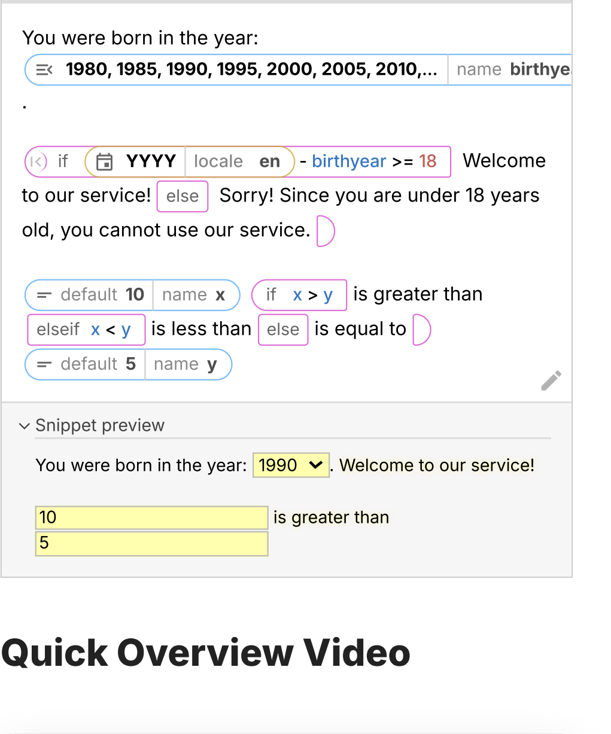
For example, if your snippet shortcut is ";go;", you can import it with "{import: \;go\;}".

Learn more about escaping characters, by visiting the Formula Reference.

> There are a few caveats when using the {snippet} command together with the `{import}` command. Click here to learn more.

{if}

Summary: Conditionally includes content in the snippet.



Sample Usage

Copy to Text Blaze

You were born in the year: 1980, 1985, 1990, 1995, 2000, 2005, 2010,… name birthye
.

if YYYY locale en - birthyear >= 18 Welcome to our service! else Sorry! Since you are under 18 years old, you cannot use our service.

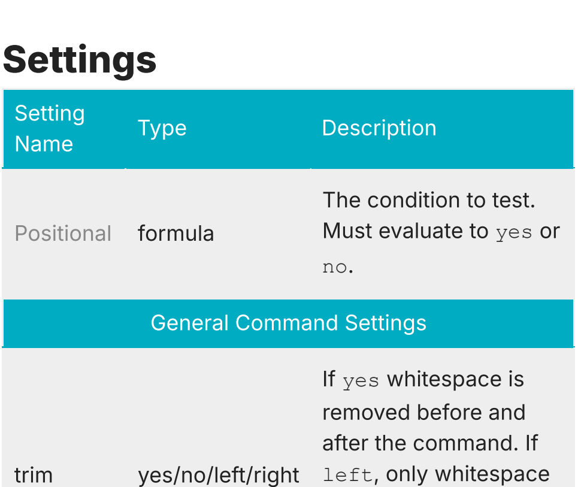
= default 10 name x if x > y is greater than elseif x < y is less than else is equal to
= default 5 name y

Snippet preview

You were born in the year: 1990. Welcome to our service!

10 is greater than
5

Quick Overview Video



How to use the IF command to change sni…

Settings

Setting Name	Type	Description
Positional	formula	The condition to test. Must evaluate to `yes` or `no`.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The {if} command lets you conditionally include or exclude content based on a condition. That condition is a Text Blaze Formula that must evaluate to either `yes` or `no`.

Everything within the {if} command and the required `{endif}` command will be inserted if the condition evaluates to `yes`.

> Note the use of {time: YYYY; locale=en} to get the current year in the `if` command in the example at the top of the page. This allows the snippet to automatically update the check when Jan 1 passes without you having to update it manually.
>
> The `locale=en` setting ensures the year number will be printed out in the Western formatting of `2017` or `2018` which is required for use in the `if` command. Some locales have other types of number formatting. You may omit this if your locale has Western number formatting.

See also the {=} (Formula Command) command and the {repeat} command for additional tools to help you build dynamic snippets.

The required {endif} command

The `if` command is similar to the {note} command in that the closing command must be present. In this case, it's the `{endif}` command.

Everything inside of these two commands will or will not get rendered depending on the output of the condition, the `if` command's positional setting.

> The `trim` setting is useful for removing whitespace around the `if`, `elseif`, `else`, and `endif` commands. See the whitespace guide to learn more.

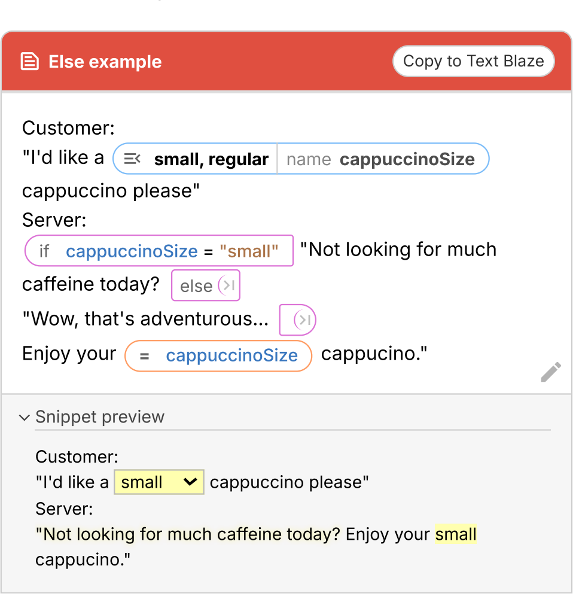
The optional {else} command

What if you would like to do something different if the condition evaluates to `no`?

In that case, you have access to an `{else}` command. The `{else}` command must be placed within the starting `if` command and the closing `{endif}` command.

There can only be one `{else}` command.

Here's an example:



Else example

Copy to Text Blaze

Customer:
"I'd like a small, regular name cappuccinoSize cappuccino please"
Server:
if cappuccinoSize = "small" "Not looking for much caffeine today? else "Wow, that's adventurous... Enjoy your = cappuccinoSize cappucino."

Snippet preview

Customer:
"I'd like a small cappuccino please"
Server:
"Not looking for much caffeine today? Enjoy your small cappucino."

> ### Try it out
>
> - Try changing the {formmenu} command's value above and notice which part of the `if` command gets rendered.

Multiple if conditions with the optional {elseif} command

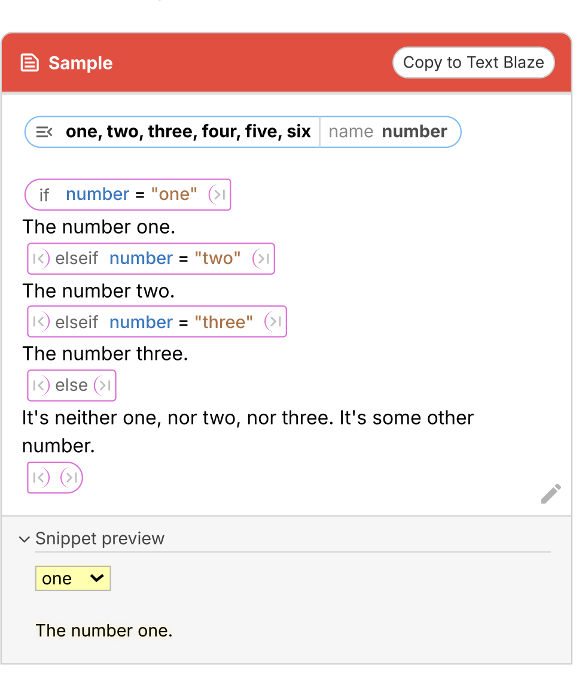
It's possible to test for multiple conditions with the help of the `{elseif}` command.

Similar to the {if} command, the `{elseif}` also takes a condition as its positional setting that is a Blaze Formula.

Also, just like the `{else}` command, they must be placed within the starting `if` command and the ending `{endif}` command.

You can have multiple `{elseif}` commands.

Here's an example:



Sample

Copy to Text Blaze

one, two, three, four, five, six name number

if number = "one"
The number one.
elseif number = "two"
The number two.
elseif number = "three"
The number three.
else
It's neither one, nor two, nor three. It's some other number.

Snippet preview

one

The number one.

Here's what we're telling Text Blaze in the snippet above:

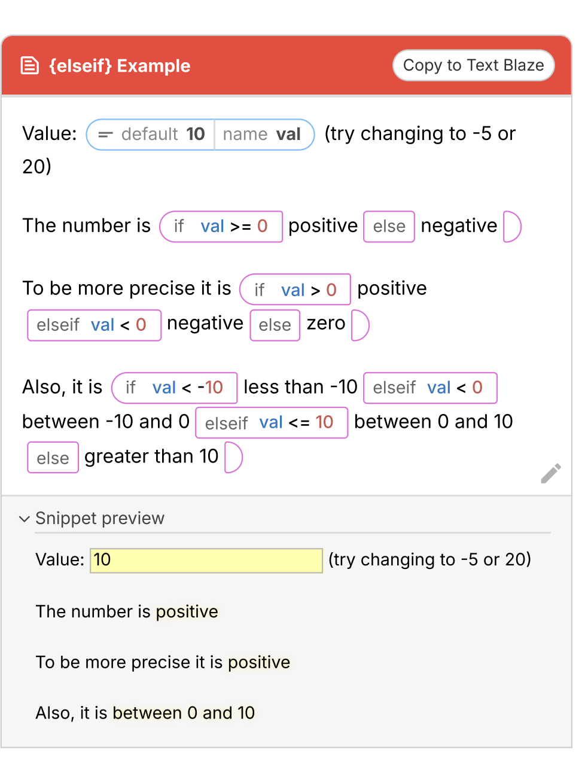
1. If the form name called *number* contains the text "one", then give us the text "The number one"
2. If the above isn't true, check whether it contains the text "two".
3. If the above is true, then give us the text "The number two". If not, check whether it contains the text "three".

And so on.

At the very end, you'll need to end with an `{endif}` command. The `{endif}` command is always placed at the very end.

If the initial `if` conditional evaluates to `no`, Text Blaze will look for the next `{elseif}` command and evaluate its condition. It evaluates to `yes`, its contents will be shown, otherwise, Text Blaze will move on to the next `{elseif}`.

If all `{elseif}` commands within the `if` command are evaluated to `no` and there is an `{else}` command, the contents of the `{else}` command will be shown. The following examples illustrate the use of `{elseif}` and `{else}`.



{elseif} Example

Copy to Text Blaze

Value: = default 10 name val (try changing to -5 or 20)

The number is if val >= 0 positive else negative

To be more precise it is if val > 0 positive elseif val < 0 negative else zero

Also, it is if val < -10 less than -10 elseif val < 0 between -10 and 0 elseif val <= 10 between 0 and 10 else greater than 10

Snippet preview

Value: 10 (try changing to -5 or 20)

The number is positive

To be more precise it is positive

Also, it is between 0 and 10

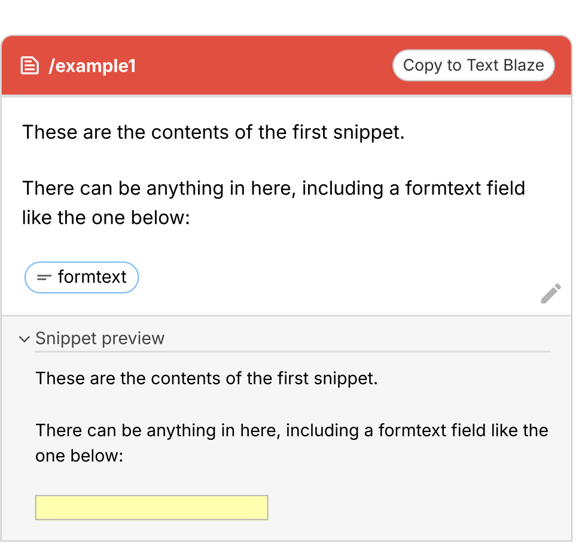
Dynamic content using the `if` command

Text Blaze becomes incredibly powerful when you start combining different commands like {formmenu}, {if} and {import}.

For this example, try saving the following three snippets below into your Text Blaze dashboard. This will help you understand better how Text Blaze uses these functions by seeing them in action.

To copy the snippet examples to your dashboard, click the “Copy to Text Blaze” button in the top right of the interactive snippet.

First, save the snippet below and give it the shortcut `/example1`.



/example1

Copy to Text Blaze

These are the contents of the first snippet.

There can be anything in here, including a formtext field like the one below:

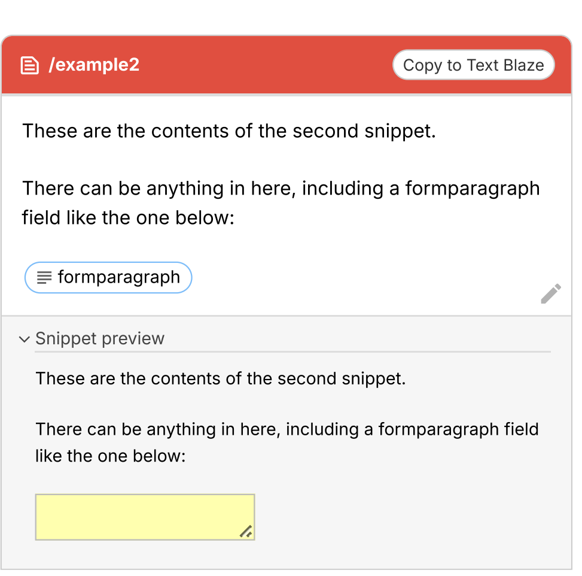
= formtext

Snippet preview

These are the contents of the first snippet.

There can be anything in here, including a formtext field like the one below:

Now, save this second snippet below and give it the shortcut `/example2`.



/example2

Copy to Text Blaze

These are the contents of the second snippet.

There can be anything in here, including a formparagraph field like the one below:

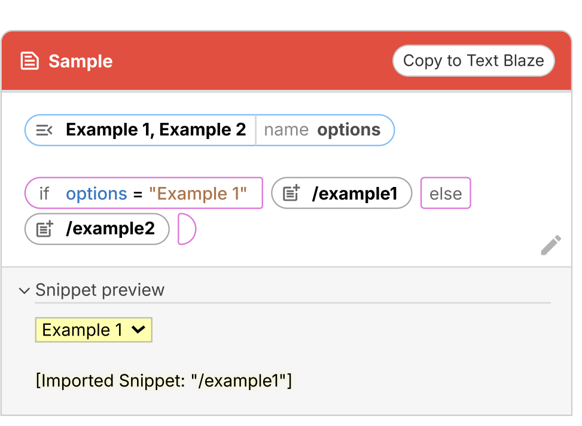
formparagraph

Snippet preview

These are the contents of the second snippet.

There can be anything in here, including a formparagraph field like the one below:

Finally, save this third snippet below and give it any shortcut you please.



Sample

Copy to Text Blaze

Example 1, Example 2 name options

if options = "Example 1" /example1 else /example2

Snippet preview

Example 1

[Imported Snippet: "/example1"]

Now try running the third snippet somewhere in a text field.

Depending on the option you choose in the drop-down menu, Text Blaze will give you the contents of the respective snippet.

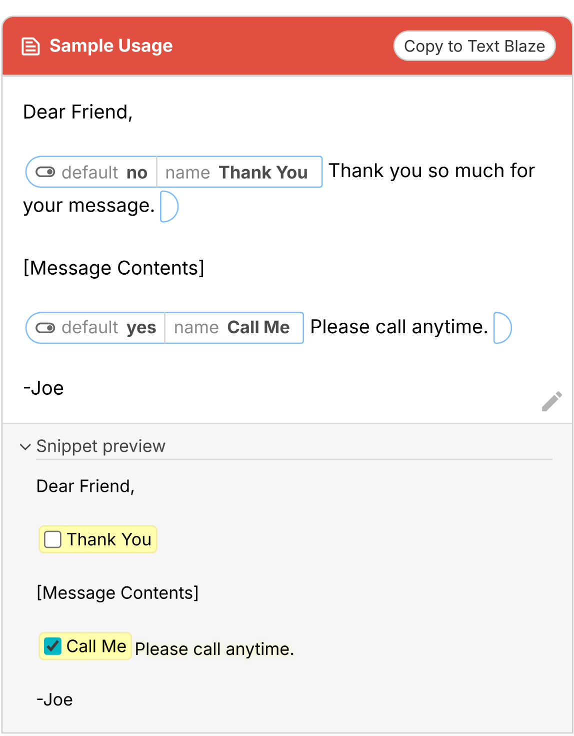
Now here's how that works.

1. We created a drop-down menu with {formmenu} and gave it two options – *Example 1* and *Example 2*. We also gave it the name "*options*".
2. We created an `if` command and gave it the following instructions:
    - If the form name called *options* is equal to "Example 1", then you should import the contents of the snippet with the shortcut `/example1`
    - If the above is not true, then you should import the contents of the snippet with the shortcut `/example2` (that's where the {else} command comes in).

You don't have to use an {import} command when using {if} commands, but they can help you better manage your snippets.

{formtoggle}

Summary: Creates a toggle switch that can enable or disable a block of text.



Dear Friend,

default **no** | name **Thank You** Thank you so much for your message.

[Message Contents]

default **yes** | name **Call Me** Please call anytime.

-Joe

Dear Friend,

☐ Thank You

[Message Contents]

☑ Call Me Please call anytime.

-Joe

Settings

Setting Name	Type	Description
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

Toggles allow you to selectively hide and show parts of your snippet. If a toggle is unchecked, everything between the `{formtoggle}` and `{endformtoggle}` commands will be hidden.

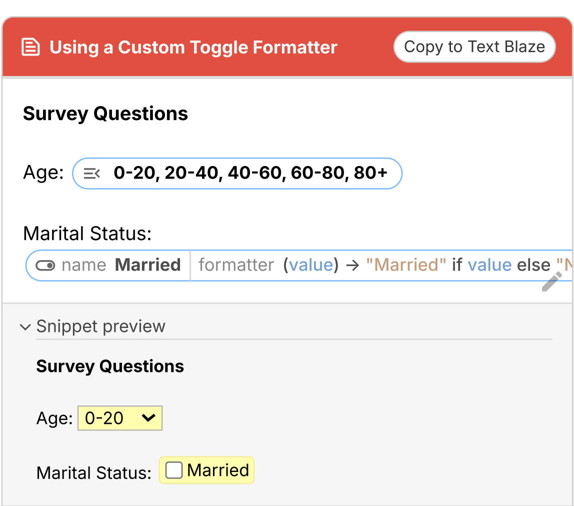
This command is similar to the {if} command, but the `{if}` command uses formulas to dynamically include or exclude content.

The optional {endformtoggle} command

The `{endformtoggle}` is optional and if omitted, `yes` or `no` will be shown when the snippet is inserted based on the value of the toggle's value.

You may change the output using the optional `formatter` setting. The `formatter` setting takes a function where the first parameter is the value of the toggle, either `yes` or `no`.

The following example illustrates this (you will need to copy it into Text Blaze and trigger the snippet to see it in practice).



Survey Questions

Age: **0-20, 20-40, 40-60, 60-80, 80+**

Marital Status:
name **Married** | formatter (value) -> "Married" if value else "N

Survey Questions

Age: 0-20

Marital Status: ☐ Married

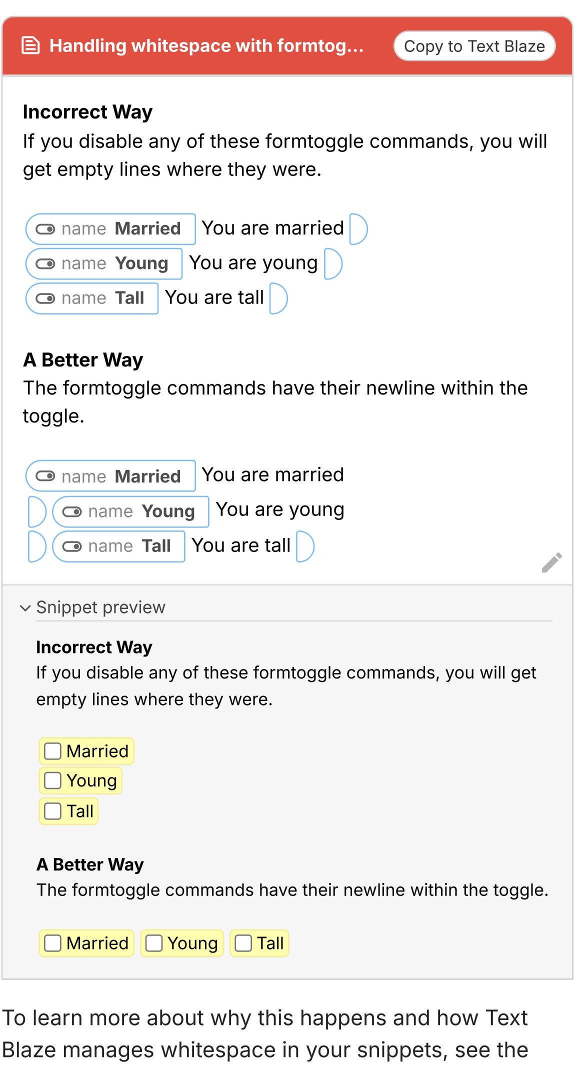
The above snippet will output `Married` instead of `yes`, or `Not Married` if the toggle's value is `no`.

Click here to learn more about the formatter function available to you.

{formtoggle} and whitespace

With multiline commands like {if} or `{formtoggle}` you generally want to have trailing or leading new lines to be *within* the command. **This can be counterintuitive as it's generally not how you would write the snippet by default**.

This prevents multiple extraneous newlines from showing up in your snippets. Here's an example demonstrating the issue:



Incorrect Way
If you disable any of these formtoggle commands, you will get empty lines where they were.

name **Married** You are married
name **Young** You are young
name **Tall** You are tall

A Better Way
The formtoggle commands have their newline within the toggle.

name **Married** You are married
name **Young** You are young
name **Tall** You are tall

Incorrect Way
If you disable any of these formtoggle commands, you will get empty lines where they were.

☐ Married
☐ Young
☐ Tall

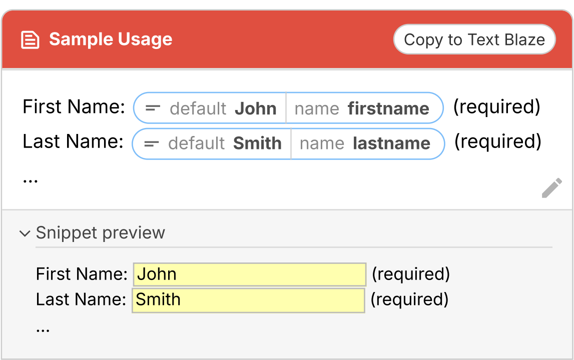
A Better Way
The formtoggle commands have their newline within the toggle.

☐ Married ☐ Young ☐ Tall

To learn more about why this happens and how Text Blaze manages whitespace in your snippets, see the whitespace guide.

{formtext}

Summary: Creates a single line text input field.



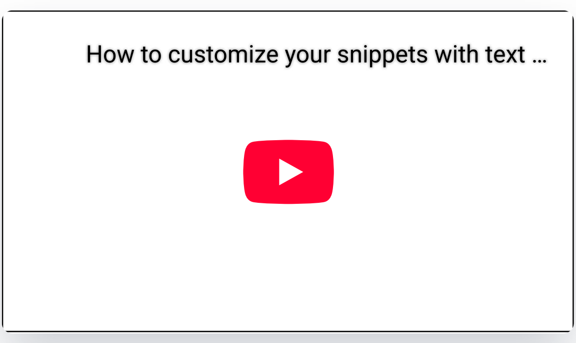
Sample Usage Copy to Text Blaze

First Name: `default John` `name firstname` (required)
Last Name: `default Smith` `name lastname` (required)
...

Snippet preview

First Name: John (required)
Last Name: Smith (required)
...

Quick Overview Video



How to customize your snippets with text ...

Settings

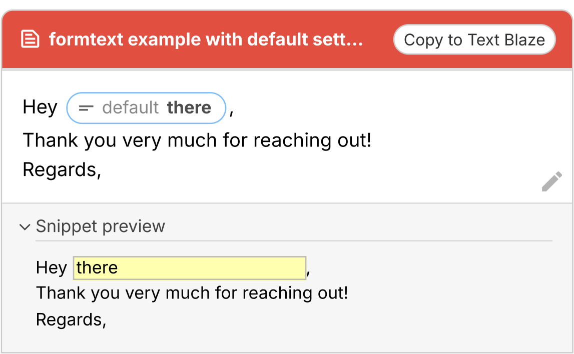
Setting Name	Type	Description
cols	number	The width of the form field in columns of characters.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{formtext}` command is like your traditional input field that you would use in forms across the web. This lets you customize your snippets before inserting them.

To learn more about form commands and their capabilities, see the **forms guide**.

The `default` setting

The `{formtext}` command has access to an optional `default` setting. One example is utilizing `there` as the value so that a generic message like `Hey there`, appears if you do not customize the form.



formtext example with default sett... Copy to Text Blaze

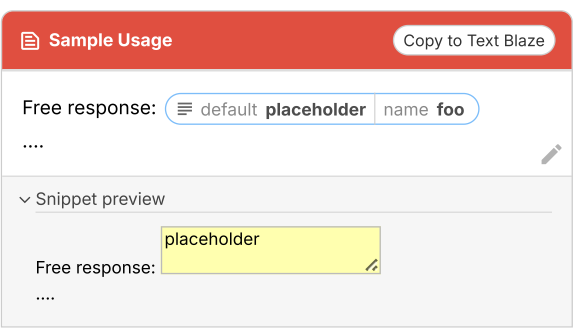
Hey `default there`,
Thank you very much for reaching out!
Regards,

Snippet preview

Hey there,
Thank you very much for reaching out!
Regards,

{formparagraph}

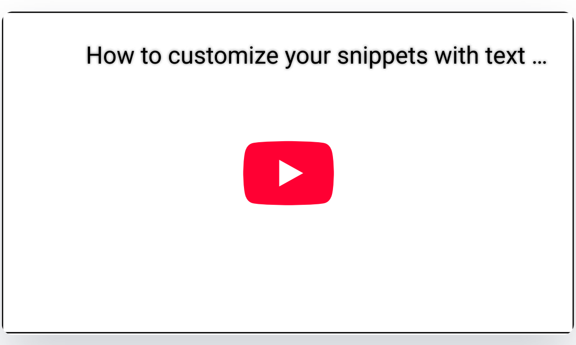
Summary: Creates a multi-line text input field.



Free response: default **placeholder** name **foo**
....

Free response: placeholder
....

Quick Overview Video



Settings

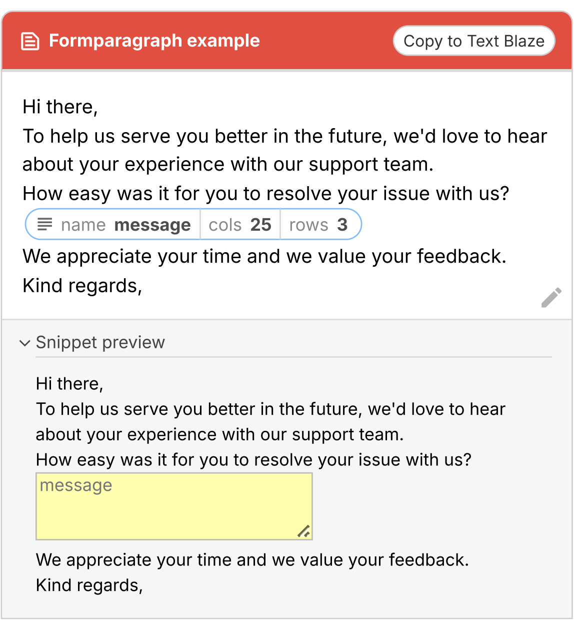
Setting Name	Type	Description
cols	number	The width of the form field in columns of characters.
rows	number	The height of the form field in rows.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

The `{formparagraph}` command is very similar to the {formtext} command, but it allows you to see more text by giving you access to the optional `rows` setting.

It's very useful if you set up a snippet as a template and you want to insert more custom content somehwhere in the snippet.

For example, you can have a general template that you use at the top and bottom of your snippet but you would like to customize the middle.



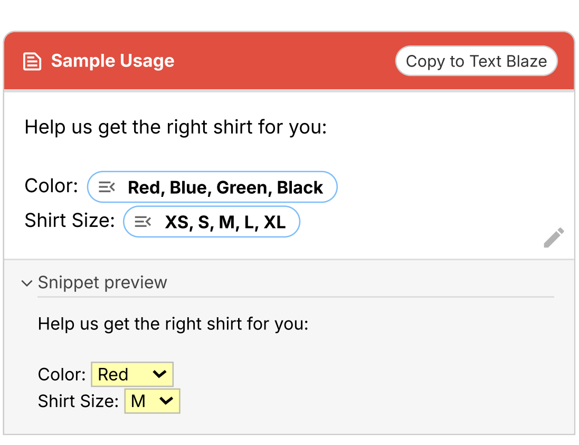
Hi there,
To help us serve you better in the future, we'd love to hear about your experience with our support team.
How easy was it for you to resolve your issue with us?
name **message** cols **25** rows **3**
We appreciate your time and we value your feedback.
Kind regards,

Hi there,
To help us serve you better in the future, we'd love to hear about your experience with our support team.
How easy was it for you to resolve your issue with us?
message
We appreciate your time and we value your feedback.
Kind regards,

You also might consider wrapping the `{formparagraph}` command with the {formtoggle} command if this will be an optional part of your snippet.

{formmenu}

Summary: Creates a menu input to select one or more items.



Sample Usage

Copy to Text Blaze

Help us get the right shirt for you:

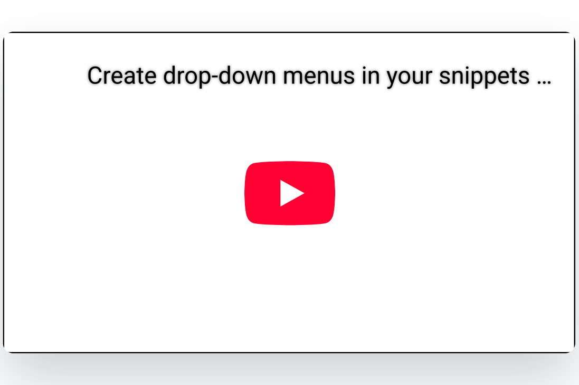
Color: Red, Blue, Green, Black
Shirt Size: XS, S, M, L, XL

Snippet preview

Help us get the right shirt for you:

Color: Red
Shirt Size: M

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	text	The ordered list of menu items.
cols	number	The width of the form menu in columns of characters.
multiple	yes/no	Allow selecting multiple items (default is no).
itemformatter	function	Formats individual menu items. Takes two arguments, the text of the menu item and the index of the menu item.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

To learn more about form commands and their capabilities, see the **forms guide**.

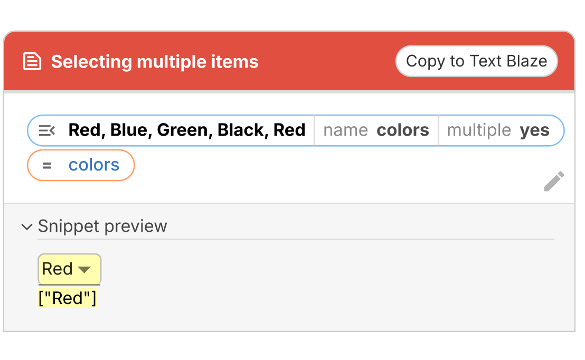
The `{formmenu}` command is a unique command in that it takes multiple positional settings. Each positional setting that you include is an item that you will be able to select in the menu.

> ### Try it out
>
> - In the example at the top of the page, add another positional setting, "Yellow", to the {formmenu} and see how another item gets added.

Selecting multiple items

The `{formmenu}` command also allows you to define a `multiple` setting. By default it's set to `no`, but if you explicitly set it to `yes`, you can now select multiple items.

If you also specifiy a `name` setting, then the results will be returned as a list if you use it in a {=} (Formula Command) command.



Selecting multiple items

Copy to Text Blaze

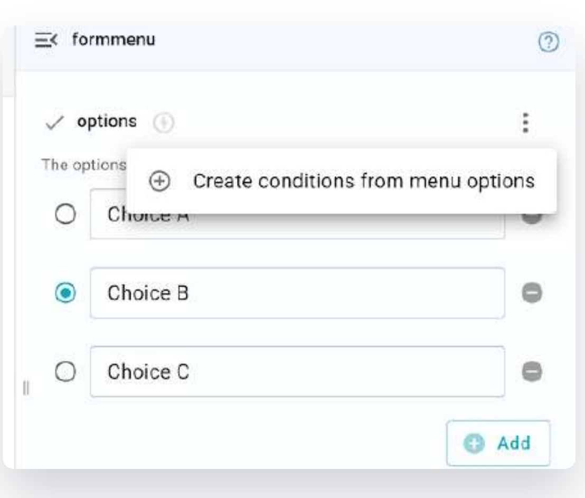
Red, Blue, Green, Black, Red | name colors | multiple yes
= colors

Snippet preview

Red
["Red"]

> ### Try it out
>
> - Notice how the result of the formmenu is initially set to "[]", an empty list if `multiple` is set to true.
> - Try adding a value for the `default` setting and see what happens to the list in output in the example above.

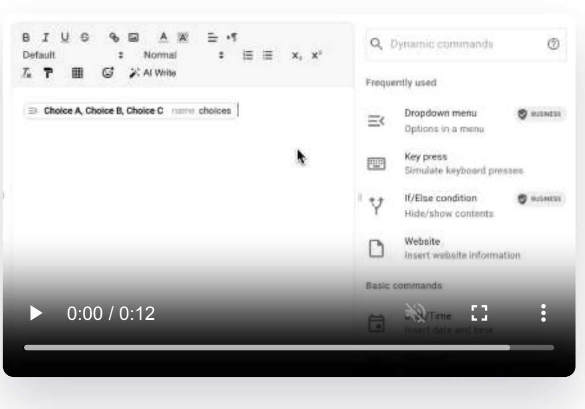
Creating conditions from menu options



You can quickly create if/else conditions for `{formmenu}` options by clicking on a form-menu, then clicking the three dots in the right corner of the menu editor.

Doing this adds conditional text that only shows up and gets inserted if a specific item in the menu is selected.

> Note that the if statements do not update if the values of the form menu change. If you change values in the menu, you will need to update the conditions accordingly.



Keep in mind that this option is not available when the 'Multiple' setting is selected. If your menu is set to 'Multiple', you will have to use the contains() function in your if statements for each item in the menu.

This option also does not work if you have the 'Advanced Editor' (lightning symbol) turned on for your drop-down menu.

Custom Formatting

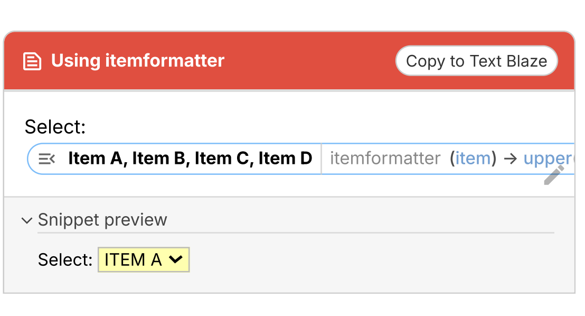
There are two settings that customize the formatting of the menu items:

- `itemformatter`: Formats each individual menu item.
- `formatter`: Formats the entire list of selected items. This is primarily useful when you have the `multiple` setting set to `yes` and are selecting multiple items from the list.

Using `itemformatter`

The `itemformatter` setting is a function that takes the text of each individual menu items and transforms it.

For example, if you wanted to uppercase each menu item, you could do something like the following.



Using itemformatter

Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | itemformatter (item) -> upper

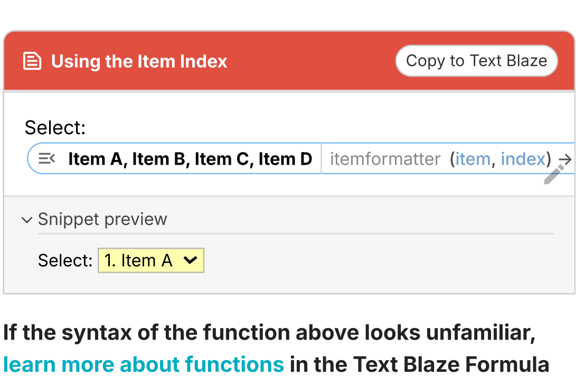
Snippet preview

Select: ITEM A

> - There are various other string predefined functions available for you in Text Blaze to use. For example: `lower()`, `reverse()`, and more.

The `itemformatter` function takes an optional second parameter which is the index of the menu item. This is the position of the item in the list.

For example, we can use that to number our items:



Using the Item Index

Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | itemformatter (item, index) ->

Snippet preview

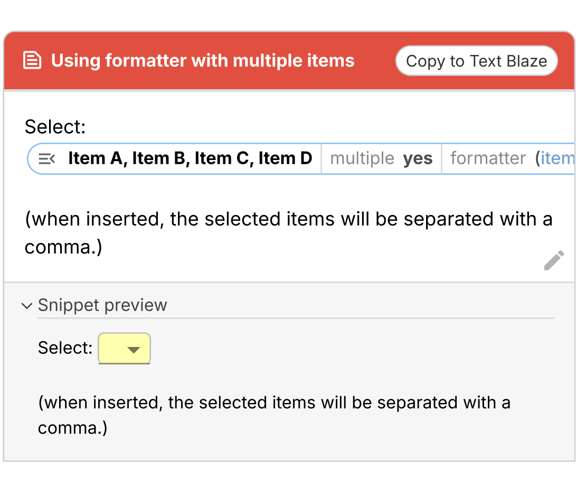
Select: 1. Item A

If the syntax of the function above looks unfamiliar, learn more about functions in the Text Blaze Formula Reference.

Using `formatter`

The `formatter` setting is a function that allows you to format **all of the items** in your `{formmenu}`.

If `multiple` is `yes`, it will be passed a list of values (which may be empty if no items have been selected).



Using formatter with multiple items

Copy to Text Blaze

Select:
Item A, Item B, Item C, Item D | multiple yes | formatter (item

(when inserted, the selected items will be separated with a comma.)

Snippet preview

Select:

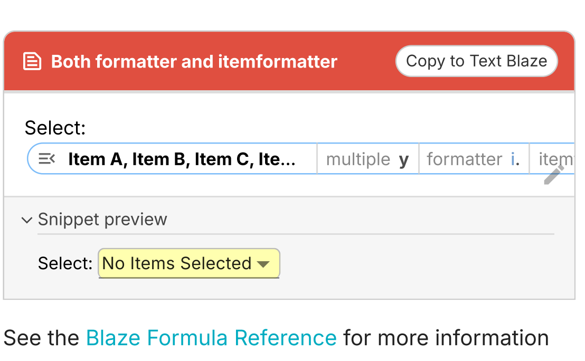
(when inserted, the selected items will be separated with a comma.)

> `join` is a predefined list function available to you. It takes items in a list and joins them together into a single string separated by a string, in this case a comma (,).

If `multiple` is `no`, the first parameter of the `formatter` setting is a single string value.

Combining Both Formatters

The following example illustrates the use of the `itemformatter` and `formatter` functions together.



Both formatter and itemformatter

Copy to Text Blaze

Select:
Item A, Item B, Item C, Ite... | multiple y | formatter i. | item

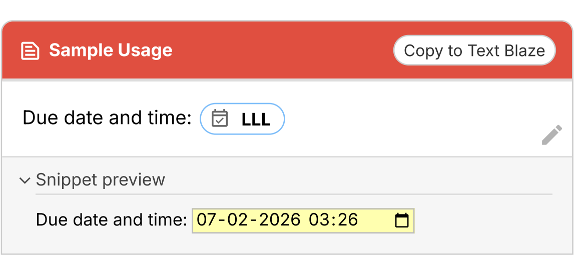
Snippet preview

Select: No Items Selected

See the Blaze Formula Reference for more information on defining formatting functions.

{formdate}

Summary: Creates a date/time input field.

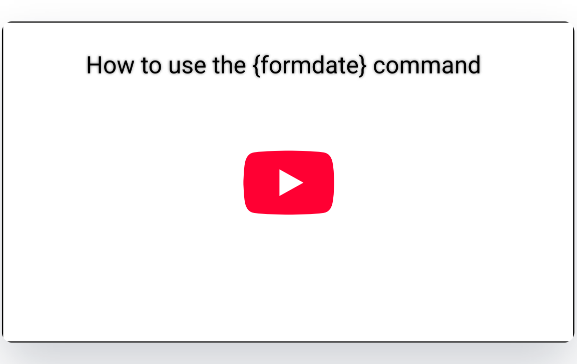

Sample Usage Copy to Text Blaze

Due date and time: LLL

Snippet preview

Due date and time: 07-02-2026 03:26

Quick Overview Video


How to use the {formdate} command

Settings

Setting Name	Type	Description
Positional	format	The format for the date/time.
start	date/time	The minimum selectable date/time.
end	date/time	The maximum selectable date/time.
cols	number	The width of the form field in columns of characters.
General Form Settings		
name	text	Name used for labels and data bindings.
default	–	The default value for the form field.
formatter	function	Formats the value of the field before snippet insertion.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

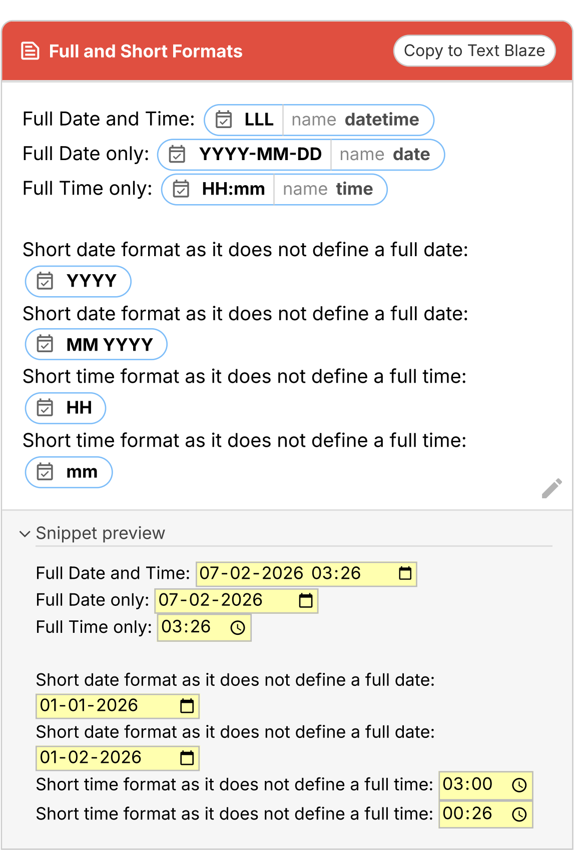
To learn more about form commands and their capabilities, see the forms guide.

Formatting Dates and Times

The `{formdate}` command requires a `format` setting that tells Text Blaze how to format the date/time when inserting. This format uses the same notation as the {time} command.

Click here to learn about time/date formatting in Text Blaze.

The `format` setting must contain enough information to define a full date or a full time. **Generally, the date needs to contain at least a day, month and year component to be full. Similarly, the time needs to contain hours and minutes to be full.** For this reason `MM DD YYYY` would be considered a full date format, whereas `YYYY` would be short as it omits the day and month. Similarly, `HH:mm` would be a full time format, while `HH` would be short as it lacks the minutes component.


Full and Short Formats Copy to Text Blaze

Full Date and Time: LLL name datetime
Full Date only: YYYY-MM-DD name date
Full Time only: HH:mm name time

Short date format as it does not define a full date: YYYY
Short date format as it does not define a full date: MM YYYY
Short time format as it does not define a full time: HH
Short time format as it does not define a full time: mm

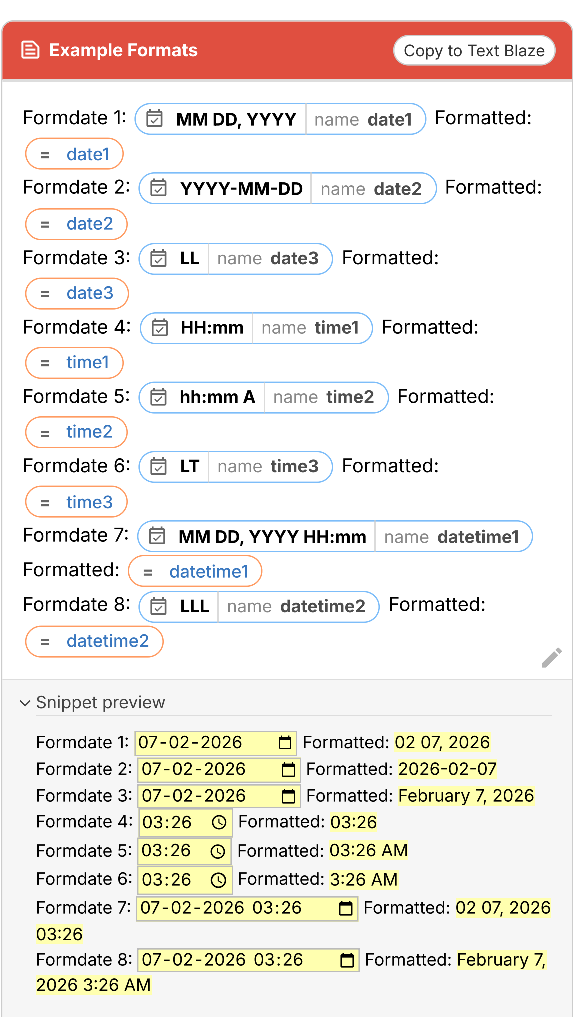
Snippet preview

Full Date and Time: 07-02-2026 03:26
Full Date only: 07-02-2026
Full Time only: 03:26

Short date format as it does not define a full date: 01-01-2026
Short date format as it does not define a full date: 01-02-2026
Short time format as it does not define a full time: 03:00
Short time format as it does not define a full time: 00:26

When entering a date/time via the `{formdate}` in the form popup, the date selector will use the same format that you define in the `{formdate}`'s positional setting.

Also, the same format will be used when using the date/time in a formula.


Example Formats Copy to Text Blaze

Formdate 1: MM DD, YYYY name date1 Formatted: = date1
Formdate 2: YYYY-MM-DD name date2 Formatted: = date2
Formdate 3: LL name date3 Formatted: = date3
Formdate 4: HH:mm name time1 Formatted: = time1
Formdate 5: hh:mm A name time2 Formatted: = time2
Formdate 6: LT name time3 Formatted: = time3
Formdate 7: MM DD, YYYY HH:mm name datetime1 Formatted: = datetime1
Formdate 8: LLL name datetime2 Formatted: = datetime2

Snippet preview

Formdate 1: 07-02-2026 Formatted: 02 07, 2026
Formdate 2: 07-02-2026 Formatted: 2026-02-07
Formdate 3: 07-02-2026 Formatted: February 7, 2026
Formdate 4: 03:26 Formatted: 03:26
Formdate 5: 03:26 Formatted: 03:26 AM
Formdate 6: 03:26 Formatted: 3:26 AM
Formdate 7: 07-02-2026 03:26 Formatted: 02 07, 2026 03:26
Formdate 8: 07-02-2026 03:26 Formatted: February 7, 2026 3:26 AM

> ### Try it out
>
> - Change the format of the `{formdate}` commands above and notice how the format of the output will also change.

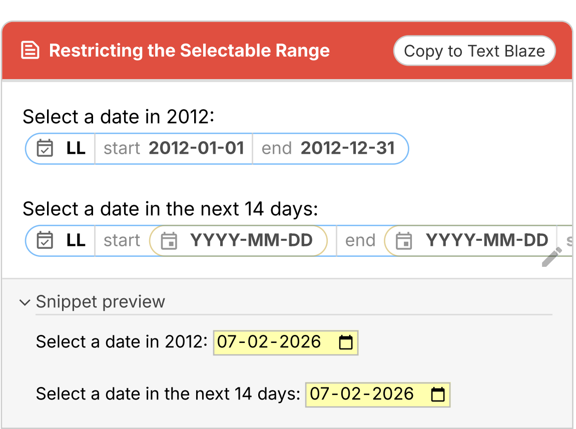
Restricting Selectable Dates and Times

You can optionally restrict the range of dates/times that can be selected using the `start` and `end` settings.

> In case of time, you will see an error notification instead of disabled values in the time picker.

The `start` and `end` settings are dates/times that **may be entered in the same format as the formdate's `format` setting**, or in a canonical `YYYY-MM-DD` or `HH:mm` format.

You can also embed {time} commands in order to create a dynamic range as seen in this example.


Restricting the Selectable Range Copy to Text Blaze

Select a date in 2012: LL start 2012-01-01 end 2012-12-31

Select a date in the next 14 days: LL start YYYY-MM-DD end YYYY-MM-DD

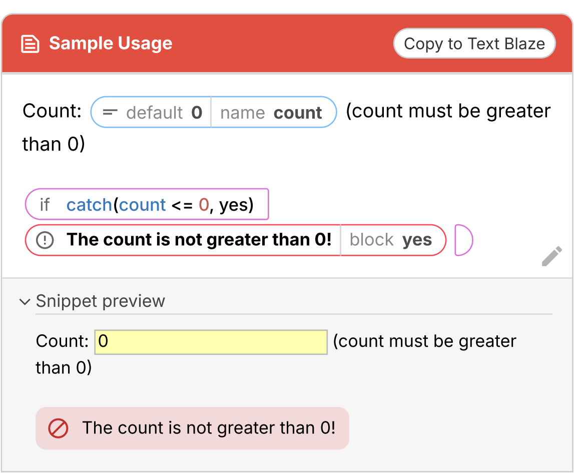
Snippet preview

Select a date in 2012: 07-02-2026

Select a date in the next 14 days: 07-02-2026

{error}

Summary: Show an error message in a snippet. Use it to validate user input and block snippet insertion.



Count: default 0 name count (count must be greater than 0)

if catch(count <= 0, yes)

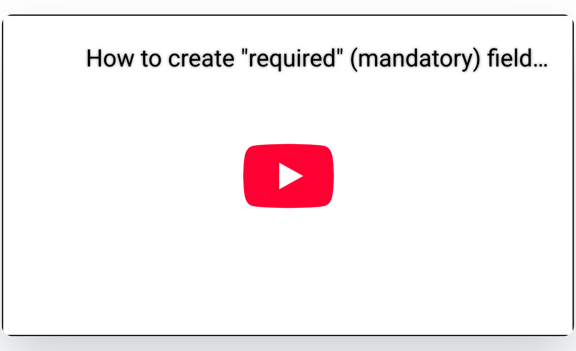
The count is not greater than 0! block yes

Snippet preview

Count: 0 (count must be greater than 0)

The count is not greater than 0!

Quick Overview Video



How to create "required" (mandatory) field...

Settings

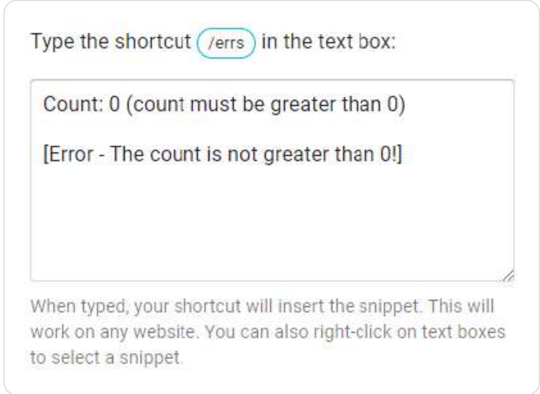
Setting Name	Type	Description
Positional	text	The error message to show to the user.
block	yes/no	If `yes`, blocks insertion while the error is shown.
show	default/validate	`validate` will show the error only after you attempt to insert the snippet.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `error` command can be used for form validation and for preventing a snippet from being inserted until certain conditions are met.

The `error` command is best used in combination with the {if} command to conditionally show an error and block snippet insertion.

The `block` setting

If the `block` setting is `no` the message that you used in the first positional setting will be inserted into your snippet.



Type the shortcut /errs in the text box:

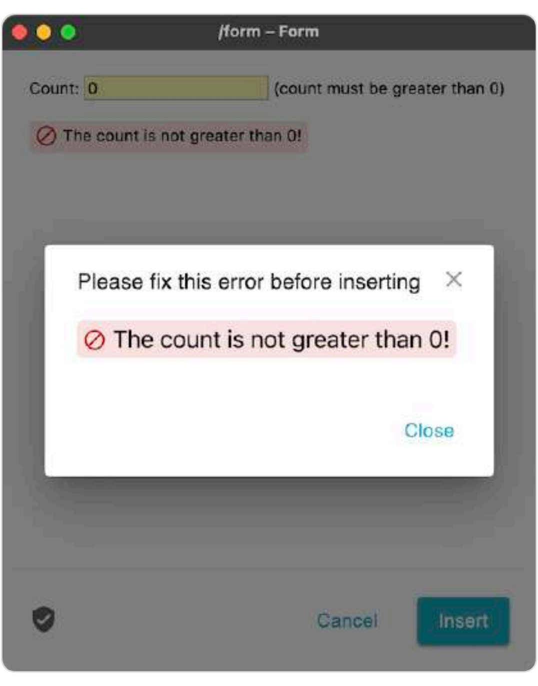
Count: 0 (count must be greater than 0)

[Error - The count is not greater than 0!]

When typed, your shortcut will insert the snippet. This will work on any website. You can also right-click on text boxes to select a snippet.

Notice how the error text is inserted into the snippet itself inside of the `[Error - ]` text.

If `block` is `yes`, the message will be displayed (not inserted) and **the snippet user will not be able to insert the snippet until the error is resolved**.



Count: 0 (count must be greater than 0)

The count is not greater than 0!

Please fix this error before inserting

The count is not greater than 0!

Text Blaze will not allow you to insert the snippet by showing an alert.

Error checking your team's snippets

A good use case for the `error` command is when you're sharing snippets with your team. It will ensure that everyone using the snippet is entering the right information and validating it correctly before the final insertion.

{cursor}

Summary: Moves the mouse cursor after the snippet has been entered.

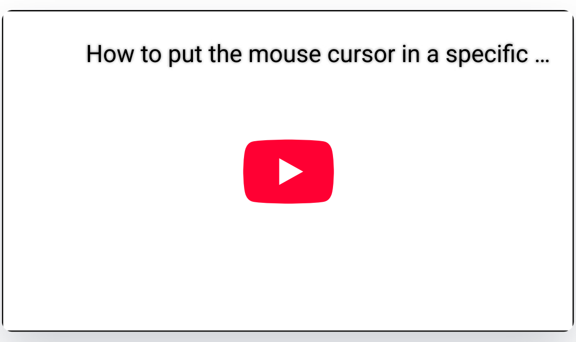
Sample Usage Copy to Text Blaze

Hi cursor ,
Thank you so much for connecting. Seems we're both very much into name **common interests** .
I look forward to some great conversations!

Snippet preview

Hi CURSOR ,
Thank you so much for connecting. Seems we're both very much into common interests .
I look forward to some great conversations!

Quick Overview Video



Settings

Setting Name	Type	Description
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

Only one `cursor` command is allowed in a snippet.

Text Blaze allows you to specify where the cursor should end up after the snippet insertion has completed. The mouse cursor is moved to the position of where the command is within the snippet.

Example

One useful thing to do with the cursor command is to place it either at the start or the end of a snippet. You can then combine it with the `{key: tab}` or `{key: shift-tab}` command to more easily navigate the page.

Example with cursor and key Copy to Text Blaze

cursor
Hi name **firstname** ,
Thank you for your message!
shift-tab **shift-tab** **enter**

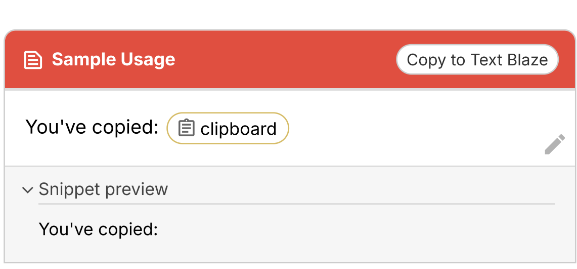
Snippet preview

CURSOR Hi firstname ,
Thank you for your message!
SHIFT-TAB SHIFT-TAB ENTER

This is an example where you might need to insert a message first and then do something else in the web page afterwards with autopilot commands.

{clipboard}

Summary: Inserts the current clipboard contents.



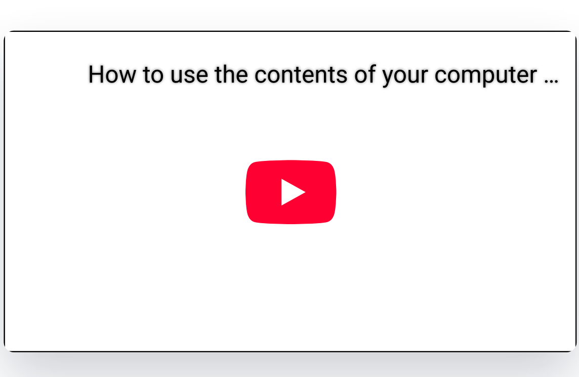
Sample Usage Copy to Text Blaze

You've copied: clipboard

Snippet preview

You've copied:

Quick Overview Video



How to use the contents of your computer ...

Settings

Setting Name	Type	Description
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `clipboard` command will insert the entire contents of your clipboard.

In many cases, you will only want to insert a part of the contents of your clipboard.

Referencing a part of the clipboard

A very common use case with the `clipboard` command is only referencing a specific part of the data in your clipboard.

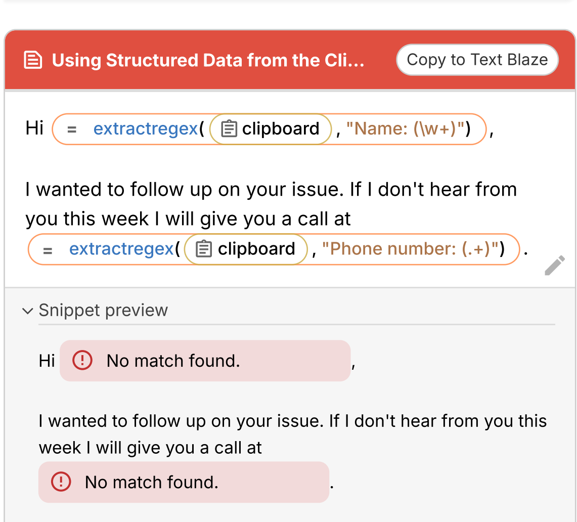
For example, you may have a customer case management system where data could be copied in the following format:

```
Name: John
Phone number: (555) 123-4567
Problem: User cannot connect to the website.
```

If you wanted to create a snippet that automatically used the name and phone number, you could do that using the `clipboard` command combined with the {=} (Formula Command) command to reference specific fields.

The following example shows this.

> To see this example work, please do the following:
>
> 1. Copy the text in the text box above
> 2. Reload this page so that the example uses your updated clipboard.
> 3. You will need to grant this website permission to access your clipboard.



Using Structured Data from the Cli... Copy to Text Blaze

Hi = extractregex(clipboard , "Name: (\w+)") ,

I wanted to follow up on your issue. If I don't hear from you this week I will give you a call at = extractregex(clipboard , "Phone number: (.+)") .

Snippet preview

Hi No match found. ,

I wanted to follow up on your issue. If I don't hear from you this week I will give you a call at No match found. .

The example above uses the `extractregex` function. This function uses a regular expression to precisely define what items of the clipboard you want to use. You can learn more about regular expressions in Text Blaze here.

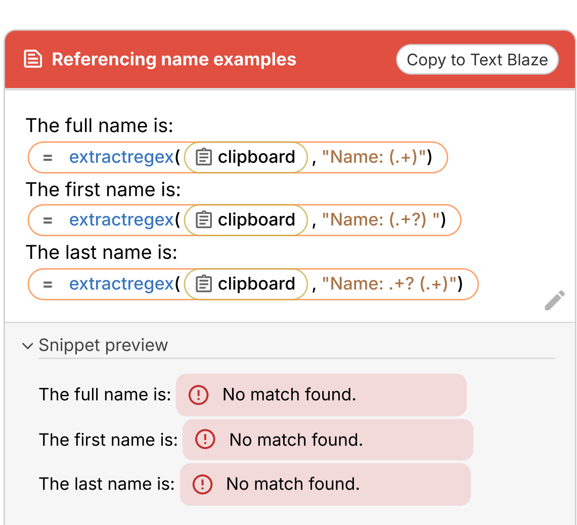
Referencing specific data in your clipboard with regular expressions

The following examples assume your computer's clipboard contains the following text:

```
Name: John Smith
Age: 32
ID: RFS4215
```

> If you have not yet copied the text and reloaded this page, please do so now.
>
> If you don't, you are going to see a lot of "No match found" errors in the following examples.

To reference the name from this text, we could use the following regular expressions:



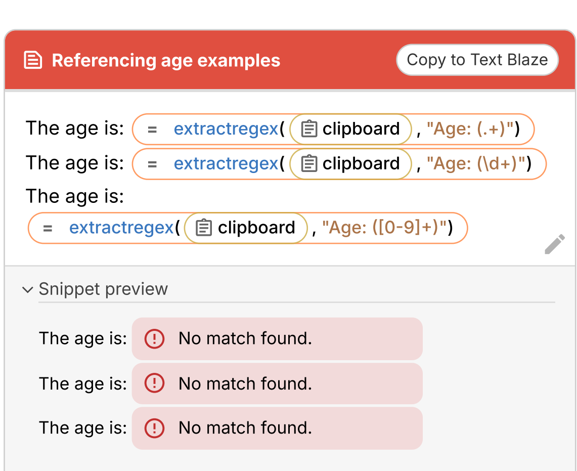
Referencing name examples Copy to Text Blaze

The full name is: = extractregex(clipboard , "Name: (.+)")
The first name is: = extractregex(clipboard , "Name: (.+?) ")
The last name is: = extractregex(clipboard , "Name: .+? (.+)")

Snippet preview

The full name is: No match found.
The first name is: No match found.
The last name is: No match found.

Now let's write a regular expression to get the user's age. Here we show a couple different approaches:



Referencing age examples Copy to Text Blaze

The age is: = extractregex(clipboard , "Age: (.+)")
The age is: = extractregex(clipboard , "Age: (\d+)")
The age is: = extractregex(clipboard , "Age: ([0-9]+)")

Snippet preview

The age is: No match found.
The age is: No match found.
The age is: No match found.

There are many ways to write a regular expression to grab a specific result that you're looking for.

> Diving in-depth into Regular Expressions is out of scope for this documentation page.
>
> To learn more, visit the section on Regular Expressions in the Blaze Formula Reference page.

Handling badly formatted data

Occasionally, you will try to use a regular expression on data that is incorrectly formatted so that the regular expression does not correctly match the data.

For example, imagine the following data was given to you:

```
Name:
Age: 32
ID: RFS4215
```

The data is missing the name. If you use the regular expression from the previous section to process this you'll get a "No match found" error.

Sometimes you want to explicitly want an error to occur, but other times you want to insert a fallback value when the regular expression match fails.

There are two basic ways to handle these cases:

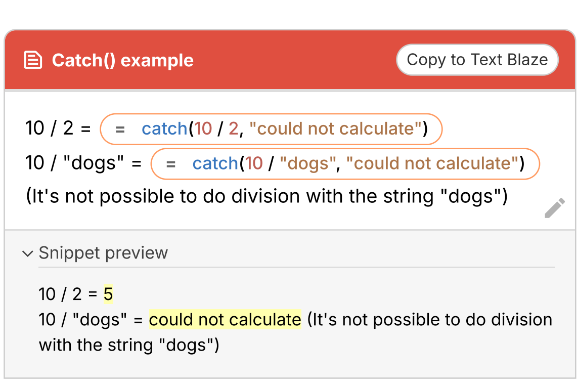
1. Using the `catch()` function.
2. Using the {if} command and the `testregex` function.

Handling errors with the `catch()` function

The `catch()` function evaluates its first argument and returns it.

However, if an error occurs when it is evaluating its first argument, it will return the second argument as a fallback.

Here is a quick example of the `catch()` function in action:



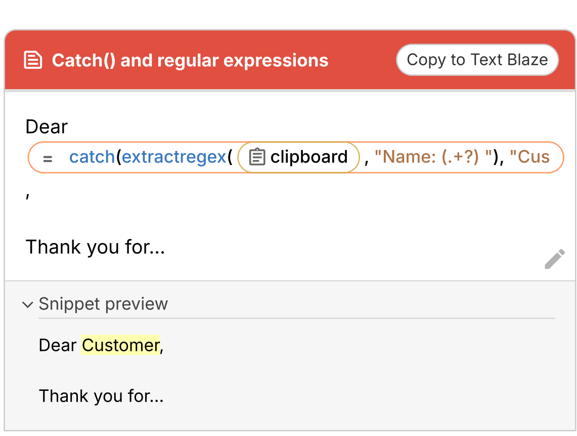
Catch() example Copy to Text Blaze

10 / 2 = = catch(10 / 2, "could not calculate")
10 / "dogs" = = catch(10 / "dogs", "could not calculate")
(It's not possible to do division with the string "dogs")

Snippet preview

10 / 2 = 5
10 / "dogs" = could not calculate (It's not possible to do division with the string "dogs")

Here's another example using the `catch()` function together with a regular expression:



Catch() and regular expressions Copy to Text Blaze

Dear
= catch(extractregex(clipboard , "Name: (.+?) "), "Cus
,

Thank you for...

Snippet preview

Dear Customer,

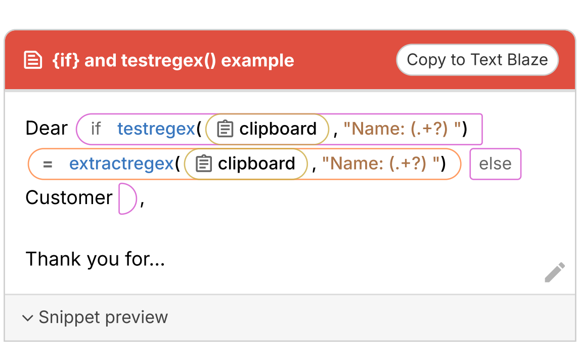
Thank you for...

If the contents of your clipboard contains the correctly formatted text "Name: John", for example, the snippet will output "Dear John".

If the regular epression above does not match with the contents of the clipboard, then there will be an error. In that case the `catch()` command's second argument will output. It will display "Dear Customer".

Handling errors with the `if` command and the `testregex()` function

The `testregex()` function returns a Boolean (yes/no) which can be used as a condition in the {if} command.



{if} and testregex() example Copy to Text Blaze

Dear if testregex(clipboard , "Name: (.+?) ")
= extractregex(clipboard , "Name: (.+?) ") else
Customer ,

Thank you for...

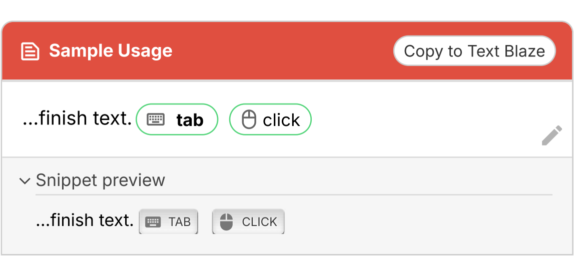
Snippet preview

That's a very short introduction to using regular expressions together with the `clipboard` command to only grab the data that you need.

To go more in-depth, see the Regular Expressions section of the Formula Reference page.

{click}

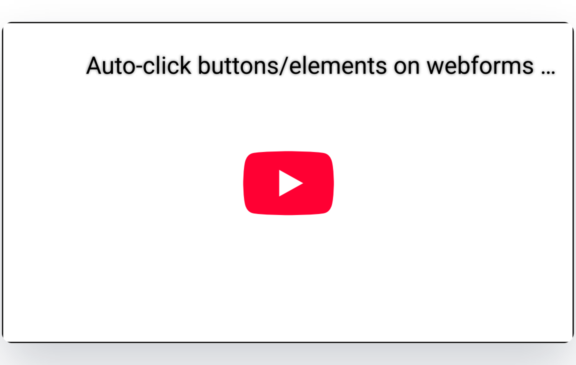
Summary: Simulates a click on any element on any open webpage.



...finish text. tab click

...finish text. TAB CLICK

Quick Overview Video



Settings

Setting Name	Type	Description
selector	selector	CSS selector to click on a specific element in the page.
maxdelay	time	The selected element is clicked as soon as it becomes clickable. If the selected element does not become clickable within 'maxdelay' time, the snippet stops and an error is shown.
frame	self/top	Controls which frame to start matching the given selector from (defaults to 'self').
page	string	When `page` is specified, the {click} command will switch to a tab whose URL matches the given `page` URL.
select	yes/ifneeded	Controls when to show the tab selector when `page` setting is specified. `yes` always shows it. `ifneeded` (default) shows it only when multiple tabs match the `page` pattern and the current page does not match.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{click}` command, an autopilot command, allows you to *emulate* a click on any element on any web page. This can be used to submit a form, focus another textbox or trigger other events in the page. You can use the `{click}` command in three approaches:

Click on the currently focused element

You can use the `{click}` command in conjunction with the {key} command, another autopilot command, to move the focus to another element in the page and then use the `{click}` command to click the element. Click here to learn more about moving the focus back and forth between different elements on a page.

For example: the snippet `{key: tab}{key: tab}{key: tab}{click}` will emulate the Tab key three times - changing the focused element on each step - and then emulate a click on the latest focused element.

Click on any element in the page

Use the "Select from website" button in the `{click}` command to select an element that you wish to click on. The Text Blaze snippet will click the selected element, which will then also become focused.

We recommend this approach for most users because it's very fast. It is useful to:

- Jump directly to another textbox in the page
  Example: running the snippet `Alpha{click: selector=selector}Beta` will insert "Alpha" in the first textbox (where you started your snippet), then jump to the second textbox (selected by "selector") and then insert "Beta" in the second textbox
- Click on consecutive checkboxes or buttons
  Example: When submitting a form, the snippet `{click: selector=button1}{click: selector=checkbox1}{click: selector=submit_button}` will click a button, then another checkbox, and then the submit button on the form.

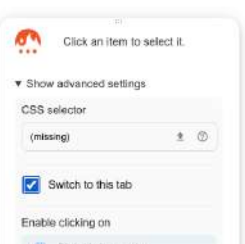
The given snippets are examples. Please use the "Select from website" button in the `{click}` command to generate a selector for your specific website.

Text Blaze will wait up to one second for an interactive element that matches the specified selector to appear on the page. If you want to wait longer, you can specify the `maxdelay` attribute. For example, the following command `{click: selector=selector; maxdelay=5s}` will wait up to five seconds to match the given selector to an element in the page.

Switch to another tab and then click an element in it

You can also use {click} to switch to another open tab and click an element there. Your snippet will continue running in that open tab. This lets you automate actions across multiple tabs with a single snippet.

To enable the cross-tab capability, check the "Switch to this tab" checkbox when setting up the {click} command's selector, under the "Show advanced settings" option, as shown in this image:



Click an item to select it.

▼ Show advanced settings

CSS selector: (missing)

☑ Switch to this tab

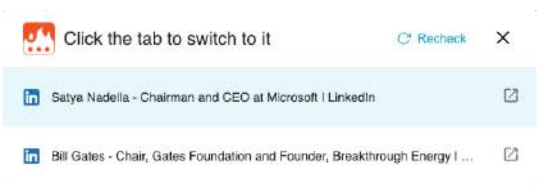
Enable clicking on
● Clickable items only
○ All items

For example, the following command switches to a tab matching `https://your-crm.com/*` and clicks the Submit button:

`{click: selector=.submit; page=https://your-crm.com/*; frame=top}`

When `page` setting is specified, the default behavior of {click} is as follows:

- If your current page matches the `page` pattern, the snippet will continue to run on the current page
- If only one other tab matches the `page` pattern, the snippet will switch to that tab automatically, and continue to run there
- If multiple tabs match the `page` pattern, you will see a tab selector dialog as shown in the image below:



Click the tab to switch to it

Satya Nadella - Chairman and CEO at Microsoft | LinkedIn

Bill Gates - Chair, Gates Foundation and Founder, Breakthrough Energy | ...

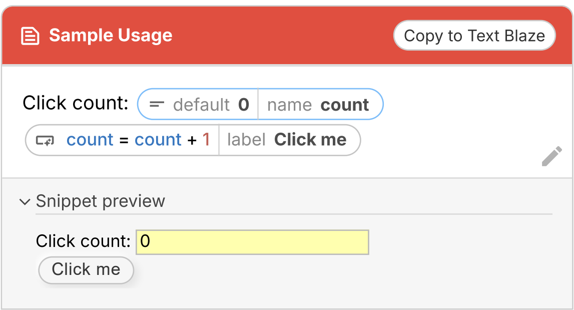
To always show the tab selector, set the `select` setting to 'yes' in your command, as shown in the example below:

`{click: selector=.submit; page=https://your-crm.com/*; frame=top; select=yes}`

To learn more about automating actions on a web page, see the **autopilot guide**.

{button}

Summary: Runs a code block when clicked.



Click count: default 0 name count
count = count + 1 label Click me

Click count: 0

Settings

Setting Name	Type	Description
Positional	code	The code block to run when the button is clicked.
label	text	The label for the button.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The `{button}` command adds an interactive button to your snippet. When clicked, the button runs a code block.

Buttons do not insert any content when the snippet is inserted. They are only for building interfaces in form snippets.

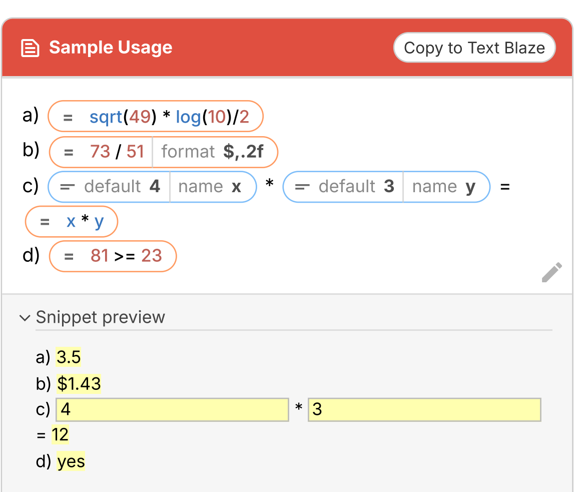
Buttons are often used with the {run} command which allows you to execute a code block once.

Learn more about code blocks in the code block reference.

> Button commands are in Beta and may change in the future.

{=} (Formula Command)

Summary: Dynamic updating formula that can use and create variables

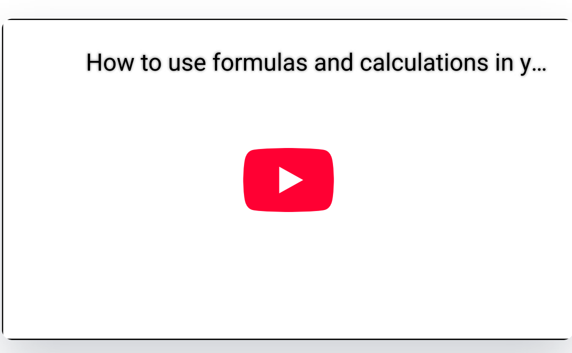


a) = sqrt(49) * log(10)/2
b) = 73 / 51 format $,.2f
c) = default 4 name x * = default 3 name y =
= x * y
d) = 81 >= 23

Snippet preview

a) 3.5
b) $1.43
c) 4 * 3
= 12
d) yes

Quick Overview Video



Settings

Setting Name	Type	Description
Positional	formula	The formula.
format	text	How to format a numeric output of the formula.
General Command Settings		
trim	yes/no/left/right	If `yes` whitespace is removed before and after the command. If `left`, only whitespace to the left is removed. If `right`, only whitespace to the right is removed.

The Formula Command is a special command in Text Blaze that allows you to use dynamic math, create variables, and utilize variables previously created throughout the snippet.

Also, notice how this command looks slightly different from other commands. It uses an equal sign between the `{}` instead of a word like the other commands.

Despite the difference in looks, the formula command has the same syntax as other commands. The first setting, a positional setting, takes a Text Blaze formula. The second optional setting, `format`, allows you to format numeric outputs.

Blaze Formula Reference

The {=} (Formula Command), {if}, and {repeat} commands, all utilize the **Blaze Formula Syntax** to let you use formulas and calculations as part of your snippets. There are features like predefined functions, lists, regular expressions, and more.

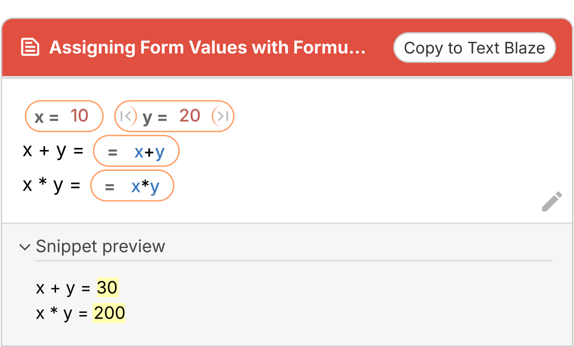
Visit the Blaze Formula Reference page and learn about what's possible.

Creating Variables With Assignment Mode

The formula command can also be used to create variables which can then be used in your other formulas within the same snippet.

To use it this way put the name of the variable before the formula command's `=` sign.

When used in assignment mode, the formula command won't display any output.



x = 10 y = 20
x + y = = x+y
x * y = = x*y

Snippet preview

x + y = 30
x * y = 200

To learn more about using variables in your snippets, visit the Blaze Formula Reference page.

It's also possible to use the {=} (Formula Command) command in combination with forms and form values.

Formatting numbers with the `format` setting

The optional `format` setting allows you to control how numeric outputs are formatted in formulas. The formatting will attempt to use the user's locale specified in the Text Blaze account settings.

The following described the available options:

```
[[fill]align][sign][symbol][0][width][,][.precision][type]
```

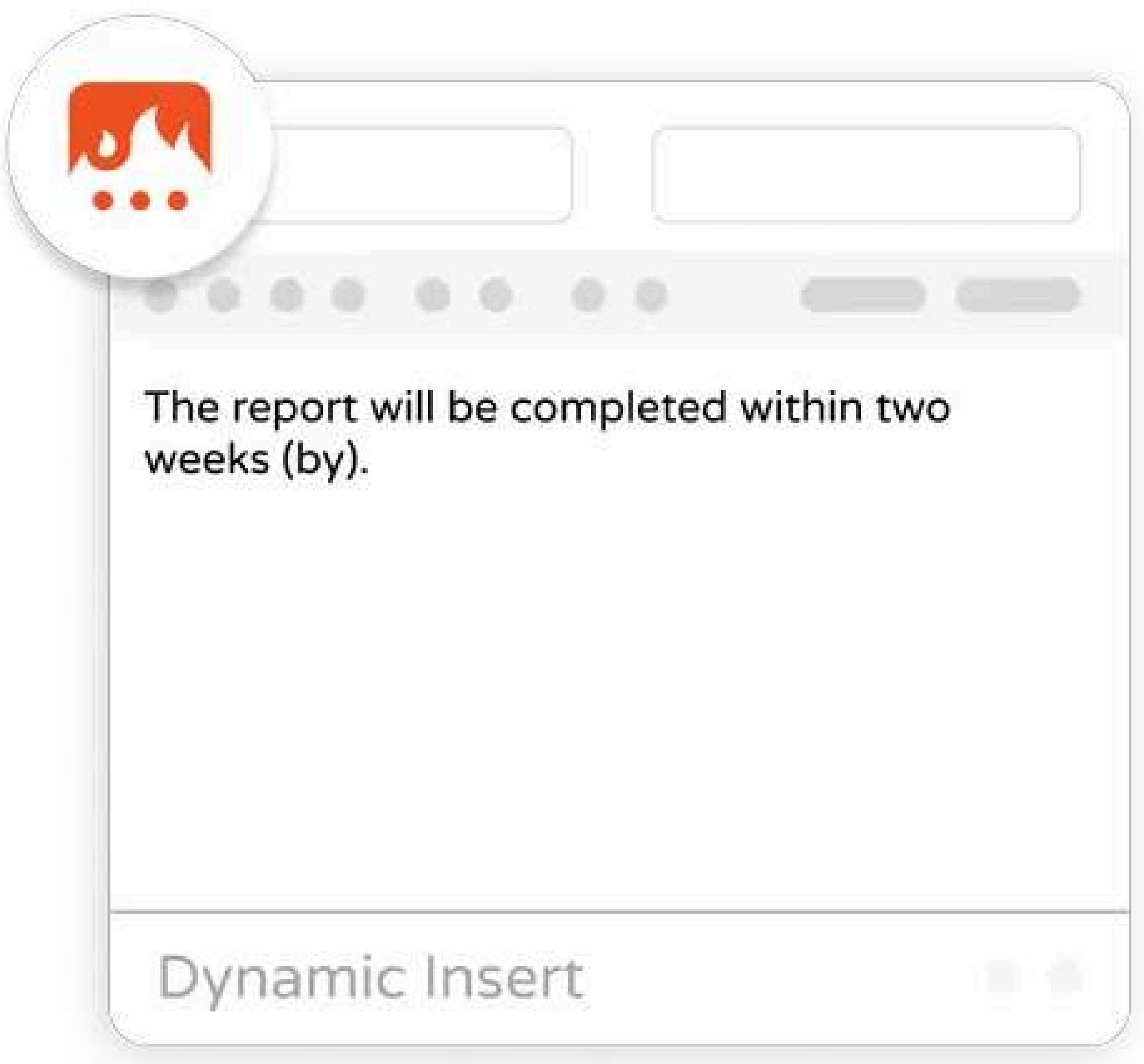
> **The output number formatting syntax and the blaze formula syntax are not the same.**
>
> More information and examples on output number formatting can be found here.

It is useful to see examples of format definitions and the resulting outputs. The following table shows how different formats would be applied to different sample numbers.

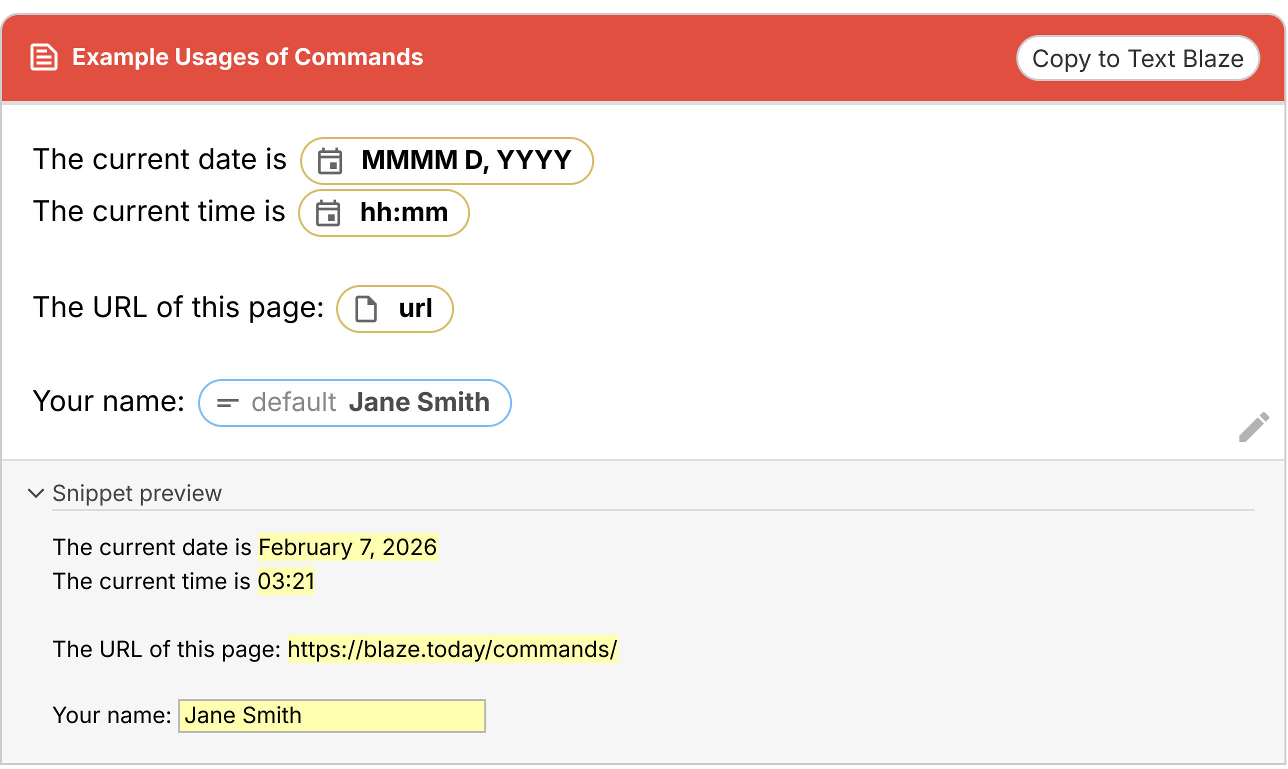
Format	Applied to `0.934`	Applied to `1245.67`
s	934.2m	1.24567k
,	0.9342	1,245.67
,%	93%	124,567%
+,	+0.9342	+1,245.67
.3r	0.934	1250
.3n	0.934	1.25e+3
.0f	1	1246
.1f	0.9	1245.7
.1e	9.3e-1	1.2e+3
$,.0f	$1	$1,246

Using Dynamic Commands

Text Blaze commands allow you to create dynamic snippets. Commands are replaced when the snippet is inserted with a dynamic value or action. For example, the {time} command lets you insert times and dates.


The report will be completed within two weeks (by).
Dynamic Insert

Here are a some command examples:



The current date is MMMM D, YYYY
The current time is hh:mm

The URL of this page: url

Your name: default Jane Smith

Snippet preview

The current date is February 7, 2026
The current time is 03:21

The URL of this page: https://blaze.today/commands/

Your name: Jane Smith

Commands are made up of two parts: the command name and the command settings which configure how the command behaves. A command and its settings are enclosed by a `{` and `}`.

For example {cursor} will move the mouse cursor after the snippet has been entered to the position where you entered the command.

Command Settings

Some commands also require settings to work correctly. There are two types of settings: **positional settings and named settings**.

For example, `{link: https://example.com/product/}Product{endlink}` inserts a link into the current snippet. The command is {link} and *"https://example.com/product/"* is a required positional setting telling Text Blaze which link to display. The text between the start command and the end command is how the link will display.

All settings are separated by semicolons (;). Positional settings come immediately after the colon following the name of the command as the first setting. They only require a value.

There can also be multiple named settings on a command. Named settings require you to insert the name of the setting followed by "=" and then the value.

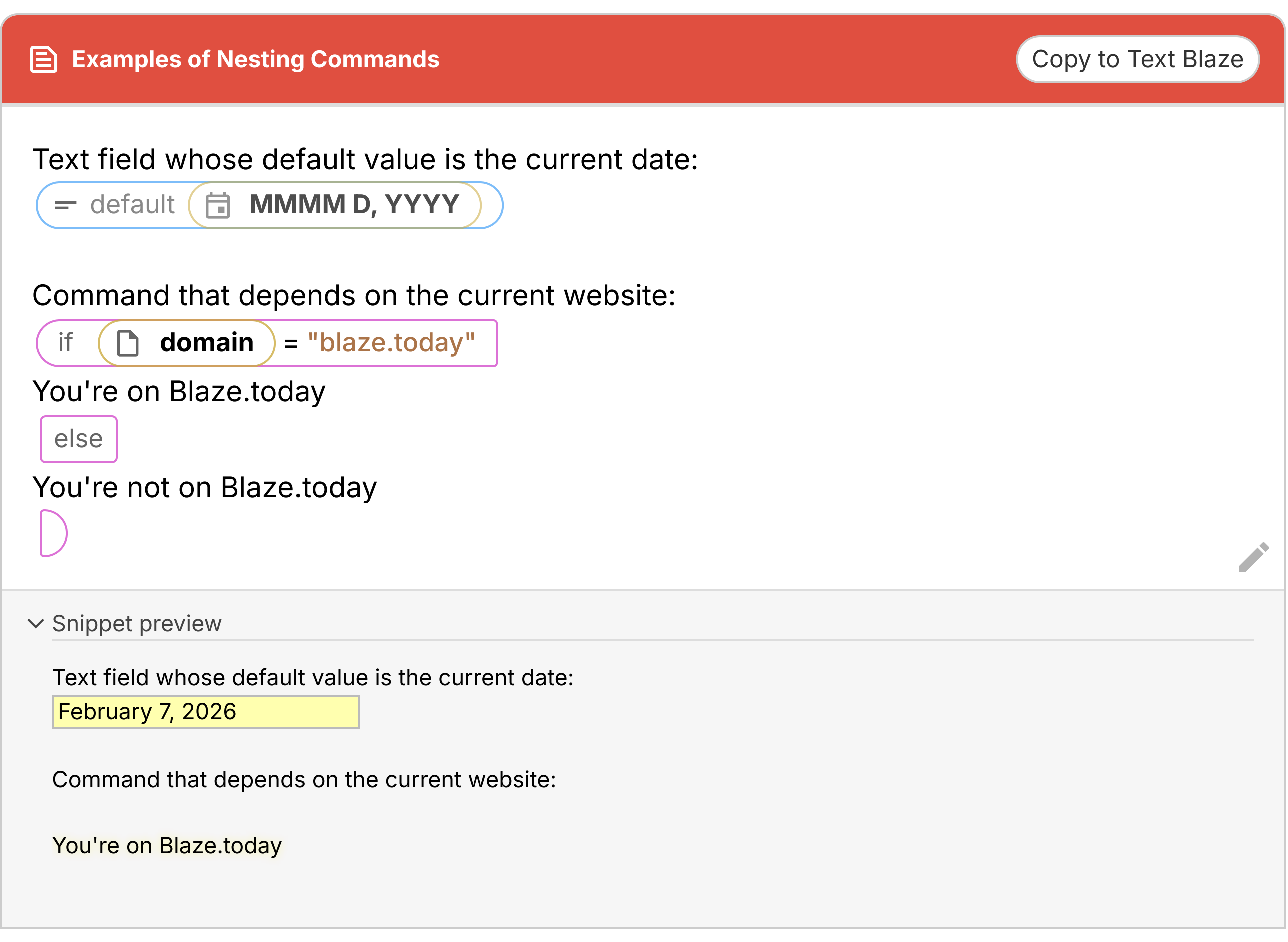
One example is `{time: YYYY; at=2000-01-01; shift=+3Y}` which displays the time. Here the command name is {time} and there are three settings. One is a positional setting and there are two named settings.

The first positional setting, `"YYYY"`, tells Text Blaze how to format the date ("YYYY" means show the current date's four digit year). The second named setting, `at`, is an optional named setting that allows you to explicitly set the date for the {time} command. If it's excluded it will use the current time. Finally, the last named setting, `shift`, tells Text Blaze to move the date forwards or backwards by some specified amount.

Commands can mix positional settings and named settings. There can also be multiple settings per command. Also, some settings are optional. See the documentation for each command to see which settings are optional and which are required.

Nesting Commands

An important feature of commands is that you can nest them within each other. For instance, you can use a {time} command as the default value for a {formtext} command or {site} command as part of the condition for an {if} command.



Text field whose default value is the current date:
default MMMM D, YYYY

Command that depends on the current website:
if domain = "blaze.today"
You're on Blaze.today
else
You're not on Blaze.today

Snippet preview

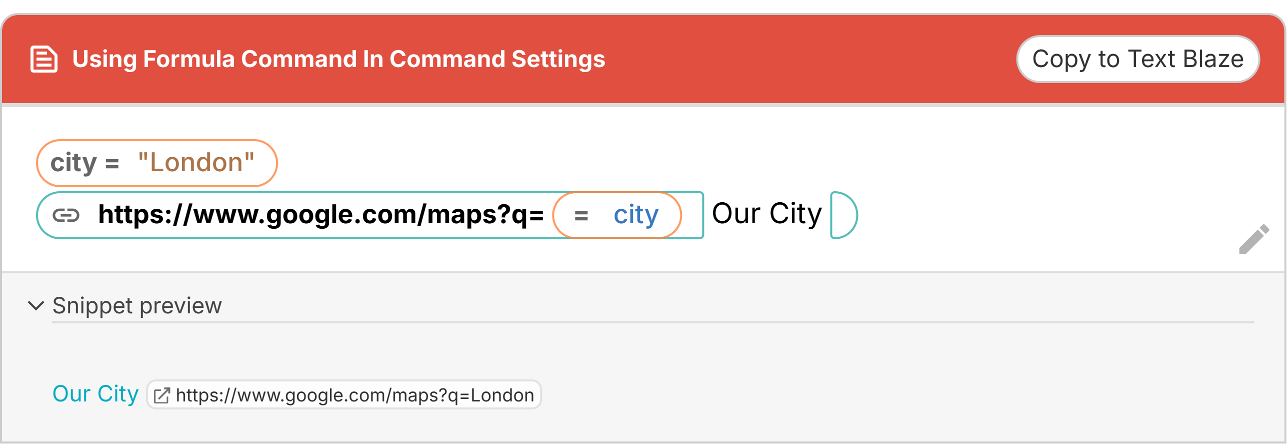
Text field whose default value is the current date:
February 7, 2026

Command that depends on the current website:

You're on Blaze.today

It's also possible for you to use the formula command in your commands for dynamic math or to utilize variables previously saved throughout the snippet.

For example, if you have a saved city or address in one your snippets you can utilize that to dynamically generate a Google Maps link.



city = "London"
https://www.google.com/maps?q= = city Our City

Snippet preview

Our City https://www.google.com/maps?q=London

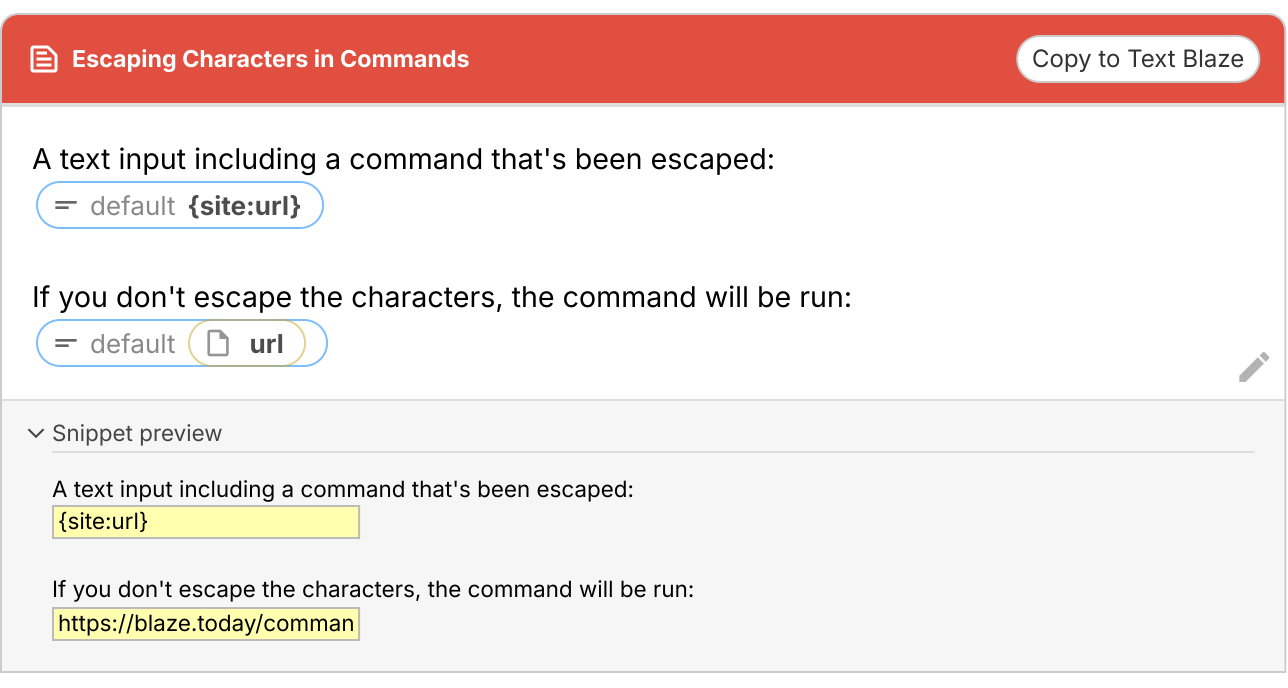
Nesting allows you to create powerful behavior be composing commands together.

Escaping Special Characters in Commands

Some characters have special meanings in commands. Primarily "{", "}", "=", and ";". If you want to use these special characters normally, you can do so by "escaping" them: by placing a backslash "" before them. For example this text input will have the default value "{site:url}":

```
{formtext: default=\{site:url\}}
```

If you didn't escape the `{` and `}`, you would get a different result as shown in the following example:



A text input including a command that's been escaped:
default {site:url}

If you don't escape the characters, the command will be run:
default url

Snippet preview

A text input including a command that's been escaped:
{site:url}

If you don't escape the characters, the command will be run:
https://blaze.today/comman

Non-escaped whitespace around a setting will be trimmed. If you want a setting value to start or end with whitespace, you need to escape the whitespace by placing a backslash in front of it.

Available Commands